

Master's Thesis

Automating Image-Based Sensitivity Analysis for GUI-Based Black-Box Simulation Software

Aleksei Lember

Supervised by:

Prof. Dr. Uwe Naumann

Software and Tools for Computational Engineering (STCE)

RWTH Aachen University

August 12, 2026

Abstract

High-fidelity numerical simulations are essential across modern engineering and scientific disciplines, yet a significant portion of industrial simulation software is distributed as closed-source binary executables encapsulated within Graphical User Interfaces (GUIs). This black-box software frequently lacks application programming interfaces (APIs) and command-line execution pathways, and its simulation output is predominantly accessible as color-coded spatial visualizations (e.g., heatmaps). This makes automated sensitivity analysis (SA) — which may require hundreds to thousands of model evaluations — infeasible through manual user interactions. This thesis addresses this gap by presenting the design, implementation, and evaluation of a non-intrusive software application that automates GUI-based SA for an arbitrary black-box simulator. The developed application operates purely external to the target simulation software: it records and replays the user’s keyboard and mouse actions required to perform a single simulation run, dynamically detects its completion (when the solver has finished its execution), and extracts the simulation output directly from the rendered color-coded images. From the obtained simulation input-output pairs, our application then computes local sensitivities (gradients) as well as global variance-based Sobol indices through integration with established SA libraries. Our developed software is validated in two stages: first on a custom synthetic benchmark with known sensitivities, then on multiple real-world industrial-grade simulators. Results show that our application reliably automates complex, multi-step GUI workflows and produces sensitivity estimates that closely match ground-truth values while remaining lightweight and resilient to various simulation software and/or operating system errors. After the evaluation, the identified advantages and problems of our application are summarized and concrete future work and research directions for extending its robustness, precision, and range of supported sensitivity analysis methods are proposed.

Keywords: simulation, black-box, sensitivity analysis, GUI automation, robotic process automation, image-based data extraction, Sobol indices, colormap

Contents

1	Introduction	3
1.1	Motivation and Context	3
1.2	Formal Problem Statement	4
1.3	Research Questions	5
1.4	Contributions	5
1.5	Thesis Structure	6
2	Background	8
2.1	Sensitivity Analysis Frameworks	8
2.1.1	Local Sensitivity Analysis and Finite Differences	8
2.1.2	Variance-Based Global Sensitivity Analysis	9
2.2	GUI Software Automation Paradigms	11
2.2.1	DOM-Based and Structural Automation	11
2.2.2	Agentic AI and Vision-Language Models	11
2.2.3	Programmatic Robotic Process Automation (RPA)	11
2.3	Related Work: Algorithmic Differentiation (AD)	12
3	Requirements and System Design	14
3.1	Functional Requirements	14
3.2	Non-Functional Requirements	16
3.3	System Architecture	17
3.3.1	Project	18
3.3.2	MainApp, SAViewer and EditableTreeview (The Views)	18
3.3.3	ActionsRecorder	18
3.3.4	ActionsReplayer	19
3.3.5	VisionEngine	19
3.3.6	Interaction Sequence	19
4	Implementation	24

4.1	Technology Stack	24
4.2	Project Creation and GUI Actions Recording	26
4.3	Workflow Replay	30
4.4	Simulation Completion Detection	35
4.5	Region of Interest Extraction	37
4.6	Colormap Inversion	37
4.7	Sensitivity Indices Calculation (SA Project Results)	39
4.7.1	Gradient	40
4.7.2	Sobol Indices	41
4.8	Project Persistence	41
4.9	Failure Recovery	44
5	Evaluation	46
5.1	Validation Framework	46
5.2	Synthetic Benchmark Analysis	46
5.2.1	Local Gradient Calculation Tests	51
5.2.2	Global Sobol Index Calculation Tests	55
5.3	Real-World Case Studies	60
5.3.1	ElmerFEM: Temperature Distribution	61
5.3.2	SimFlow: Internal Pipe Pressure	65
5.3.3	Ansys Electronics: Electromagnetic Brake (EMB)	73
5.4	Results	78
5.4.1	Advantages and Problems of Our Application	78
6	Conclusion and Future Work	83

Chapter 1

Introduction

1.1 Motivation and Context

In computational science and engineering, high-fidelity numerical simulation constitutes a critical backbone of modern industrial design and scientific research [1,2]. Sophisticated software packages deploy advanced numerical schemes to model complex physical phenomena. These computational tools enable virtual prototyping, thereby reducing the necessity for costly physical experimentation, mitigating the risks of structural failures, and accelerating the overall engineering lifecycle.

Despite their capabilities, a substantial fraction of simulation software is distributed as closed-source, pre-compiled binary executables. To facilitate user interaction, these systems are overwhelmingly encapsulated within Graphical User Interfaces (GUIs). While such GUIs provide an intuitive mechanism for a human operator to construct complex geometries, specify boundary conditions, and define material properties, they can introduce operational bottlenecks when researchers attempt to construct automated computational workflows.

Many simulation packages lack Application Programming Interfaces (APIs) or headless command-line execution pathways. Consequently, interacting with the underlying numerical solver logic frequently requires manual intervention by a human operator through the desktop GUI environment. Furthermore, while the raw, floating-point scalar matrices generated by the underlying numerical solver might be accessible, extracting them may not always (easily) be possible. Therefore, users frequently rely on the visual output, where the software renders the multidimensional data as color-coded spatial representations (e.g., heatmaps) [3–5].

A question of great interest may be to do Sensitivity Analysis (SA) on the input parameters of the simulation [6–8]. Traditional sensitivity analysis frameworks intrinsically assume that the underlying mathematical model can be seamlessly wrapped into an automated software loop, allowing for programmatic parametric sweeps¹. However, in the absence of source code access, executing a sensitivity analysis requires a practitioner to manually adjust input parameters within the GUI, trigger the computational solver, visually inspect the resulting color-coded image, and record the corresponding changes. This manual execution loop is labor-intensive and fundamentally unscalable, as variance-based global sensitivity studies often necessitate hundreds

¹A parametric sweep involves running a simulation multiple times across a defined range of input variable value combinations to observe the effect on outputs.

to thousands of model evaluations (simulation runs) [9].

Therefore, there is a practical need for a framework that can bridge this operational gap. We address this need by proposing an automated, non-intrusive (without the need for the source code or any other low-level access — not "intruding" there) software capable of orchestrating third-party GUI applications, executing automated parametric sweeps, reconstructing raw simulation output values directly from the color-coded images and performing the sensitivity analysis on this reconstructed data.

1.2 Formal Problem Statement

The computational challenge addressed in this thesis is formalized by modeling a desktop simulation software as a deterministic black-box function. Let the underlying physical simulation be represented by the mapping:

$$f : \mathcal{P} \times \Omega \rightarrow \mathbb{R} \quad (1.1)$$

where $\mathcal{P} \subset \mathbb{R}^N$ denotes an N -dimensional continuous parameter domain (box), and $\Omega \subset \mathbb{R}^2$ represents the two-dimensional spatial domain corresponding to the simulation's visual output canvas. For any given input parameter vector $\mathbf{p} \in \mathcal{P}$, the numerical solver computes a continuous physical scalar field $f_{\mathbf{p}}(x, y) = f(\mathbf{p}, x, y)$ over the spatial coordinates $(x, y) \in \Omega$.

Due to the closed nature of the compiled GUI binary, the exact numerical magnitude of the scalar field $f_{\mathbf{p}}(x, y)$ is often practically inaccessible to external automated scripts. Instead, the application's graphics pipeline renders a colored image representing the spatial field projected through an arbitrary, not-necessarily-linear colormap mapping $\mathcal{M}$:

$$\mathbf{C}(x, y) = \mathcal{M}(f_{\mathbf{p}}(x, y)) \quad (1.2)$$

where $\mathbf{C}(x, y) = [R, G, B]^T \in [0, 255]^3$ denotes the observed pixel color vector at the screen coordinates corresponding to (x, y) .

To perform an automated sensitivity analysis without possessing internal software access, we need to execute the three coupled tasks in a non-intrusive manner:

1. **Unsupervised GUI Control:** Programmatically interfacing with the host operating system and sending it keyboard and mouse events in order to navigate the simulator's GUI, inject perturbed input parameter values $\mathbf{p}_i \in \mathcal{P}$ into the corresponding input fields, trigger the simulation execution and dynamically detect the completion of the solver.
2. **Image Data Extraction and Scalar Reconstruction:** Capturing a predefined region of the color-coded simulation output image displayed on the screen and inverting the colormap mapping (computing $\mathcal{M}^{-1}$) to reconstruct the underlying scalars $\hat{f}_i$ from the observed RGB pixel values.
3. **Sensitivity Estimation:** Computing some sensitivity index or indices over the aggregated reconstructed input-output tuples $(\mathbf{p}_i, \hat{f}_i)$.

1.3 Research Questions

To systematically guide the design, implementation, and empirical validation of the proposed automated software, this research formulates four central research questions (RQs):

- **RQ1:** To what extent can the GUI-based sensitivity analysis be automated (by writing a dedicated software application)?

This question investigates the feasibility of designing a non-intrusive workflow capable of entirely bypassing the lack of APIs and source code access in legacy and/or proprietary engineering software. It focuses on whether an external orchestration engine can reliably simulate human inputs, dynamically track changing software states, and cleanly scale to manage large parametric loop configurations without encountering fatal deadlocks or requiring manual operational interventions.

- **RQ2:** How reliable and convenient can such software be for an end user?

Beyond pure functional feasibility, this question evaluates the overall user experience and stability of the engineered tool. It assesses the complexity involved in configuring a sensitivity analysis by the user and our software’s resilience against same operations (for example, a single simulation run) taking different time on a PC (due to indeterministic variations of CPU usage by other processes over time as well as different convergence speed of the simulation solver for different simulation input parameter values), against suddenly popped up notifications or other applications’ GUI windows, PC power outages, and so on.

- **RQ3:** How accurately can this software estimate the sensitivity indices compared to a precise analytical or numerical calculation?

This question addresses the mathematical validity of the data pipeline. Because our application relies on screen-level color extraction rather than native floating-point matrices, it is important to understand which errors occur at which stages of the sensitivity analysis and how they propagate between stages to the final sensitivity indices. The accuracy is assessed by comparing our software’s sensitivity indices output with exact analytical solutions on a benchmark with known sensitivities.

- **RQ4:** What are the limitations of such GUI-based sensitivity analysis automation software?

This final question explores the boundaries, problems, and operational constraints of our application. It analyzes how the proposed software behaves in the reality of complex modern feature- and dialog-window-rich GUI simulation software, of indeterminism introduced by other applications running on the same PC, etc. Finally, it examines the performance constraints when dealing with high-dimensional parameter spaces.

1.4 Contributions

To close the identified gap in the area of non-intrusive sensitivity analysis and address the research questions, this thesis presents the architectural design, implementation, and rigorous evaluation of

a specialized Minimum Viable Product (MVP) [10,11] application and introduces several practical contributions to the domain of computational tool integration and applied sensitivity analysis:

- **A Non-Intrusive Automation Engine for Scientific Software:** The thesis presents a robust operating-system-level recorder and replayer that functions by recording and replaying operating system keyboard and mouse events. This allows our application to automate operations with any GUI desktop simulator. In addition, our software includes a simple mechanism which handles the issue that a single simulation can take a variable amount of time.
- **Integrated Local and Global Sensitivity Analysis Modes:** Our software supports computation of local gradients via finite difference schemes (it also provides error bounds for the result which account for inaccuracies of the color-coded graphical representation of the underlying physical simulation scalar output field). Additionally, our software supports calculating global variance-based Sobol indices [9] by seamlessly integrating with external sensitivity analysis libraries.
- **Comprehensive Multi-Stage Empirical Validation:** The software implementation is validated through extensive experiments. Initial evaluations utilize a custom synthetic GUI benchmark (developed by us from scratch) simulating a highly non-linear function, the true sensitivity indices of which are known and serve as the ground truth to which we compare the sensitivity indices calculated by our SA automation application. Subsequent real-world case studies test our software’s applicability across diverse commercial and open-source simulators.

1.5 Thesis Structure

To answer all the formulated research questions, as we have already stated, we have developed and thoroughly tested a specialized software application (MVP). But before we go to its detailed description and discussion, it is first necessary to establish the theoretical and technological background governing sensitivity analysis and programmatic UI interactions. The remaining part of this Master’s Thesis Report is organized as follows:

Chapter 2 (Background) establishes the mathematical foundations of sensitivity analysis, reviews GUI automation paradigms and examines related work in the area of algorithmic differentiation.

Chapter 3 (Requirements and System Design) details the functional and non-functional requirements to our MVP which we have formulated prior to its development and presents its architectural design diagrams.

Chapter 4 (Implementation) provides a comprehensive breakdown of the software stack which we have selected for our MVP development and then details the implementation of the various modules of our application, including recording and replaying engines, simulation completion detection mechanism, colormap inversion, and so on.

Chapter 5 (Evaluation) presents the empirical results derived from both the synthetic Ishigami benchmark and the real-world case studies and discusses the advantages and problems of the developed MVP.

Finally, **Chapter 6 (Conclusion and Future Work)** summarizes what we have done in this Master's thesis and outlines prospective avenues for future research and development.

Chapter 2

Background

2.1 Sensitivity Analysis Frameworks

Sensitivity analysis systematically quantifies how the uncertainty or variation in the output of a mathematical model can be apportioned to different sources of uncertainty in its inputs [6]. It serves as a foundational tool for parameter calibration, uncertainty quantification, model simplification, and structural robustness assessment. Sensitivity analysis techniques are broadly classified into local and global methodologies [7], each possessing distinct mathematical properties, operational costs, and vulnerabilities to systemic noise.

2.1.1 Local Sensitivity Analysis and Finite Differences

Local sensitivity analysis evaluates the impact of small perturbations of input parameters around a specific point in the parameter space. For a continuous, differentiable scalar-valued simulation function $f(\mathbf{p})$, the local sensitivity with respect to the input parameter p_i evaluated at a parameter vector $\mathbf{p}_0$ is formally defined by the partial derivative:

$$\frac{\partial f(\mathbf{p}_0)}{\partial p_i} = \lim_{h_i \rightarrow 0} \frac{f(\mathbf{p}_0 + h_i \mathbf{e}_i) - f(\mathbf{p}_0)}{h_i} \quad (2.1)$$

where $\mathbf{e}_i$ denotes the i -th standard basis vector in the N -dimensional input space, and h_i represents a small scalar perturbation step size. The aggregation of these local sensitivities constitutes the gradient vector $\nabla f(\mathbf{p}_0)$.

In the context of black-box software lacking analytical derivatives, local sensitivities can be approximated using numerical finite difference schemes [12]. The forward difference formulation requires $N + 1$ function evaluations to calculate the gradient and incurs a first-order truncation error $\mathcal{O}(h_i)$:

$$\frac{\partial f(\mathbf{p}_0)}{\partial p_i} \approx \frac{f(\mathbf{p}_0 + h_i \mathbf{e}_i) - f(\mathbf{p}_0)}{h_i} \quad (2.2)$$

Conversely, the central difference formulation provides a higher-order approximation with an $\mathcal{O}(h_i^2)$ truncation error, though it doubles the computational cost to $2N$ independent simulation

evaluations to compute the gradient:

$$\frac{\partial f(\mathbf{p}_0)}{\partial p_i} \approx \frac{f(\mathbf{p}_0 + h_i \mathbf{e}_i) - f(\mathbf{p}_0 - h_i \mathbf{e}_i)}{2h_i} \quad (2.3)$$

2.1.2 Variance-Based Global Sensitivity Analysis

While local analysis provides insights into how small perturbations in each parameter change the model output at specific points, global sensitivity analysis tries to provide a general picture of how sensitive the model is to its parameters over its entire bounded parameter domain $\mathcal{P}$.

Variance-based methods, specifically Sobol indices, decompose the total unconditional variance of the model output $V(Y)$ into fractions attributable to individual inputs and their higher-order interactions [9]. Let $Y = f(\mathbf{p})$ where $\mathbf{p} = (p_1, p_2, \dots, p_N)$ are independent input variables. Without loss of generality, it is assumed that $\mathbf{p}$ is uniformly distributed on the unit hypercube $[0, 1]^N$, as any set of independent variables on a bounded, box-shaped domain can be mathematically transformed to this standard space.

The High-Dimensional Model Representation (HDMR) decomposes the function into summands of increasing dimensionality:

$$Y = f(\mathbf{p}) = f_0 + \sum_{i=1}^N f_i(p_i) + \sum_{1 \leq i < j \leq N} f_{ij}(p_i, p_j) + \dots + f_{1,2,\dots,N}(p_1, \dots, p_N) \quad (2.4)$$

Rather than being abstract components, these terms are explicitly constructed using conditional expectations to isolate the effects of the inputs:

$$f_0 = \mathbb{E}[Y] \quad (2.5)$$

$$f_i(p_i) = \mathbb{E}[Y|p_i] - f_0 \quad (2.6)$$

$$f_{ij}(p_i, p_j) = \mathbb{E}[Y|p_i, p_j] - f_i(p_i) - f_j(p_j) - f_0 \quad (2.7)$$

and so on.

Here, f_0 is the unconditional expected value of the output. The term $f_i(p_i)$ represents the main effect of parameter p_i , while $f_{ij}(p_i, p_j)$ represents the second-order interaction effect between p_i and p_j , explicitly excluding their individual main effects to avoid double-counting.

By construction, these summands are mutually orthogonal and (generally, except f_0) have zero mean. Specifically, a key property of this expansion is that integrating any component $f_{i_1 \dots i_s}$ (for $s \geq 1$) over any single variable $p_{i_k} \in \{p_{i_1}, \dots, p_{i_s}\}$ yields zero:

$$\int_0^1 f_{i_1 \dots i_s}(p_{i_1}, \dots, p_{i_s}) dp_{i_k} = 0 \quad \text{for any } k \in \{1, \dots, s\} \quad (2.8)$$

As a direct consequence of this condition, any two distinct components in the expansion are orthogonal over the hypercube $[0, 1]^N$:

$$\int_{[0,1]^N} f_u(\mathbf{p}_u) f_v(\mathbf{p}_v) d\mathbf{p} = 0 \quad \text{for } u \neq v \quad (2.9)$$

where $u, v \subseteq \{1, \dots, N\}$ represent subsets of parameter indices. This follows from Fubini's Theorem [13].

To derive the variance decomposition, we calculate the total variance $V(Y) = \mathbb{E}[Y^2] - (\mathbb{E}[Y])^2$. When the HDMR expansion is squared and integrated over the parameter domain, the orthogonality condition dictates that the expectation of all cross-product terms vanishes. Consequently, the total variance decomposes neatly into the sum of the variances of the individual orthogonal components:

$$V(Y) = \sum_{i=1}^N V_i + \sum_{1 \leq i < j \leq N} V_{ij} + \dots + V_{1,2,\dots,N} \quad (2.10)$$

where the partial variances are defined as:

$$V_i = \text{Var}(f_i(p_i)) = \text{Var}(\mathbb{E}_{\mathbf{p}_{\sim i}}[Y|p_i]) \quad (2.11)$$

$$V_{ij} = \text{Var}(f_{ij}(p_i, p_j)) = \text{Var}(\mathbb{E}_{\mathbf{p}_{\sim ij}}[Y|p_i, p_j]) - V_i - V_j \quad (2.12)$$

and so forth for higher-order interactions. Here $\mathbb{E}_{\mathbf{p}_{\sim i}}[Y|p_i]$ represents the expected value of the output Y taken over all parameters *except* p_i , while keeping p_i fixed.

From this decomposition, several crucial sensitivity indices are derived [6]:

- The **First-Order Sobol Index** (S_{1_i}) measures the isolated, main-effect contribution of the parameter p_i to the total variance:

$$S_{1_i} = \frac{V_i}{V(Y)} = \frac{V_{p_i}(\mathbb{E}_{\mathbf{p}_{\sim i}}[Y|p_i])}{V(Y)} \quad (2.13)$$

- The **Second-Order Sobol Index** ($S_{2_{ij}}$) quantifies the variance contribution specifically caused by the statistical interaction between p_i and p_j :

$$S_{2_{ij}} = \frac{V_{ij}}{V(Y)} = \frac{V_{p_i, p_j}(\mathbb{E}_{\mathbf{p}_{\sim ij}}[Y|p_i, p_j]) - V_i - V_j}{V(Y)} \quad (2.14)$$

- The **Total-Effect Sobol Index** (S_{T_i}) accounts for the main effect of p_i plus all higher-order interactions involving p_i :

$$S_{T_i} = 1 - \frac{V_{\mathbf{p}_{\sim i}}(\mathbb{E}_{p_i}[Y|\mathbf{p}_{\sim i}])}{V(Y)} \quad (2.15)$$

Since we cannot compute the true variance indices analytically for black-box simulation models, we approximate them by sampling the parameter space using Monte Carlo methods [14]. To optimize convergence, quasi-random sequences (like Saltelli sampling) are often utilized. A standard Saltelli sweep requires $N(D+2)$ simulation evaluations to compute the first-order and total-order indices, where N is the base sample size (the bigger it is, the tighter the confidence intervals of the computed indices are) and D is the dimensionality (number of parameters). If second-order indices are included, the formula expands to $N(2D+2)$ evaluations [14, 15]. This places requirements for a simulator to be fast and/or parallelizable because the number of simulation evaluations (runs) required may be huge (often in the thousands).

2.2 GUI Software Automation Paradigms

Automating interactions with desktop software relies on passing instructions from the automation software to the software under automation through the operating system. The trajectory of this field reveals a persistent tension between robustness to UI layout changes and execution speed.

2.2.1 DOM-Based and Structural Automation

For applications operating within web browser environments, automation frameworks interface directly with the Document Object Model (DOM). These tools locate buttons, forms, and canvases using structural selectors (e.g., XPath or CSS queries). Notable examples include Selenium [16], Puppeteer [17], and Playwright [18]. Because the interaction relies on the logical, hierarchical tree rather than absolute screen coordinates, structural automation is exceptionally resilient to dynamic layout changes or viewport resizing. However, desktop simulation software usually contains compiled graphics windows without any machine-friendly access to the button and input field structure, rendering these structural tools entirely inapplicable.

2.2.2 Agentic AI and Vision-Language Models

Recent advancements in Multimodal Large Language Models (M-LLMs) have spurred the development of autonomous Agentic AI that can operate GUIs [19–21]. These systems process screenshots of a GUI, infer the semantic purpose of UI elements, predict bounding box coordinates, and generate natural language reasoning to decide the next interaction step. While Agentic AI demonstrates remarkable capability in zero-shot, cross-application navigation, LLM inference introduces massive latency, often requiring several seconds to process a single visual frame and formulate a response. Over a standard Sobol sampling sweep requiring thousands of solver runs, semantic AI inference would add days of computational overhead. Furthermore, visual hallucination and imperfect bounding box estimation frequently lead to misclicks on dense, complex graphical user interfaces, which may corrupt or completely break down a parametric sweep.

2.2.3 Programmatic Robotic Process Automation (RPA)

To circumvent the limitations of DOM frameworks and the latency of Agentic AI, this thesis utilizes Low-Level Programmatic Robotic Process Automation [22, 23]. Our application interacts with the simulation software by communicating synthetic mouse click events, as well as keyboard key press and release events, to the operating system. This approach offers potential sub-second execution latency, absolute determinism, zero inference cost, and universal applicability across all graphical user interfaces. Because running many simulations with different parameters requires repetitive, pixel-perfect interactions to enter numeric parameter values into fixed input fields, programmatic RPA becomes the optimal choice. See Table 2.1 for a summary comparison of the RPA with the other two considered paradigms.

Table 2.1: Comparison of GUI Software Automation Paradigms

Feature	DOM-Based	Agentic AI (M-LLMs)	Programmatic RPA
Target Applications	Web Browsers	Any GUI	Any GUI
Execution Speed	Fast	Slow	Fast
Reliability	High	Medium	High
Setup Required	Finding specific DOM tags	Writing quality language prompts	Recording mouse and keyboard actions and coordinates of different GUI elements (buttons, input fields, etc.)
Additional Costs	Zero	Varies by M-LLM provider	Zero
Compatibility With Desktop GUI Simulation Software	Incompatible	Compatible	Compatible

2.3 Related Work: Algorithmic Differentiation (AD)

As we are dealing in this thesis with computation of sensitivities (including derivatives) of, basically, computer programs (the simulation function is represented by a computer program — simulation software), we would like to mention important related work in the computation of derivatives of computer programs (computation of derivatives of their outputs with respect to their inputs). Algorithmic Differentiation (AD) is a technique used to evaluate the exact mathematical derivatives of functions defined by computer programs [24]. Unlike numerical approximations such as finite differences, which are prone to truncation and rounding errors, AD computes the derivatives of computer programs exactly, up to machine precision. It achieves this by systematically applying the chain rule of differential calculus to the sequence of elementary operations (such as additions, multiplications, and basic mathematical functions) executed by the program’s code.

AD typically operates in two fundamental modes, each with distinct computational complexities and operational trade-offs:

- **Tangent-Linear (Forward) Mode:** In this mode, derivatives are computed and propagated forward in tandem with the original (primal) program execution. The computational cost of calculating a full gradient (for programs with multiple inputs and a single output) using the forward mode scales linearly with the number of input variables. Consequently, it is computationally efficient primarily for programs that have a small number of inputs but a large number of outputs. Its primary advantage is its minimal memory overhead, as it only requires storing the current state of the derivatives alongside the primal variables.
- **Adjoint (Reverse) Mode:** This mode computes derivatives in a two-step process. First, it executes the primal program and records every intermediate mathematical operation and variable state into a specialized data structure (the "tape"). Second, a reverse pass traverses this tape backwards, propagating derivatives from the outputs down to the inputs. The

computational cost of the reverse mode scales proportionally to the number of outputs rather than inputs. This makes it exceptionally powerful for computing gradients of computer programs representing scalar-valued functions (where there is only a single output), as it can compute the derivative with respect to an arbitrary number of inputs at a time cost that is only a small, constant multiple of a single primal program evaluation [24]. However, its significant drawback is the massive memory requirement needed to store the execution tape during the forward pass, which can become a bottleneck for lengthy or complex programs.

Advanced implementations of AD, such as the `dco/c++` library developed by the STCE group at RWTH Aachen University [25], often utilize techniques like custom classes and operator overloading to seamlessly replace standard data types with custom ones that record these operations automatically behind the scenes.

Despite its exactness and computational efficiency for large input spaces, AD is fundamentally limited by one strict requirement: it relies on direct semantic access to the simulation software’s source code. For the vast ecosystem of proprietary, closed-source industrial simulation software encapsulated in desktop GUIs, accessing, modifying or recompiling the source code is impossible. The non-intrusive application developed in this research provides a practical alternative, enabling sensitivity sweeps (including gradient calculation) in scenarios where the closed-source-code restrictions obstruct the use of AD.

Chapter 3

Requirements and System Design

The transition from the theoretical foundations of global sensitivity analysis and graphical user interface (GUI) automation to a functional, robust software requires a detailed elicitation of system requirements and a formalized architectural design to be done first (in accordance with the best practices of the Software Development Life Cycle [26]). This chapter delineates the complete functional and non-functional requirements [27, 28] to our MVP that were formulated prior to its development and outlines its modular object-oriented system architecture.

3.1 Functional Requirements

Functional requirements define the precise capabilities, behaviors, and operations our software must exhibit and support to successfully orchestrate a full sensitivity analysis cycle without exposing the user to programmatic scripting, internal application programming interfaces (APIs), or source-code modifications. The core functional requirements (FRs) to our sensitivity automation application are outlined below.

FR1: Workflow Recording and Replay Our software must be capable of tracking, recording, and persisting human interactions with the host operating system. When a person demonstrates the sequence of steps required to configure simulation parameters, run the numerical solver, and navigate to the post-processing results viewport, the application must hook into the hardware event stream to log exact mouse pointer coordinates, clicks, and precise keyboard button states (presses and releases). This recorded sequence of mouse and keyboard events forms the immutable operational baseline that our software will then iteratively repeat (replay) during the parametric sweep.

FR2: Simulation Parameter Value Injection During the Supervised Workflow Recording, our application must allow the user to designate specific physical interactions — such as clicking on a coordinate and typing a numerical value into a text field — as the "enter simulation parameter value" event. The user must be able to define continuous real-valued parameters, assign them unique names, and establish their allowable mathematical domain (minimum and maximum boundaries). During the automated parametric sweep, the our application must dynamically

sample values for these parameters according to the chosen configuration of the sensitivity analysis and subsequently inject the corresponding numerical values into the target GUI parameter input fields when required by the recorded workflow.

FR3: Asynchronous Simulation Finish Detection Our software must not rely exclusively on static, hard-coded temporal delays to determine when a computationally intensive simulation has finished evaluating. Because complex numerical solvers may exhibit variable execution times due to, for example, variable convergence speed for different simulation input parameter values or variable CPU load, our application must have a more advanced mechanism to dynamically determine the exact moment at which the simulation has ended: it should use some visual indicator for that (e.g., a window with the caption "Simulation Completed!" pops up or the solver's progress bar reaches 100%), just like a human would do. We call such a visual indicator a Simulation Completion Indicator (SCI). An absolute timeout mechanism must also be supported as a fallback to prevent infinite waiting for the SCI to appear on the screen in the event of some operating system and/or simulation software freeze or error.

FR4: Region of Interest and Supported Colormaps During the Supervised Workflow Recording, the user must be able to select a rectangular area of arbitrary size on the color-coded simulation output image as the "Region of Interest" (ROI). Our software must analyze the sensitivity of the average simulation function output value in that Region of Interest to the simulation input parameters specified by the user. Since the color-coded simulation output images can be rendered using various colormaps (e.g., "Viridis", "Magma", "Jet", "Turbo", "Gray", etc. [29]), our application must support multiple such popular colormaps.

FR5: Supported Sensitivity Indices Computation Our software must aggregate the Regions of Interest across multiple simulation runs in order to compute the sensitivity indices. Depending on the configuration set by the user, it must perform either a localized Gradient calculation or a global variance-based Sobol index calculation. Our software must autonomously generate the required sample points in the simulation input parameter domain according to the sensitivity analysis configuration set by the user, store the generated points together with their corresponding simulation function output values (when they become available after corresponding simulation runs) and calculate the final sensitivity indices (such as the first-order S_{1_i} and total-order S_{T_i} Sobol indices, or the multi-dimensional gradient vectors) upon the completion of all the required simulation runs.

FR6: Additional Region of Interest Preservation During the Supervised Workflow Recording, beyond selecting the primary ROI, which represents the target of the sensitivity analysis, our application must allow the user to select an "Additional ROI" (a rectangular area of arbitrary size, not necessary on the color-coded simulation output image, it can be anywhere on the screen), which will then be captured and stored as an image for each simulation run. This feature enables the preservation of the broader visual context (for example, not just the small ROI, but the whole simulation output image) in order to prevent the simulation output data loss

and to enable potential subsequent separate analysis of this extended data.

Table 3.1: Summary of Core Functional Requirements

ID	Short Name	Summarized Description
FR1	Workflow Recording and Replay	Record, serialize, deserialize and replay human GUI interactions.
FR2	Parameterization	Sample and inject different parameter values into the corresponding simulation software’s GUI input fields each new simulation run.
FR3	Simulation Completion Detection	Detect solver completion relying not on a fixed pre-specified time delay, but on a visual indicator, just like a human would do.
FR4	Simulation Result Extraction	Extract user-specified Region of Interest from the color-coded simulation result image while supporting multiple colormaps for that image.
FR5	Sensitivity Analysis Indices Calculation	Support calculation of gradients and Sobol indices.
FR6	Additional Data Preservation	Be able to capture and save supplementary visual data per simulation run.

3.2 Non-Functional Requirements

While functional requirements dictate the explicit actions the software must (be able to) perform, non-functional requirements (NFR) constrain *how* it performs these operations, necessitating its specific architectural characteristics, resilience, execution speed, and mathematical precision.

NFR1: Non-Intrusive Interoperability Our application must operate entirely externally to the target simulation software. It must not require the injection of dynamic-link libraries (DLLs), the parsing of proprietary application memory states, the reading of raw internal intermediate and/or output files, or the existence of a native scripting API. By interacting exclusively through the peripheral hardware (synthetic mouse and keyboard events) and the operating system’s display buffer, our application guarantees universal applicability. This allows it to wrap any GUI-based simulation suite, including highly restrictive proprietary tools and legacy simulators that lack modern programmatic interfaces.

NFR2: Avoidance of Agentic AI Automation Suites While modern Multimodal Large Language Models (M-LLMs) and Agentic AI frameworks can theoretically navigate GUIs by semantically analyzing screenshots and inferring bounding boxes, they must be strictly excluded from the execution loop. Agentic AI automation is inherently slow (often requiring several seconds of inference latency per action), computationally costly, and non-deterministic. In a high-throughput parametric sweep requiring hundreds to thousands of deterministic evaluations, an AI agent hallucinating a button click or misinterpreting an input field would irreparably corrupt the entire process. Therefore, the automation must rely on deterministic, low-level programmatic Robotic Process Automation (RPA).

NFR3: High-Throughput Execution and Minimal Resources Consumption Sensitivity analysis is fundamentally expensive in terms of computation (and, sometimes, memory). Our sensitivity analysis automation software must be highly optimized to prevent excessive CPU and memory consumption, ensuring that the host workstation’s compute resources remain entirely dedicated to the numerical solver of the simulation software. Our application must introduce virtually zero overhead to the underlying simulation execution cycle. For example, the latency between recognizing the simulation finish and executing the next action to extract the Region of Interest must occur on the order of milliseconds.

NFR4: Environmental Resilience and State Preservation Desktop graphical environments are inherently volatile ecosystems. Sudden operating system notifications, focus stealing by background applications, or temporary UI unresponsiveness due to high RAM utilization can easily disrupt a naive automated sequence. Our application must provide robust environmental resilience by regularly persisting the current state of the sensitivity analysis it is performing (including the runs it has already completed). If the execution pipeline is interrupted, no previously computed simulation data should be lost. Our software must be capable of reconstructing the last state of the sensitivity analysis it was performing before it was interrupted and resuming the parametric sweep exactly from the last successful iteration.

Table 3.2: Summary of Core Non-Functional Requirements

ID	Short Name	Short Description
NFR1	Interoperability	100% External / Black-box interaction only in order to ensure compatibility with (almost) every GUI simulator.
NFR2	Determinism and Exactness	Exclude stochastic imperfect AI/LLM inference in order to prevent sensitivity analysis process corruption during long parametric sweeps.
NFR3	Performance and Light-Weightness	The proposed sensitivity analysis software should be highly-optimized in terms of CPU and memory usage in order to minimize computational overhead and give maximum computing resources to the simulation solver. The software should not introduce additional time delays when executing recorded user actions.
NFR4	Resilience	Our application should save the current sensitivity analysis progress regularly in order to prevent data loss in the event of OS or simulation software errors and should be able to restore the saved state and continue the sensitivity analysis from the point where it stopped before the error had occurred.

3.3 System Architecture

The next step in the Software Development Life Cycle after the requirements have been formulated is performing the design of the software architecture. We did it adhering to a modular, object-oriented approach [30,31]. The architecture we designed separates the logic responsible for the

sensitivity analysis project creation, its update and persistence, user actions recording, their replay with parameter value injection, simulation completion detection, Region of Interest and Additional Region of Interest capture, calculation of the sensitivity indices and the visual sensitivity analysis results presentation. This separation of concerns ensures that individual components can be optimized or swapped without destabilizing the entire application.

Our SA automation software’s architecture is fundamentally composed of the following core classes and components:

3.3.1 Project

The **Project** class is the central sensitivity analysis project state management entity and acts as the singular source of truth for our application. It encapsulates all the metadata required to define and execute a single sensitivity analysis. This includes parameters defined by the user and their corresponding domains, chosen sensitivity index (gradient or Sobol), colormap used by the simulation software, bounding box dimensions of the primary ROI and the Additional ROI and possibly some other required metadata. The **Project** class manages the dataset of $(\mathbf{p}_i, \hat{f}_i)$ simulation input-output tuples (it extends this dataset as the parametric sweep progresses). The **Project** class is also responsible for persisting all the above-mentioned data and restoring it.

3.3.2 MainApp, SAViewer and EditableTreeview (The Views)

The **MainApp** class is responsible for building and orchestrating all the GUI windows of our application and enabling the user to interact with it. The **MainApp** dynamically surfaces context-specific dialogue windows for configuring different stages of a single sensitivity analysis project: creating a new project, specifying simulation input parameters and their corresponding domains, configuring the sensitivity analysis itself (for example, whether or not to calculate the S_2 Sobol index in addition to S_1 and S_T), selecting the Region of Interest, and others.

The **SAViewer** is a specialized interface module dedicated exclusively to the analytical post-processing phase. Once a sensitivity analysis project completes its parametric sweep, the **SAViewer** queries the **Project**, retrieves the aggregated data, and utilizes embedded visualization libraries to render interactive graphical plots — such as bar charts for gradient components and heatmaps for second-order Sobol index.

The **EditableTreeview** is a custom-built GUI component that is responsible for creating and managing intuitive, functional spreadsheet-like tabular displays. Such displays are heavily utilized during the sensitivity analysis configuration phase, allowing the user to seamlessly select cells, input floating-point parameter boundaries, delete parameters, resize columns by dragging and scroll the table both vertically and horizontally (so that the user can manage arrays of simulation input parameters of different size in the same simple manner).

3.3.3 ActionsRecorder

The **ActionsRecorder** class provides the low-level interception and recording of human mouse and keyboard actions. Utilizing global OS-level hardware hooks, this class monitors peripheral devices

events in the background. When our application enters the recording phase, the **ActionsRecorder** records every user action in a chronological order. It logs the exact Cartesian coordinates of mouse clicks and the exact sequence of keyboard scan codes. To ensure that the subsequent replay phase mimics the exact physical pacing of the human operator, the **ActionsRecorder** remembers not just the consecutive actions themselves, but also the time delays between them.

3.3.4 ActionsReplayer

The **ActionsReplayer** class is the "inverse" of the **ActionsRecorder**. During the parametric sweep, **ActionsReplayer** consumes the user action logs generated by the **ActionsRecorder** and translates them back into synthetic mouse and keyboard events, which it then sends to the operating system, effectively mimicking (accurately repeating) the corresponding recorded human actions. When it encounters a specific action within the log (like the one for entering the simulation input parameter value), it halts the raw playback, queries the **Project** for the specific sampled parameter vector $\mathbf{p}_i$ corresponding to the current simulation run and injects its component (the individual parameter value) into the corresponding simulation software's input field.

3.3.5 VisionEngine

The **VisionEngine** class is solely responsible for all the interactions and operations involving what is currently visible on the computer screen. It is a crucial part of such operations as the Region of Interest and the Additional Region of Interest capture, as it extracts the exact raw pixel matrices directly from the OS graphical backend, reconstructs the scalar simulation function output value $\hat{f}$ for every captured pixel (by performing the colormap inversion $\mathcal{M}^{-1}$) and then averages the reconstructed scalars, getting the $\hat{\bar{f}}$ — the average (reconstructed) simulation function output value in the Region of Interest, sensitivity of which to the simulation input parameters we are analyzing (as stated in the **FR4**). Another important responsibility of the **VisualEngine** module is detecting the simulation completion by repeatedly checking whether the SCI appeared on the screen after the simulation has started.

3.3.6 Interaction Sequence

The interactions between these classes and the user can be best understood through the two primary chronological phases of our application: the **Recording Loop** and the **Execution Loop**.

In the **Recording Loop**, the human operator initializes a new **Project** via the **MainApp**. The **MainApp** launches the **ActionsRecorder**, which binds to the global OS input streams. As the user physically clicks through the target simulation software, configuring a baseline run, the actions are continuously logged. If the user triggers the designated hardware interrupt key (e.g., the "Home" key), the **ActionsRecorder** suspends logging and **MainApp** surfaces a Configuration Menu window. Here, the user may invoke configuration windows for the different stages of the sensitivity analysis workflow: adding and editing simulation input parameters, selecting colormap, specifying ROI coordinates, configuring the sensitivity index calculation, setting up SCI, etc. All

the settings user tweaks (but not the individual mouse and keyboard actions) are recorded into the project metadata by the **Project** module and the corresponding commands are added to the actions log by **ActionsRecorder** (an example of such command may be "capture the region of interest" or "wait for simulation to finish with timeout 10"). Once the user records all the actions needed to perform a single simulation run: enter the parameter values, run the solver with those parameter values, wait for the SCI to appear on the screen, capture the output image patch (ROI) and, finally, navigate back to the beginning (where the user was in the simulation software before she recorded her first action); and once she configures everything else that is required to carry out the sensitivity analysis (e.g., specifies the perturbation step size for each parameter in the gradient calculation, selects colormap, etc.) and generates the full set of points $\mathbf{p}_i$ in the parameter space, evaluating the simulation function at which is required for the calculation of the chosen sensitivity index (the points generation is done by our application once the SA is fully configured, the user would just need to click the corresponding button), the **ActionsRecorder** module persists the final version of the actions log and the **Project** module persists the final version of the project metadata (including the generated points in the parameter space), making the user completely ready to perform the fully-automated sensitivity analysis from now on.

In the **Execution Loop**, **ActionsReplayer** module reads the user actions log saved by the **ActionsRecorder** module, the project metadata saved by the **Project** module and checks if there are still points $\mathbf{p}_i$ for which the simulation result $\hat{f}_i$ has not yet been captured. If there are such points, it picks the first of them. For the picked point **ActionsReplayer** replays all the recorded user actions: it interacts with the simulation software's GUI just like the user did it during the **Recording Loop**. **ActionsRecorder** performs the same mouse clicks and keyboard button presses to navigate the simulator's GUI, injects the simulation input parameter values selected for this simulation run ($\mathbf{p}_i$) into the corresponding input fields, triggers the numerical solver to run the simulation, waits for it to finish (for the SCI to appear), navigates to the window where the simulation output (the color-coded image) is located, uses **VisionEngine** to capture the ROI (and, if user specified it as well, the Additional ROI), reconstruct the scalars from it, average them and save (by calling the **Project** module), and, finally, navigates back to the beginning, becoming ready to do the same sequence of actions for the next parameter vector $\mathbf{p}_{i+1}$. This loop (parametric sweep) continues until the simulation result $\hat{f}_i$ is captured and stored for every parameter vector $\mathbf{p}_i$. Once this is done, the sensitivity analysis project becomes completed and the user can open it and view its comprehensive results in tabular and visual form (we can call it the third and last chronological phase — the **Viewing SA Results** phase). The results will be rendered from the stored dataset of $(\mathbf{p}_i, \hat{f}_i)$ tuples on the fly (since it is a quick process), potentially using some third-party libraries, for example, for the Sobol indices calculation.

The complete interaction sequence of the user with all the modules of our application for a single sensitivity analysis project is shown in Figures 3.1, 3.2, 3.3.

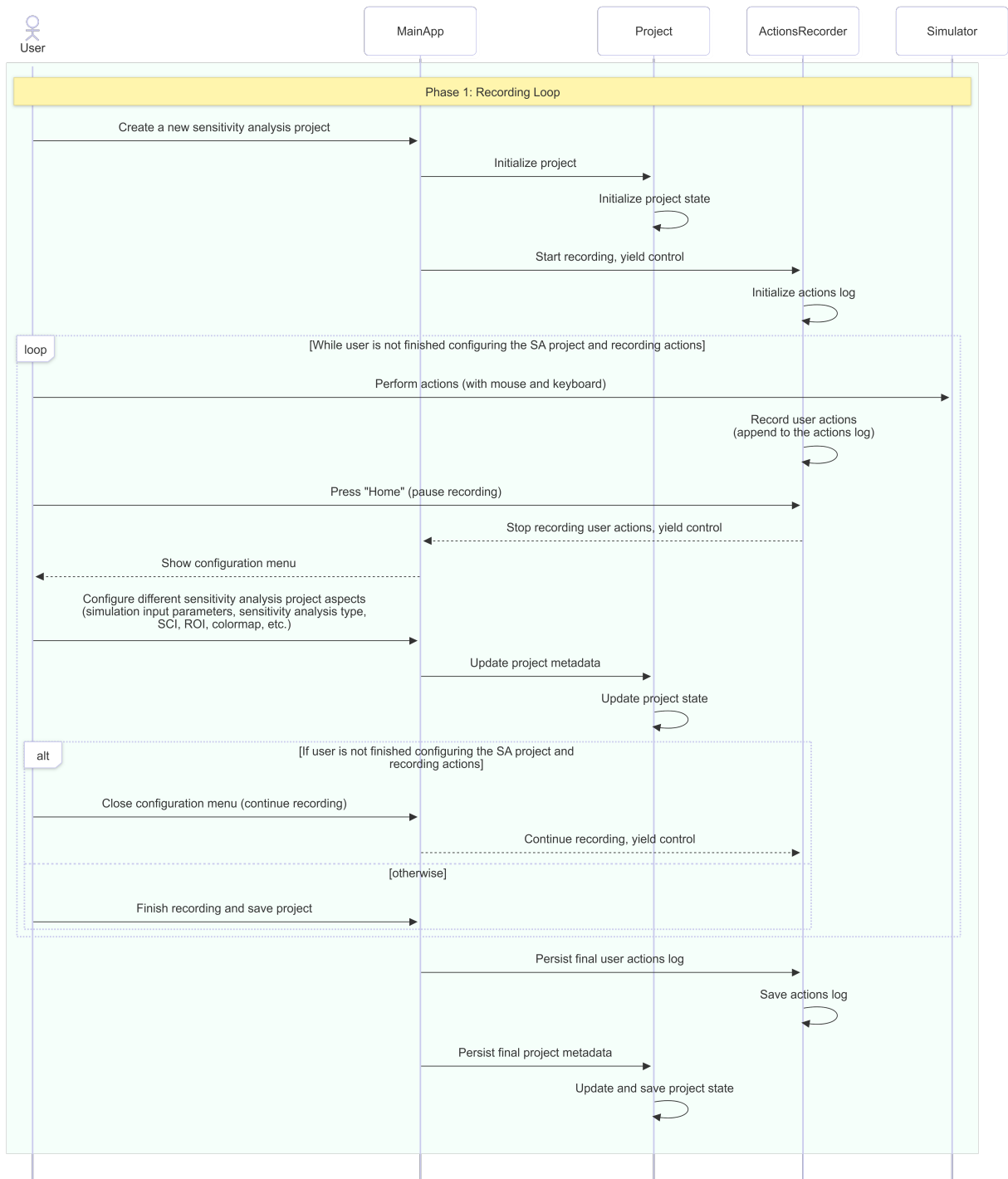


Figure 3.1: Recording Loop

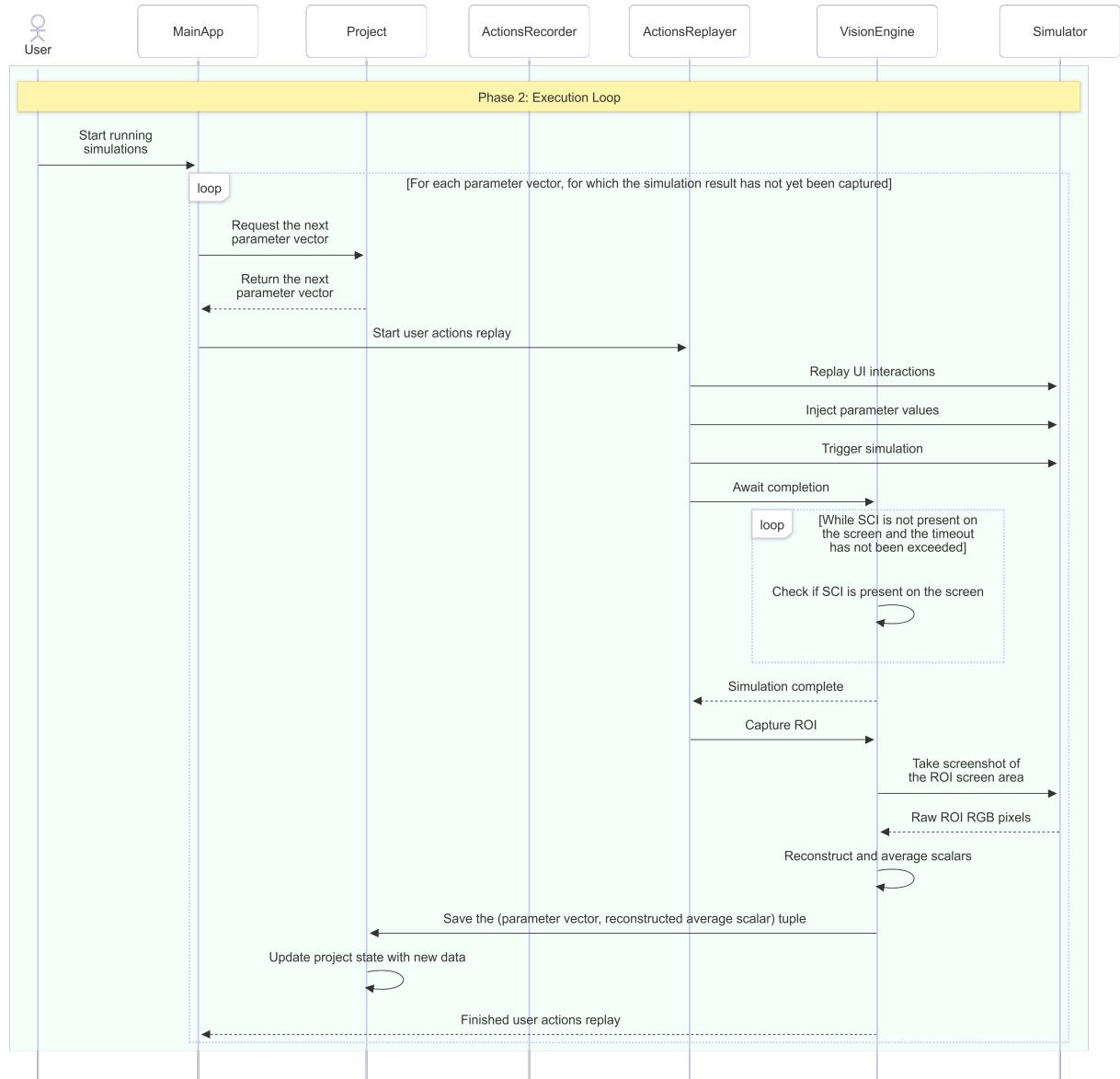


Figure 3.2: Execution Loop

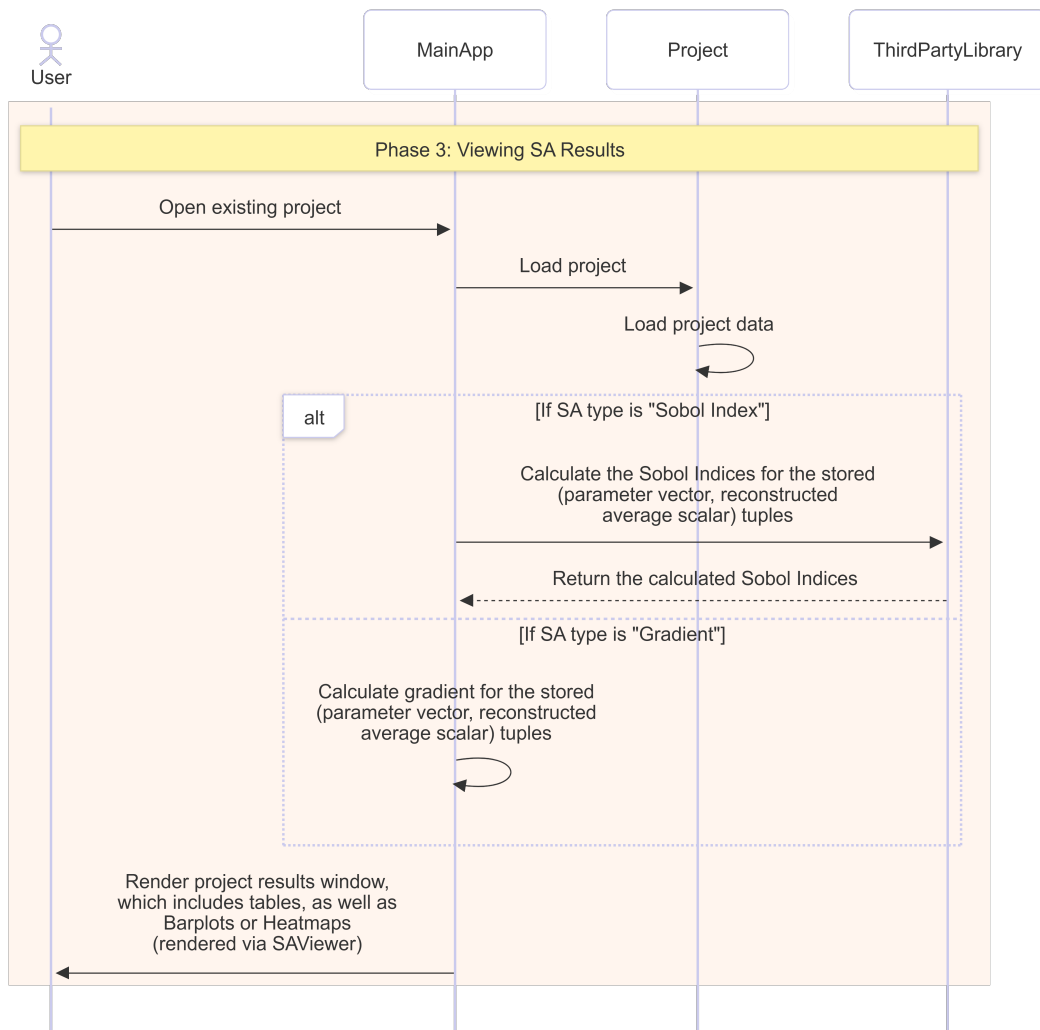


Figure 3.3: Viewing SA Results

Chapter 4

Implementation

The designed architecture, functional and non-functional requirements delineated in the previous chapter must now be translated into a complete working software implementation. This chapter provides an overview of the specific technology stack chosen to realize our application and then explains in detail how the various types of its functionality are implemented: user actions recording and replay, SCI detection, ROI extraction, colormap inversion and sensitivity indices calculation.

4.1 Technology Stack

Our overarching automation and analysis software is implemented entirely in Python [32] (version 3.13.13). Python was explicitly selected due to its high simplicity and flexibility, coupled with the fact that the performance it provides is more than enough for our use case. Python has a huge ecosystem of libraries of various kinds, and, by being able to interact with the C code, it can provide seamless integration between low-level hardware control APIs, high-speed computer vision processing, and various analytical libraries. The implementation of our sensitivity automation software relies on the following set of foundational dependencies:

- **GUI Framework (tkinter) [33]:** Our application’s graphical user interface is constructed using `tkinter` and its themed extension `ttk`. While relatively austere compared to modern web-based frameworks, `tkinter` has all the features we need for our use case. Moreover, it is natively integrated into the Python standard library. This guarantees deep cross-platform compatibility and eliminates the need for heavyweight third-party GUI frameworks (like PyQt).
- **Robotic Process Automation (pynput) [34]:** The `pynput` library is the core driver for the `ActionsRecorder` and `ActionsReplayer` modules. It allows our application to install low-level, global event hooks to capture human mouse and keyboard actions and subsequently inject synthetic ones, effectively simulating user activity.
- **High-Speed Display (Screenshot) Capture (mss) [35]:** Taking screenshots frequently (for example, to check whether the SCI appeared on the screen) from a high-resolution display requires a performant and highly-optimized library. Standard tools like `PIL.ImageGrab` [36]

are notoriously slow, often taking hundreds of milliseconds per frame [37,38], which severely degrades the responsiveness of asynchronous state detection loops. Our implementation utilizes `mss` (Multiple Screen Shots), an ultra-fast modern library that can take screenshots in the time of a few milliseconds, achieving almost zero overhead and providing the maximum performance for the `VisionEngine` routines which take screenshots as one of their steps.

- **Data Analysis and Visualization (NumPy [39], Matplotlib [40], SALib [41]):** NumPy is used for all high-performance, vectorized mathematical operations over multidimensional arrays, specifically during the massive Mean Absolute Distance computations required for the SCI detection (see section 4.4). Matplotlib provides us with the access to standardized, mathematically defined colormaps (for example, "Viridis", "Magma", "Cividis", "Jet", "Turbo") [29] and is used to render the final calculated sensitivity indices plots (see section 4.7). The SALib (Sensitivity Analysis Library) package serves as the core computational engine for the calculation of Sobol indices.
- **Computer Vision Backend (OpenCV) [42]:** The `cv2` module from the OpenCV library provides the heavily optimized C++ backend for executing a huge variety of image processing operations (for example, color space conversion, template matching, etc.). It is worth mentioning that in our application's source code we have implemented some simple image operations with NumPy instead (which is by no means less efficient). However, there are still some places in our code in which we use OpenCV as the image processing engine.

Table 4.1: Software Dependencies, Their Categorization and Justification

Dependency	Domain / Function	Justification
<code>tkinter</code>	GUI Building	Lightweight, rich enough, zero external dependencies (part of the Python Standard Library).
<code>pynput</code>	Robotic Process Automation	Global OS mouse and keyboard event listening and injection.
<code>mss</code>	Screenshot Capture	Highly-performant engine for (frequent) computer screen snapshots.
<code>Matplotlib</code>	Colormaps and Plot Display	Provides access to standard, popular, widely-used colormaps, as well as utilities for building and displaying plots of various kinds (barplot, heatmap, etc.).
<code>SALib</code>	Sensitivity Analysis	Mathematically correct and optimized implementations of Sobol index calculation and Saltelli sampling scheme.
<code>NumPy</code>	Multidimensional Array Operations	Provides an engine to execute numerous different operations with multidimensional arrays in a performant, vectorized fashion.
<code>OpenCV</code>	Image Processing Operations	Provides an immense diversity of highly-optimized image processing operations.

4.2 Project Creation and GUI Actions Recording

The initial phase of the automation workflow requires the human user to create a new sensitivity analysis project and then to translate her knowledge of how to execute a single simulation run into a reproducible sequence of actions (the **Recording Loop**). From the application’s main menu (Figure 4.1), the user opts to create a new SA project by supplying a designated file system directory and project name, as shown in Figure 4.2.

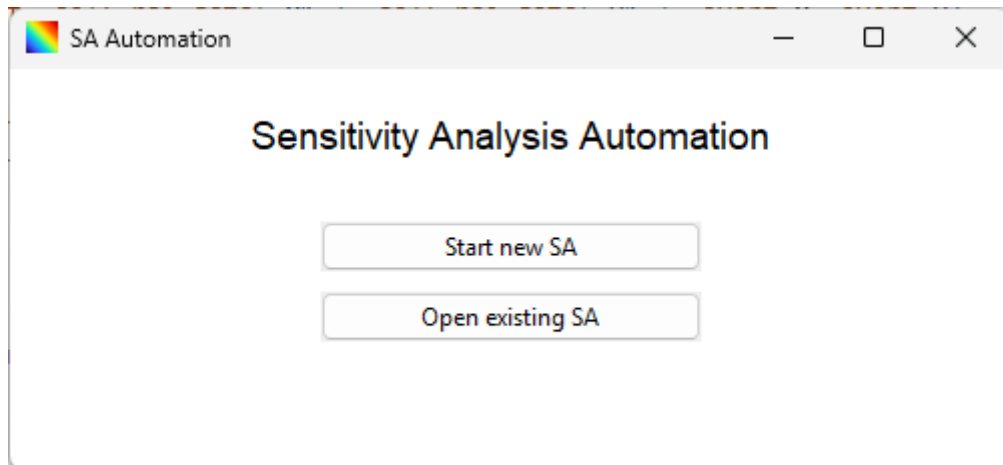


Figure 4.1: Main Menu Window

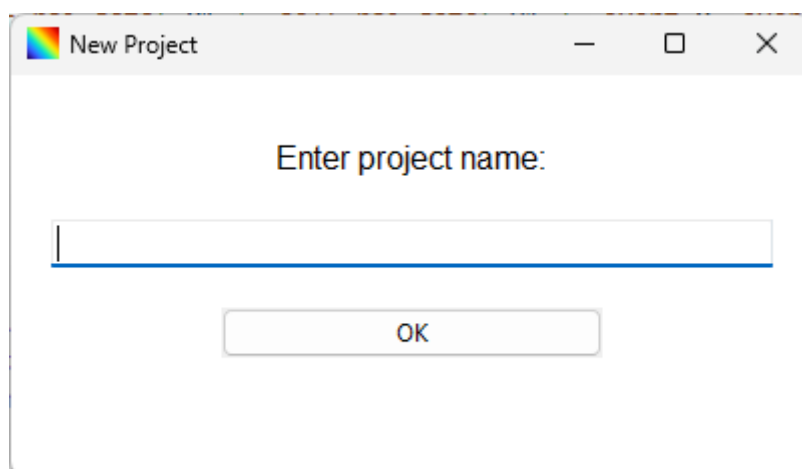
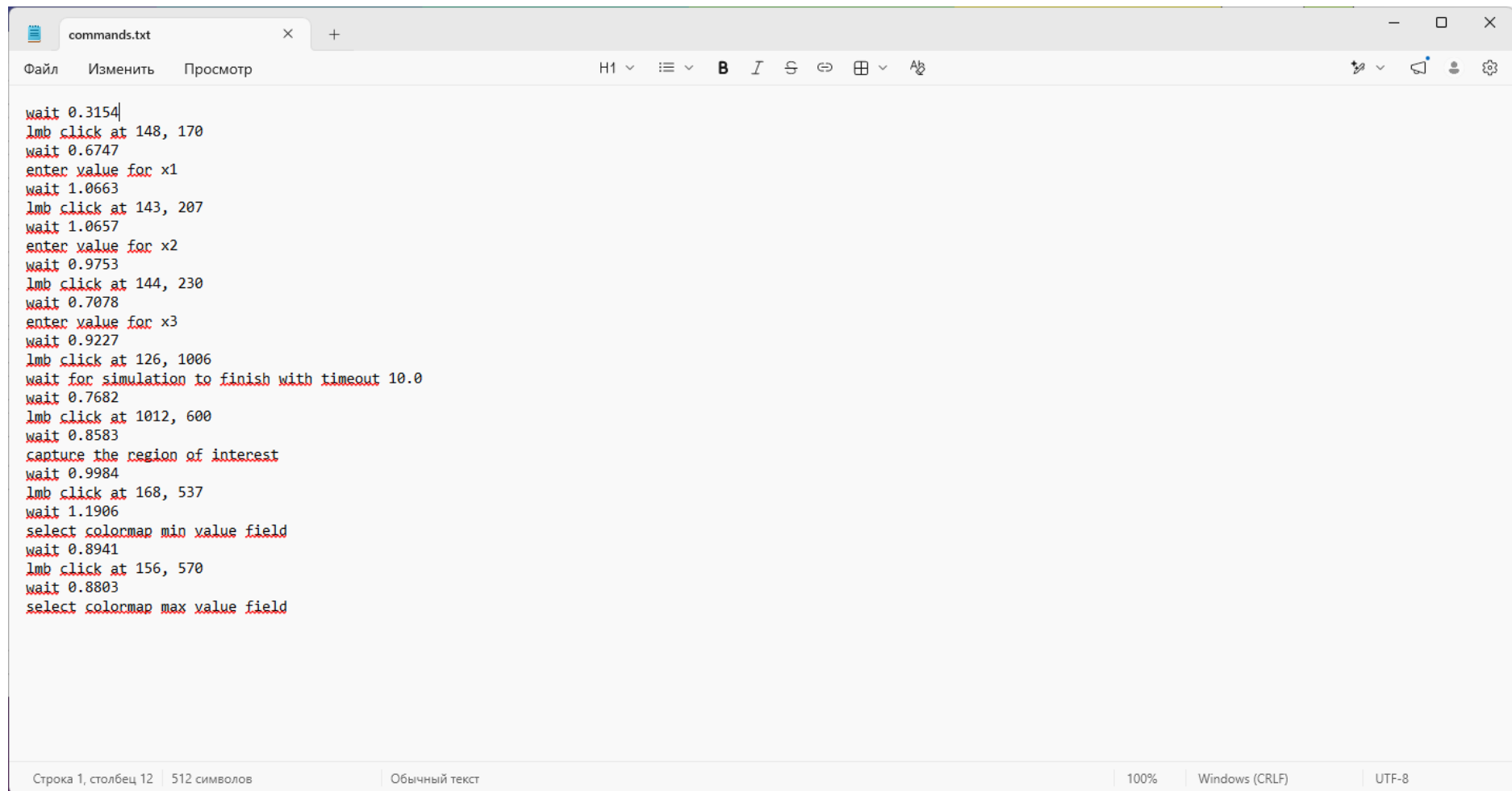


Figure 4.2: Enter Project Name Window

Once the project is initialized, our application launches the **ActionsRecorder** module. It immediately registers global asynchronous listeners via `pynput.mouse.Listener` [43] and `pynput.keyboard.Listener` [44]. Because these listeners execute in separate background threads, they do not block the main execution loop of our Python application. From this exact timestamp, every mouse and keyboard action the user performs is logged into a simple text file (.txt) — the Command File. To ensure that the subsequent replay maintains the exact pacing necessary for graphical components (such as drop-down menus or new windows) to load, the **ActionsRecorder** calculates and logs the precise floating-point time delta (in seconds) between consecutive user action events. An example of a Command File is shown on Figure 4.3. This command file serves

as the operational blueprint. It contains the mouse click events with the exact (x, y) Cartesian screen coordinates, the keyboard button press and release events, and the special commands (for example, "enter value for x_1 ", encountering which will make our application inject the sampled value for the parameter x_1 into the corresponding input field (it is assumed that the previous command — "lmb click at 148, 170" — selects the corresponding input field)). The user can open this file at any time by clicking on the "View command file" button in the Configuration Menu. Moreover, the user can **edit** this file and save it back. This is useful, for example, if she did some wrong mouse click or keyboard key press and wants to remove it from the recorded workflow without requiring a full workflow re-demonstration.



```
wait 0.3154|
lmb click at 148, 170
wait 0.6747
enter value for x1
wait 1.0663
lmb click at 143, 207
wait 1.0657
enter value for x2
wait 0.9753
lmb click at 144, 230
wait 0.7078
enter value for x3
wait 0.9227
lmb click at 126, 1006
wait for simulation to finish with timeout 10.0
wait 0.7682
lmb click at 1012, 600
wait 0.8583
capture the region of interest
wait 0.9984
lmb click at 168, 537
wait 1.1906
select colormap min value field
wait 0.8941
lmb click at 156, 570
wait 0.8803
select colormap max value field
```

Строка 1, столбец 12 | 512 символов | Обычный текст | 100% | Windows (CRLF) | UTF-8

Figure 4.3: Command File

During this **Recording Loop**, the user must add the simulation input variables that will constitute the sensitivity analysis parameters. By pressing a designated keyboard button (specifically, the "Home" one), the user suspends the **ActionsRecorder**'s event listening, safely preventing the application's own Configuration Menu interactions from being logged and polluting the target simulation's recorded workflow.

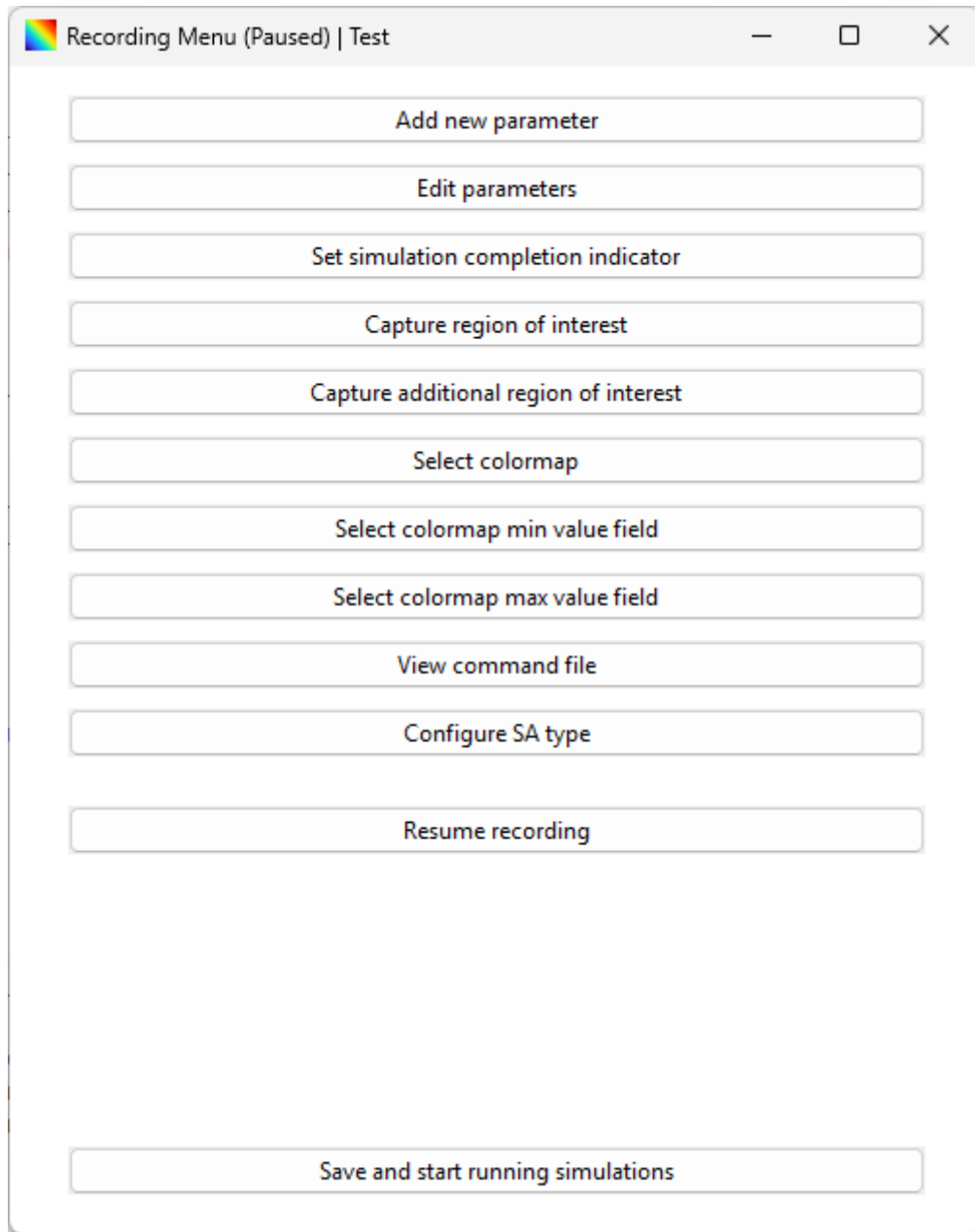


Figure 4.4: Recording Menu Window

Pressing "Home" invokes the primary configuration overlay (we call it the Configuration Menu or Recording Menu) (Figure 4.4), from which the user can explicitly map the current UI input field (the mouse click on which was her last recorded action before she pressed the "Home" key) to a formally named parameter, as shown in Figure 4.5.

Here, the user defines the parameter's name and establishes its continuous bounded domain $[p_{min}, p_{max}]$. Our application provides a dedicated interface to review, modify, or delete these

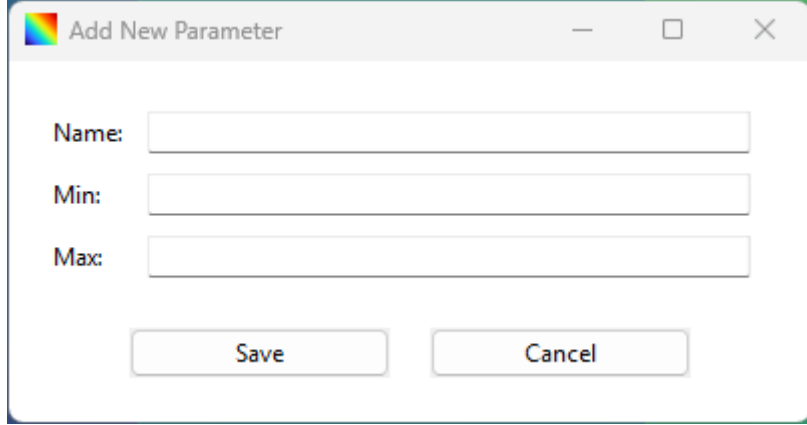


Figure 4.5: Add New Parameter Window

variables from the sensitivity analysis project (Figure 4.6).

When all the simulation input parameters are added, the user must explicitly configure the sensitivity analysis itself, which she wants to perform, as shown (for the Gradient-type SA) in Figure 4.7.

The user selects whether to perform a localized Gradient analysis or a global variance-based Sobol index calculation. Our application automatically determines the total number of required simulation runs based on the number of input parameters and the sensitivity analysis type when the user clicks the "Generate sample" button. For a Gradient analysis across N parameters, our application requires $2N$ iterations to compute the central second-order-finite-difference partial derivative approximations (Figure 4.8).

Furthermore, the user must configure the Simulation Completion Indicator (explained in the section 4.4), select the colormap that the target simulation software utilizes to render its output as a color-coded image (to ensure correctness of the later scalar reconstruction phase) (Figure 4.9), configure its minimum and maximum value fields (explained later in the section 4.6), the ROI and, optionally, the Additional ROI (explained in the section 4.5).

Once the entire operational cycle of a single simulation run — inputting parameters, triggering the solver, capturing the output, and navigating back in the simulator's GUI to the place we were at the beginning (before inputting parameters) — is completely recorded and configured, the user is ready to press the "Save and start running simulations" button, which will save the final versions of both all the project metadata (settings) and the command file and will start running simulations, thereby transitioning to the **Execution Loop** phase.

4.3 Workflow Replay

The execution of the parametric sweep is governed by the `ActionsReplayer` module. Given a pre-generated collection of simulation input parameter vectors $\mathbf{p}_i$, for each such vector, `ActionsReplayer` parses the command file line-by-line, translates each command into the corresponding pynput controller (`pynput.mouse.Controller` [45] or `pynput.keyboard.Controller` [46]) actions and executes them.

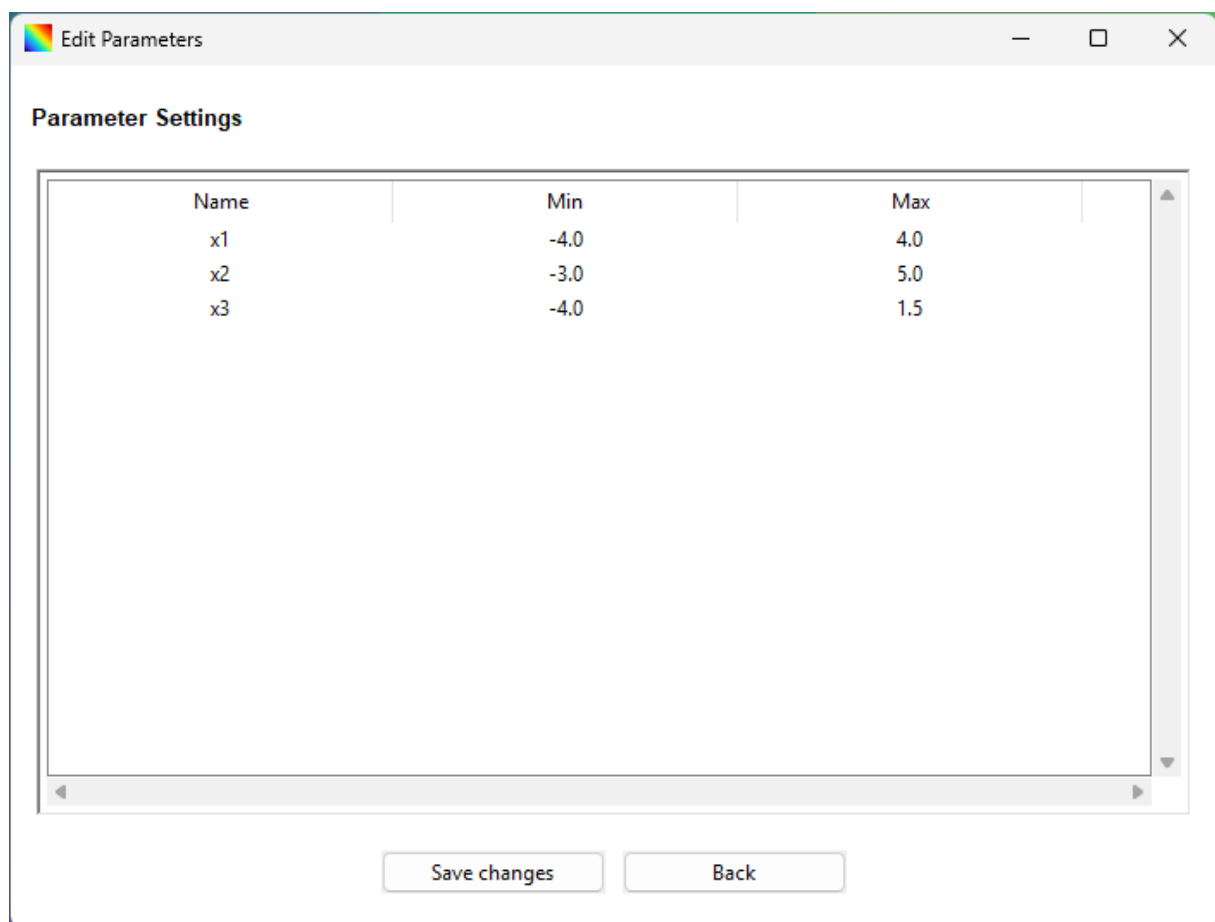


Figure 4.6: Edit Parameters Window

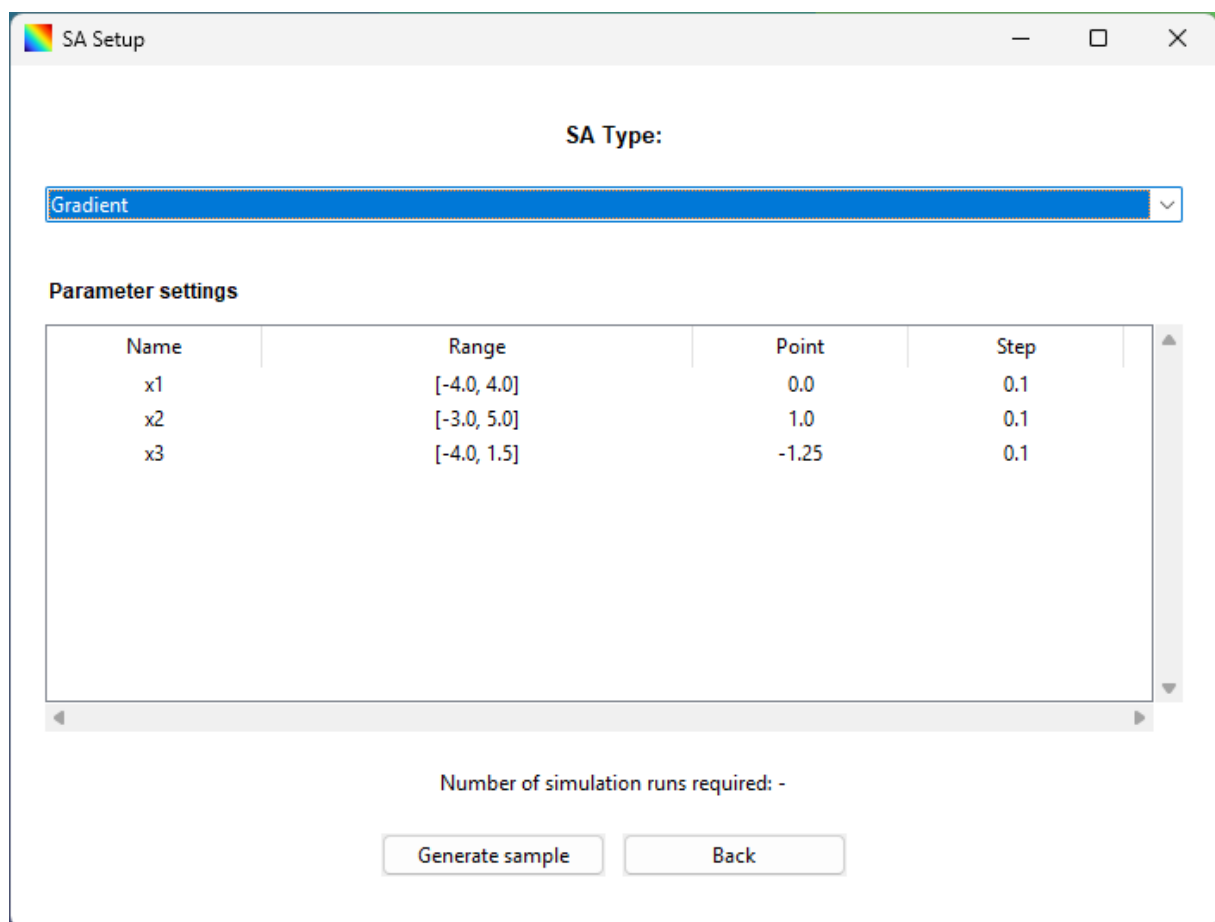


Figure 4.7: Configure SA Type Window

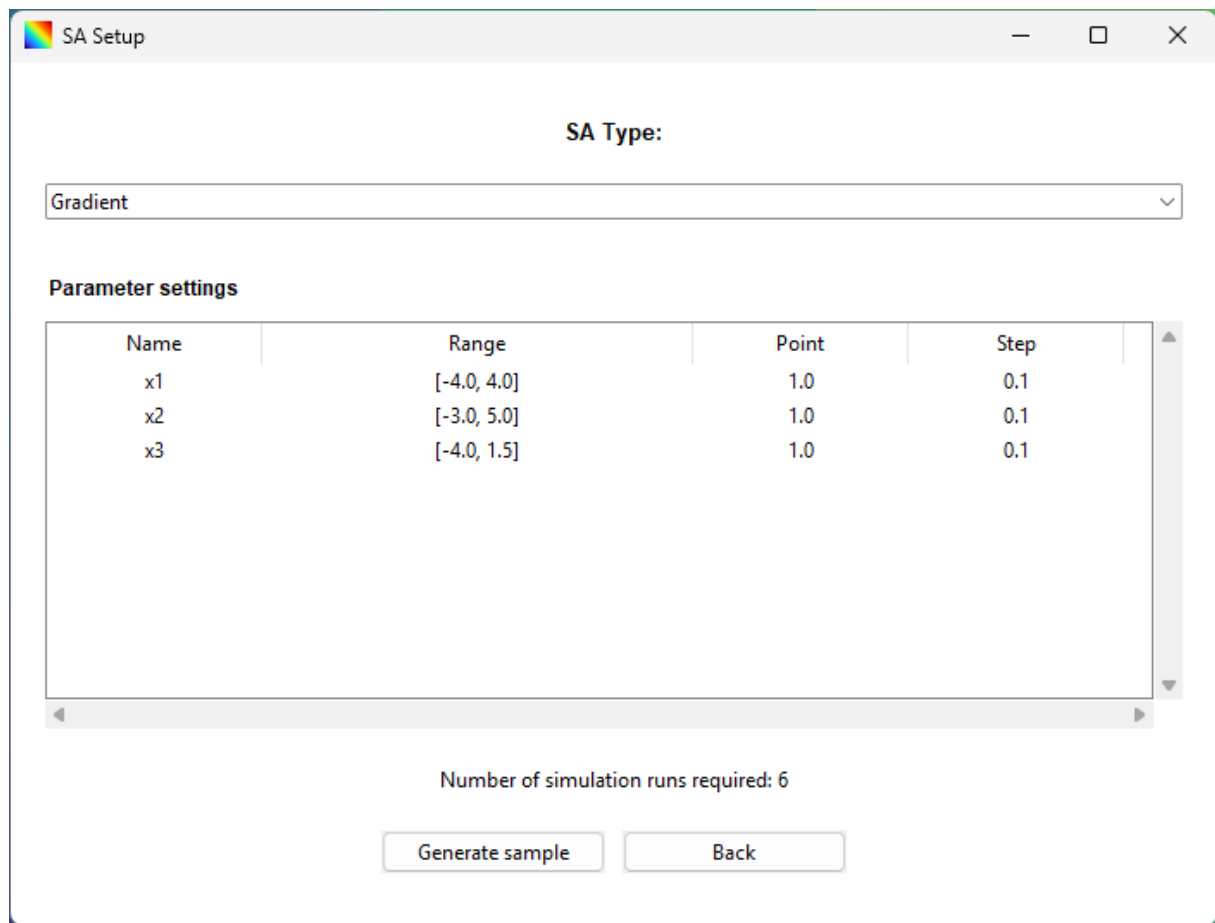


Figure 4.8: Configure SA Type Window: Number of Simulation Runs Required

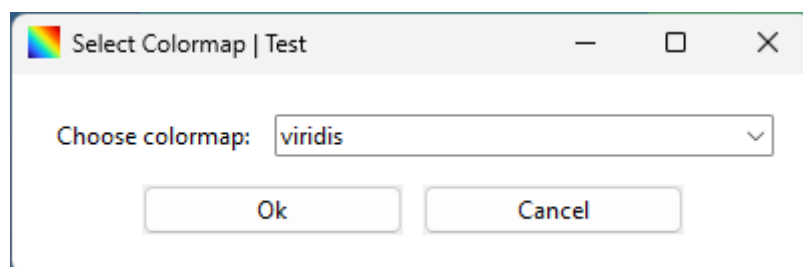


Figure 4.9: Select Colormap Window

When a standard mouse click or keyboard button press is parsed, they generate exactly one corresponding `pynput` command. When a "wait" command (for example, "wait 0.7") is encountered, `ActionsReplayer` commands the executing thread to sleep for the specified time duration (in seconds). The "wait" commands in the Command File are the delays between consecutive user actions logged during the recording phase. When the parser encounters a command designated for parameter injection (e.g., "enter value for x1"), the `ActionsReplayer` extracts the specific floating-point value for x_1 from the current parameter vector $\mathbf{p}_i$. Then it converts this floating-point scalar into a character string, and simulates the individual keystrokes required to type that exact numerical value into the target simulation software's active input field. To prevent potential GUI anomalies and errors caused by synthetic typing speeds exceeding that of a human by orders of magnitude, the `ActionsReplayer` module waits for some time after each keystroke. Additional special commands (like "capture the region of interest") are detailed in the corresponding sections of this chapter.

Because a comprehensive global sensitivity analysis can take a long time, our software allows the user to safely halt the execution pipeline. By pressing "Home" during the execution phase, the custom `PauseRequested` exception is raised and subsequently caught by the `ActionsReplayer` module, which suspends the replay of the recorded user's actions, discards any intermediate results the current simulation run might have produced, displays the corresponding "Replay paused" GUI window (Figure 4.10) and allows the user to reclaim manual control of the workstation.

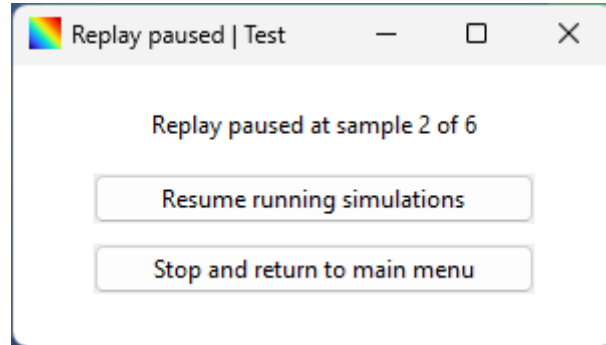


Figure 4.10: Replay Paused Window

The user may do whatever she needs with the PC and then either to resume running simulations by pressing on the corresponding button or to stop running simulations and close the SA project. In that case, when she later wants to continue running simulations, she should just open this project as an existing one (Main menu → Open existing SA). There she will be able to see the Project Dashboard (Figure 4.11), which reflects the current state of the sensitivity analysis project, including its results, if they are available (the project must have the status "Completed"). For example, the user can see which simulation input parameters are taking part in the sensitivity analysis, how many required simulation runs have already been performed, and so on. The project has the status "Setup" when a user is still in progress of recording her actions and configuring the SA (is still in the **Recording Loop**), "In progress" when the user has finished configuring the project, started the simulation runs and then suspended them (she is in the **Execution Loop**) and "Completed" when all the required simulation runs have been successfully performed and the corresponding simulation results $\hat{f}_i$ have been obtained for all the generated input parameter

vectors $\mathbf{p}_i$.

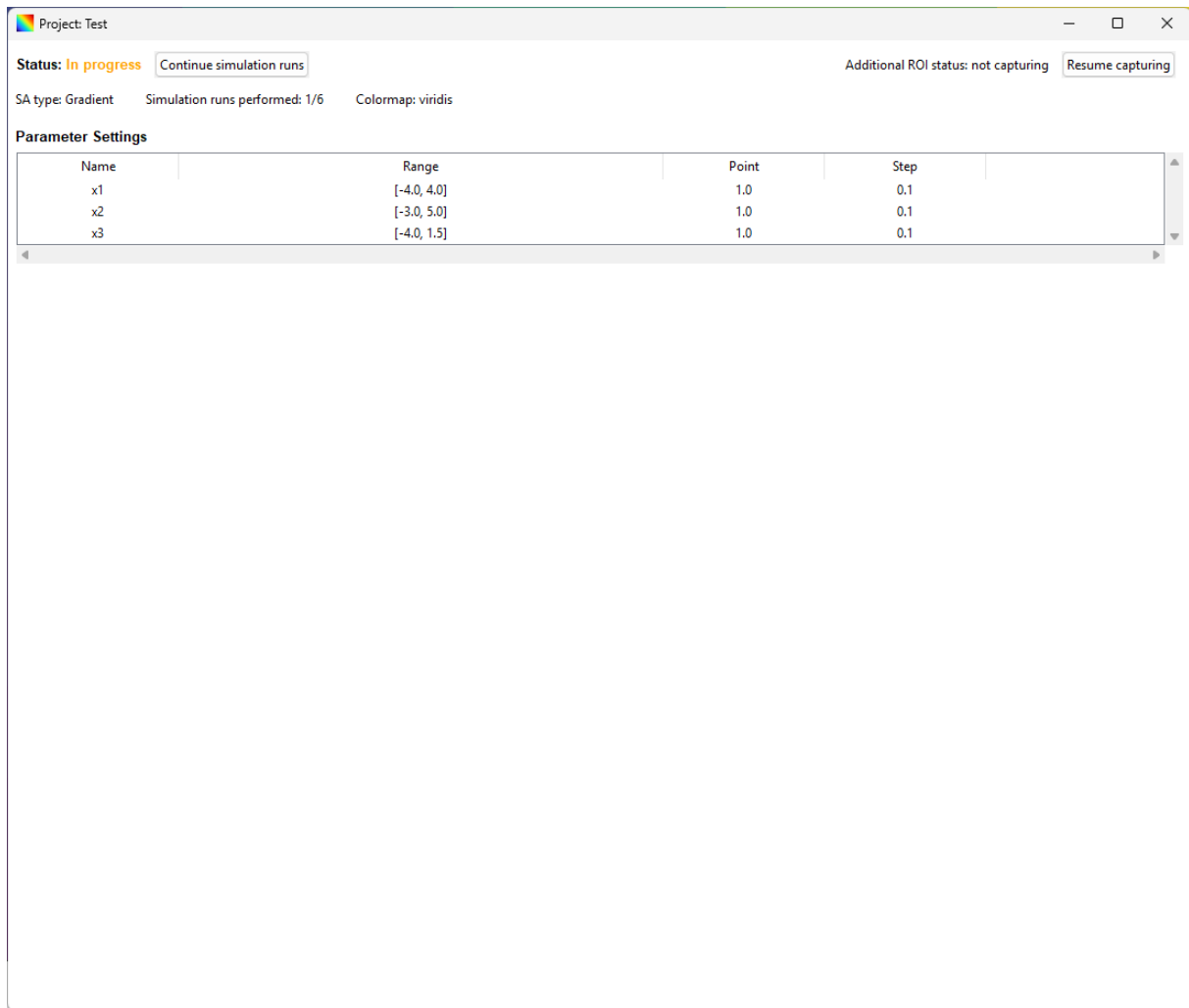


Figure 4.11: Project Dashboard Window (In Progress)

4.4 Simulation Completion Detection

One of the crucial steps in our black-box interaction with the simulator is accurately determining when its numerical solver has finished its execution and the graphical output (color-coded image) is ready for visual extraction. Static pre-specified time delays are quite a bad solution: they either waste massive amounts of computational time if set too conservatively, or they cause catastrophic desynchronization and the complete failure of the **Execution Loop** if a single simulation time slightly exceeds the allotted wait period.

To resolve this structurally, during the recording phase, the user captures (takes a screenshot of) and saves (via the corresponding Configuration Menu button (Figures 4.12, 4.13)) a Simulation Completion Indicator — a specific template of a (rectangular) part of the screen, presence of which on the screen reliably signifies that the simulation has been completed. It can be, for example, a "Done" GUI window popped up, some status indicator turned green, the simulation progress value turned "100%" and so on. The successful capture of the SCI by the user produces

2 outputs: the image template (how the SCI looks like), which is stored on disk in the SA project folder, and the screen region coordinates $[X_{min}, Y_{min}, X_{max}, Y_{max}]$ (where the SCI should appear), which are stored in the project metadata.

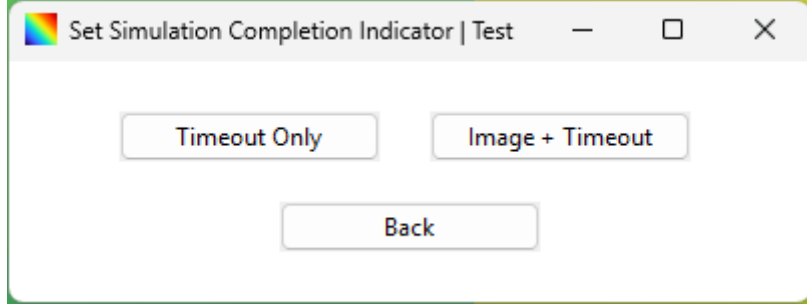


Figure 4.12: Set Simulation Completion Indicator Window

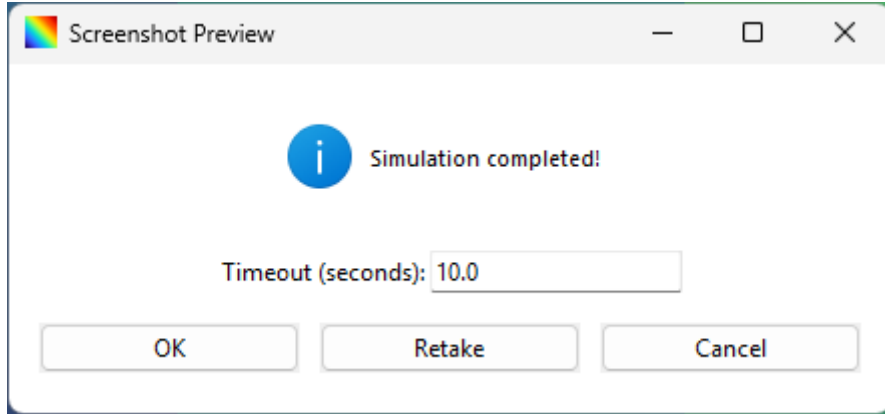


Figure 4.13: Set Simulation Completion Indicator: Screenshot Preview Window

During the replay phase, once the simulation has started and the "wait for simulation to finish with timeout" command was encountered in the Command File, the **VisionEngine** module starts repeatedly (with some small time interval) capturing the screen region (using `mss`) at the same screen coordinates at which the user did it during the recording phase. **VisionEngine** then compares the captured screen region with the stored template using simple Mean Absolute Distance approach:

$$D = \frac{1}{H \times W \times C} \sum_{y=1}^H \sum_{x=1}^W \sum_{c=1}^C |I_{\text{screen}}(x, y, c) - I_{\text{template}}(x, y, c)| \quad (4.1)$$

where H , W , and C represent the height, width, and color channels (3 in our RGB case) of the SCI image, while I_{screen} and I_{template} denote the pixel intensity values of the captured screen region and stored template image, respectively.

If D is below a high-confidence threshold (e.g., 10^{-7}), **VisionEngine** definitively concludes that the SCI has appeared on the screen and, thus, the simulation has been finished. This asynchronous SCI polling loop lets our SA automation application effectively handle unpredictable solver run durations.

4.5 Region of Interest Extraction

Once the Simulation Completion Indicator is successfully detected and the parser encounters the "capture the region of interest" command in the Command File, **VisionEngine** proceeds to extract the Region of Interest (an array of raw RGB pixels) from the simulation color-coded output picture. For that, during the recording phase, via the "Capture region of interest" Configuration Menu button, the user should define the coordinates of this primary Region of Interest by drawing a rectangular bounding box over the color-coded output picture of the simulator (the process of defining the ROI looks absolutely the same as the process of defining (capturing) the SCI (Figures 4.12, 4.13)). The coordinates of this bounding box $[X_{min}, Y_{min}, X_{max}, Y_{max}]$ are stored in the project metadata.

The **VisionEngine** module executes a final **mss** snapshot of the ROI screen region. The resulting 3-dimensional RGB pixel array is passed directly into the **rgb_to_scalar** colormap inversion routine. If during the recording phase the user configured an "Additional ROI" (in the same fashion as the primary ROI), when the parser encounters the "capture additional region of interest" command in the Command File, it performs another screen region snapshot at the corresponding coordinates (which are also stored in the project metadata). Unlike the primary ROI, the Additional ROI is not subjected to scalar reconstruction; it is simply saved to disk as a **.png** image, thereby preserving additional data for subsequent human usage (e.g., some extended manual analysis).

4.6 Colormap Inversion

After a successful solver run, the simulation software produces a color-coded image showing us the distribution of some physical quantity (e.g., temperature) over some 2d domain (e.g., a wall of a house, a computer motherboard, a longitudinal section of a pipe). From the mathematical perspective, a color-coded image is a function:

$$\mathbf{C}(x, y) : \mathbb{R}^2 \rightarrow \mathbb{R}^3 \quad (4.2)$$

For each given tuple of screen coordinates (x, y) it outputs a pixel color — an RGB vector. But the color (an RGB vector) is just a representation of the underlying scalar field (e.g., the temperature). For the sensitivity analysis purposes, we do not need the function above, we need the underlying scalar field:

$$f(x, y) : \mathbb{R}^2 \rightarrow \mathbb{R} \quad (4.3)$$

This scalar field is the actual solver's output. It is then being transformed into the color-coded image by a colormap function, which takes a scalar and assigns it a color (an RGB vector):

$$C(\tau) : [0, 1] \rightarrow \mathbb{R}^3, \quad \tau \mapsto \begin{bmatrix} R(\tau) \\ G(\tau) \\ B(\tau) \end{bmatrix} \quad (4.4)$$

There are many such colormap functions ("Viridis", "Magma", "Rainbow", etc.). Each of them is basically a parametric curve in the 3-dimensional RGB space. The input scalar argument τ specifies the position on this curve, with 0 pointing at the "beginning" of the curve and 1 pointing at its "end".

The colormap function takes in a scalar in the interval $[0, 1]$, but the scalar field $f(x, y)$ has values not necessarily in this interval. How is this scalar field then transformed into colors by a colormap function? It is first min-max-rescaled. Let $a \leq x \leq b$ and $c \leq y \leq d$ (our color-coded image is located in the rectangular region of the screen, the coordinates of which are bounded by a, b, c, d). Then:

$$f_{\min} = \min_{\substack{a \leq x \leq b \\ c \leq y \leq d}} f(x, y) \quad (4.5)$$

$$f_{\max} = \max_{\substack{a \leq x \leq b \\ c \leq y \leq d}} f(x, y) \quad (4.6)$$

$$f_n(x, y) = \frac{f(x, y) - f_{\min}}{f_{\max} - f_{\min}} \quad (4.7)$$

This normalized scalar field $f_n(x, y)$ is then transformed by the colormap function.

In the definition above, the colormap function is continuous. However, a lot of popular colormaps (such as "Viridis", "Magma", "Jet", "Turbo", "Cividis" and others) are defined not as continuous curves, but just as discrete lists of colors (with the number of entries (unique colors) usually being equal to 256) [47, 48]. This entails that the domain of the colormap function's input argument τ (the interval $[0, 1]$) is discretized into 256 bins b_k (Figure 4.14).

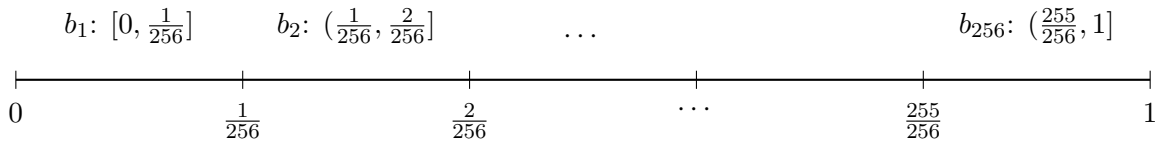


Figure 4.14: Discretization of the τ domain

Each such bin is then assigned a distinct color.

Our task is, having the colors of the color-coded image $C_{ij} = C(\tau_{ij}) = C(f_n(x_i, y_j))$, to reconstruct all the corresponding initial scalar arguments τ_{ij} from them, which will give us the complete normalized scalar field $f_n(x, y)$. This is what we call the "colormap inversion". As we know the colormap which was used to transform scalars τ_{ij} to colors C_{ij} (the user must specify it during the recording phase), for each color C_{ij} we can find the corresponding bin b_k of width w (usually, for most popular colormaps, and in our example, $w = \frac{1}{256}$, in general, $w = \frac{1}{N}$, where N is number of bins in the colormap). At this point we understand that we are unable to

reconstruct the scalars τ_{ij} perfectly (without an error) since the colormap's data compression into 256 bins introduces an irreparable information loss: we only know that the true scalar τ_{ij} lies in a given bin b_k : $[\frac{k-1}{256}, \frac{k}{256}]$ of width $w = \frac{1}{256}$. Thus, our target is, for each i, j to reconstruct an approximation $\hat{\tau}^*$, which will be as close to the true τ as possible, for example, in terms of expected squared error:

$$\hat{\tau}^* = \min_{\hat{\tau} \in \hat{T}} \mathbb{E}[(\hat{\tau} - \tau)^2] \quad (4.8)$$

Here $\hat{T}$ is the class of all possible approximations of τ .

Assuming that the true scalar τ may lie at every point of its bin with equal probability (the uniform prior assumption), the minimizer $\hat{\tau}^*$ of the expected squared reconstruction error is the bin's midpoint: $\hat{\tau}^* = \frac{k - \frac{1}{2}}{256}$. The maximum and minimum possible error of such reconstruction is $\frac{w}{2}$ (since we know the true scalar lies in this bin).

Having reconstructed the scalars $\hat{\tau}_{ij}^*$, we average them:

$$\bar{\tau} = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W \hat{\tau}_{ij}^* \quad (4.9)$$

Finally, in order to get the average reconstructed simulation function value $\bar{f}$, we min-max-unscale the $\bar{\tau}$, using the known minimum and maximum function values f_{min} and f_{max} for the given simulation run:

$$\bar{f} = f_{min} + \bar{\tau} \times (f_{max} - f_{min}) \quad (4.10)$$

Where do we get the f_{min} and f_{max} from? During the recording phase, after the simulation has finished and the color-coded output image has appeared on the screen, the user has to navigate to this image's display settings, find there the field containing the f_{min} value, click on that field, pause the actions recording and press the "Select colormap min value field" in the Configuration Menu. Then the user has to do the same for the field containing the f_{max} value. During the **Execution Loop**, when the parser encounters the "select colormap min value field" and "select colormap max value field" commands in the Command File, **ActionsReplayer** will emulate the user keyboard "Ctrl + A, Ctrl + C" presses, thereby copying the value in the selected field into clipboard, and then it will extract the value from the clipboard using the **pyperclip** [49] library. The complete implementation of the colormap inversion is located in the **VisionEngine** module in the **rgb_to_scalar** function.

4.7 Sensitivity Indices Calculation (SA Project Results)

Once the **Execution Loop** has been fully completed and we have obtained the necessary simulation function output values $\bar{f}_i$ for every parameter vector $\mathbf{p}_i$, we can finally perform the sensitivity analysis based on the tuples $(\mathbf{p}_i, \bar{f}_i)$, the results of which will occupy the main part of the sensitivity analysis project results area in the Project Dashboard (the only small additional

result we include there is the statistics (min, max, mean, standard deviation, etc.) computed over the reconstructed simulation function values $\hat{f}_i$). Our MVP supports calculation of the two basic well-known sensitivity indices: gradient and Sobol.

4.7.1 Gradient

For local sensitivity analysis, our application computes the multi-dimensional gradient vector using the second-order central finite difference approximations. During the recording phase, the user has to specify just the point in the simulation input parameter value space at which the gradient should be estimated (let us call it $\mathbf{p}_0$) and the step size for each parameter (h_j). Our application then automatically generates the perturbed parameter vectors $\mathbf{p}_0 + h_j \mathbf{e}_j$ and $\mathbf{p}_0 - h_j \mathbf{e}_j$ for each parameter (when the user clicks the "Generate sample" button in the SA configuration window). An example result of a Gradient SA project is shown in Figure 4.15.

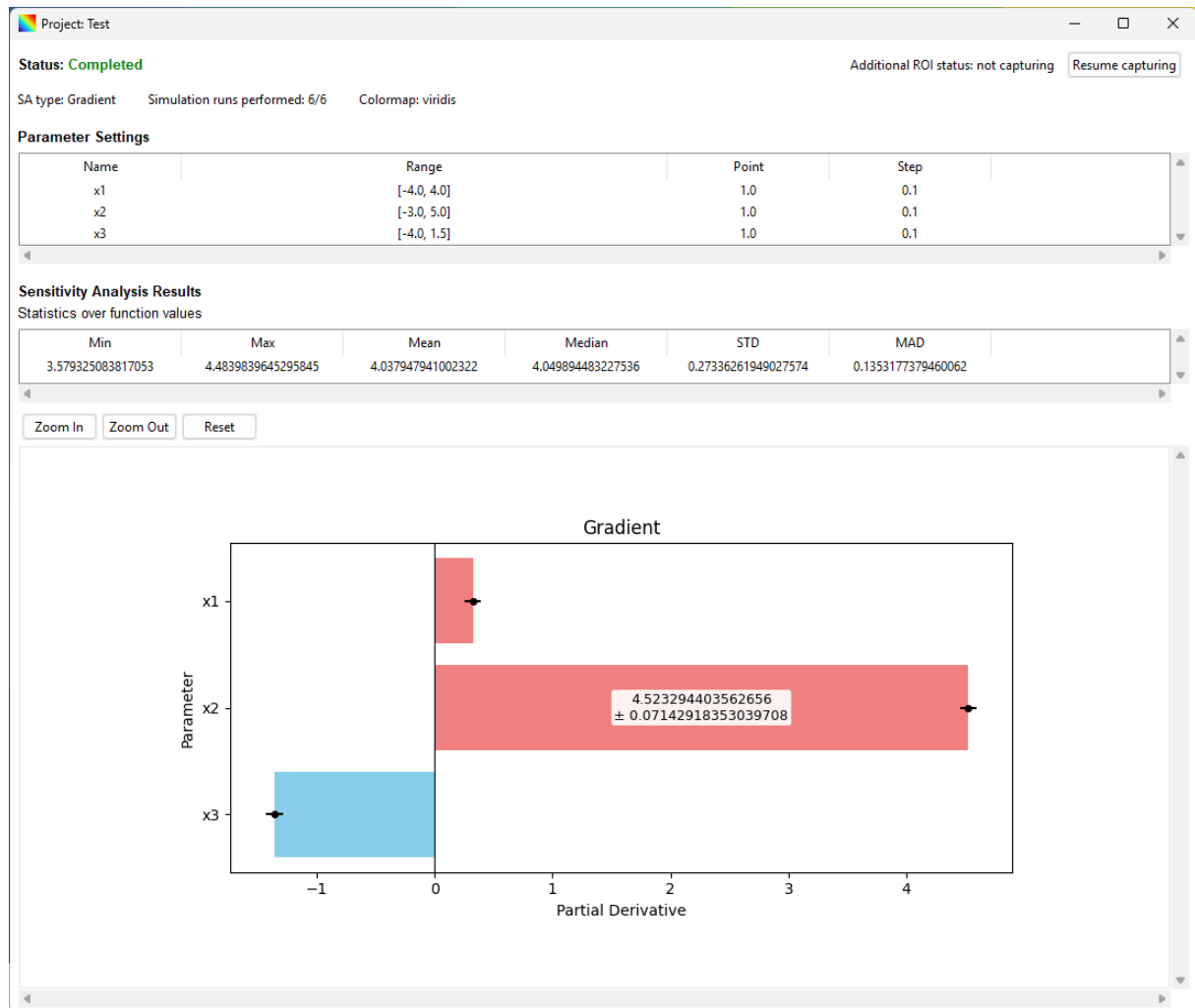


Figure 4.15: Gradient SA Results in Project Dashboard

The coloring of the partial derivative bars is blue for a negative value and red for a positive value. The black symmetric error bars (whiskers) growing from the top of each partial derivative estimate's bar represent the 100% confidence interval for that partial derivative estimate with respect to the value of the second-order finite difference scheme which uses the true function

values in the numerator (instead of the reconstructed-from-color ones) and the same step size in the denominator. More information about this is provided in the chapter 5, section 5.2.1. When the user hovers her mouse over a bar, the annotation with the exact bar and whiskers value is displayed in the yellow rectangle.

4.7.2 Sobol Indices

To assess global, non-linear parameter influences and interactions across their entire bounded domain, our application seamlessly integrates the **SALib** package. Unlike localized gradients, which focus on a single point in the parameter space, variance-based global sensitivity analysis explores all that space, providing a picture of how each simulation input parameter affects the output globally, which makes this kind of SA valuable for complex, black-box engineering simulations.

When during the recording phase the user clicks "Generate sample" for a Sobol-type SA, our application invokes the `saltelli.sample` [50] method of the **SALib**. Given the predefined boundaries of D continuous parameters, the algorithm generates an optimally distributed quasi-random Saltelli sequence of points in the parameter space designed to uniformly explore it all.

When **ActionsReplayer** completes the **Execution Loop**, the results $(\mathbf{p}_i, \hat{f}_i)$ are passed into the `sobol.analyze` [51] function. This function mathematically estimates the variance fractions and calculates bootstrap confidence intervals, yielding a comprehensive global sensitivity profile. An example result of a Sobol index SA project is shown in Figures 4.16 and 4.17. In Figure 4.16 the barplot for the S_1 index is displayed (with the whiskers showing the bootstrap confidence intervals), the plot for the S_T looks the same, just different values and name. In Figure 4.17 the S_2 index values for each pair of simulation input parameters are displayed as a heatmap.

4.8 Project Persistence

Setting up the sensitivity analysis project and running the complete parametric sweep may take a long time. Furthermore, the obtained results (ROIs, Additional ROIs, reconstructed simulation function values $\hat{f}_i$) may be needed many times for multiple different manual human analyses. That is why all the changes of the sensitivity analysis project's state are saved to disk.

All the configuration data — including parameter names and domains, selected colormap, the chosen sensitivity analysis and its configuration, the bounding coordinates for the simulation completion indicator and the corresponding image template, the bounding coordinates for the ROI and optional Additional ROI — are saved in a lightweight, human-and-machine-readable JSON format.

The matrix of stacked row vectors $\mathbf{p}_i^T$ representing sampled points in the simulation input parameter domain, required to complete the configured SA, is stored in a native NumPy array file format (`.npy`) [52], just as the one-dimensional array of corresponding reconstructed simulation function output values $\hat{f}_i$.

The Command File, as it was stated earlier, is stored in a simple `.txt` file format. Finally, the

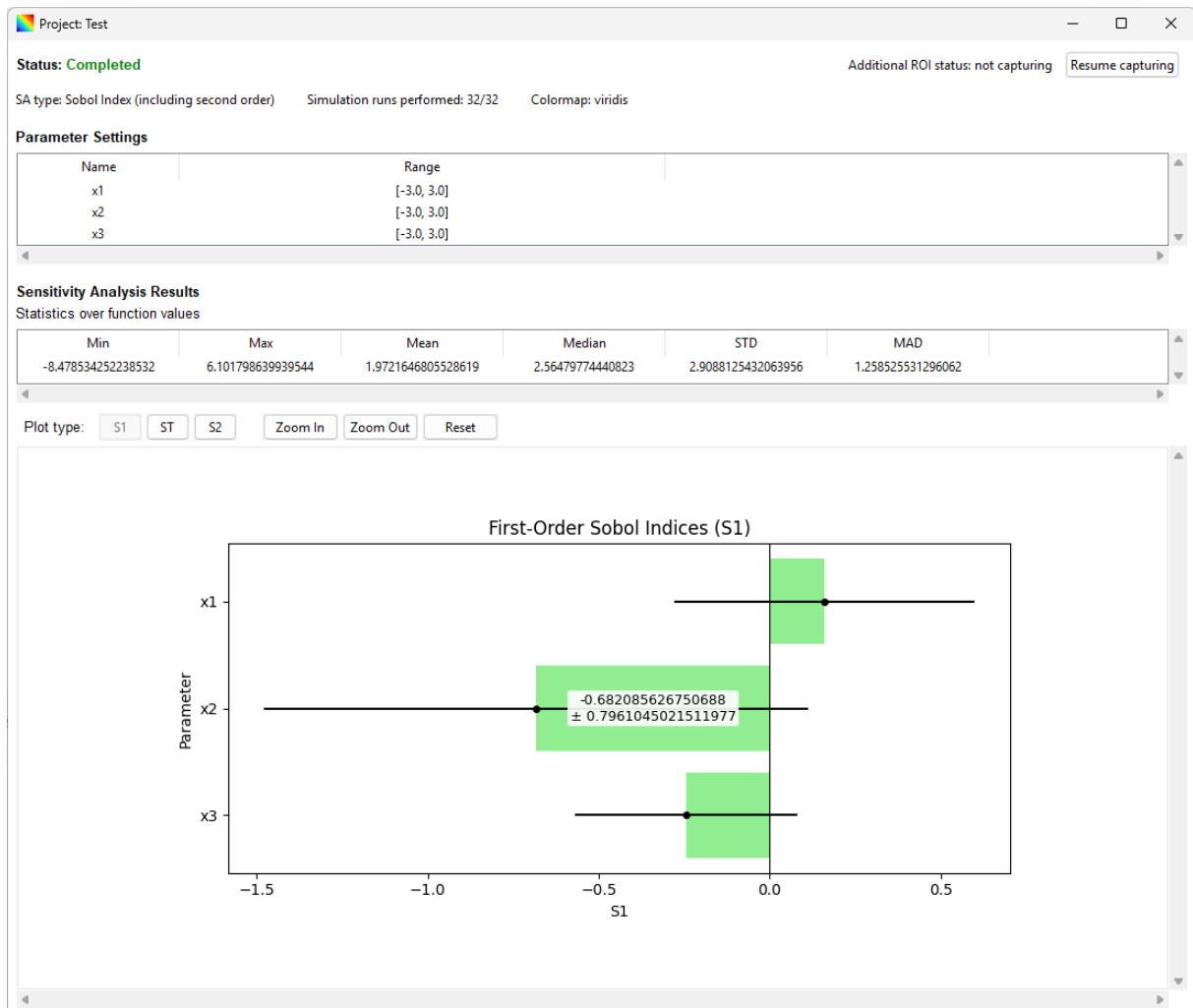


Figure 4.16: Sobol Index SA Results in Project Dashboard: S_1 Plot

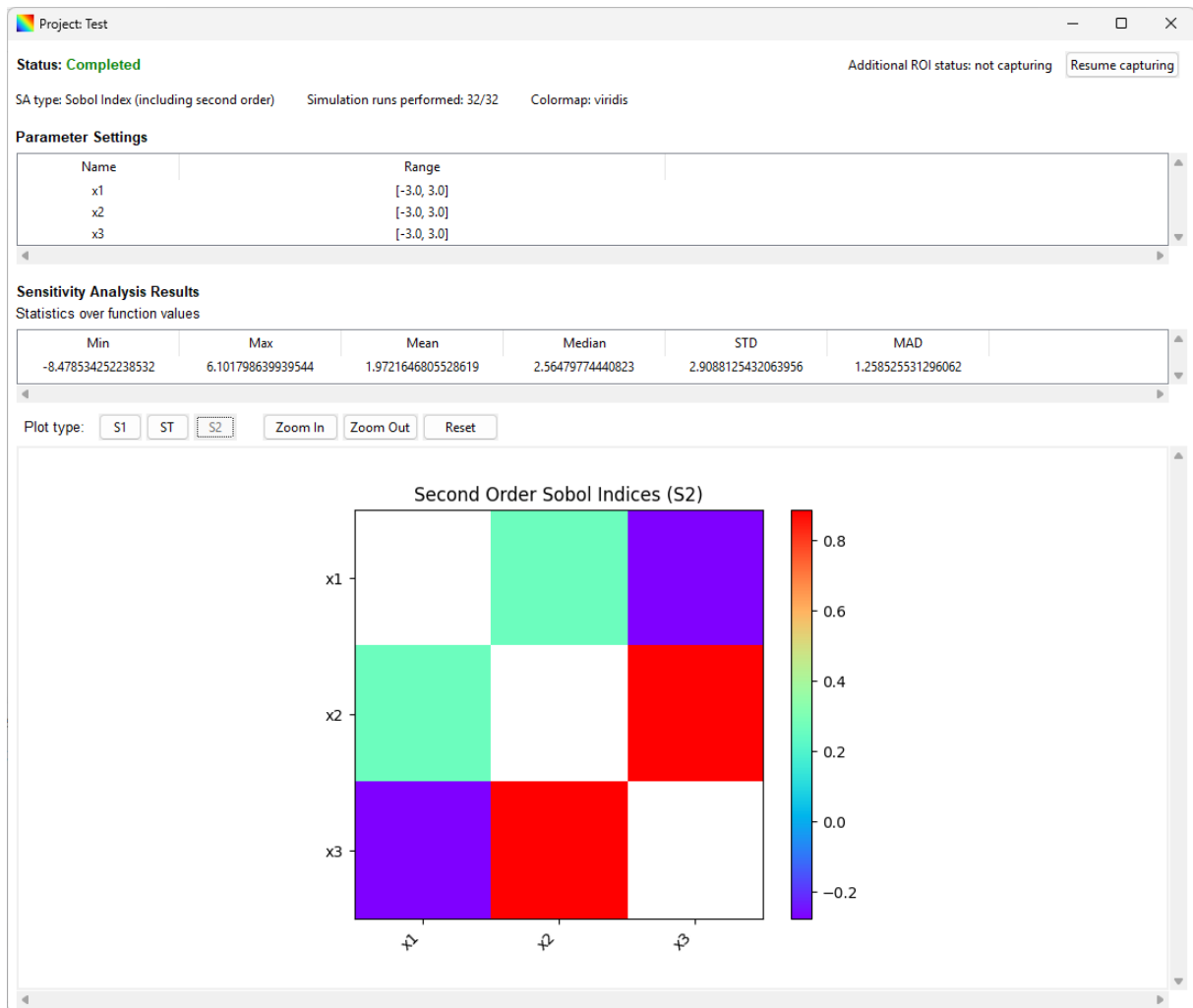


Figure 4.17: Sobol Index SA Results in Project Dashboard: S_2 Plot

SCI template and the captured ROIs for each simulation run are stored as `.png` files in order to prevent any sort of information loss. An example structure of a sensitivity analysis project folder on disk is shown in Figure 4.18 (the Additional ROI was not configured for that project, otherwise we would have one more corresponding sub-folder).

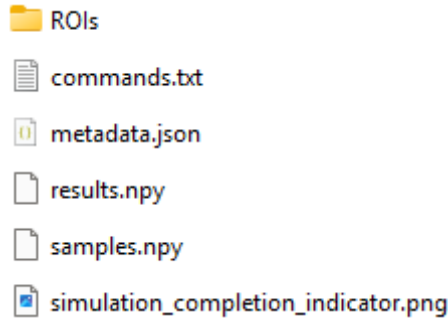


Figure 4.18: SA Project Folder

4.9 Failure Recovery

A fundamental reality of orchestrating complex, third-party desktop engineering applications via GUI automation is that it is highly susceptible to both deterministic and non-deterministic faults. The target numerical simulations may fail to converge for the specified number of iterations, causing the simulation software to enter an unforeseen-at-the-recording-phase state; the operating system may push sudden update notifications that obscure critical GUI elements of the simulator; or the sheer computational load of the solver may trigger a temporary interface freeze. All this can cause subsequent automated mouse clicks and keyboard button presses performed by the **ActionsReplayer** module to target the absolutely wrong graphical context (window, button, text input field, etc.). As a result, the whole automated sensitivity analysis process (the **Execution Loop**) may become completely broken.

Given that during a variance-based global sensitivity analysis hundreds of simulation runs may be happening continuously over hours, it is absolutely unacceptable that a sudden failure on the side of the OS or the simulator completely destroys the progress of the entire parametric sweep. To mitigate this risk, our implementation contains two main countermeasures.

The first of them is the mandatory timeout in the **VisionEngine**'s Simulation Completion Indicator waiting loop, which user must specify during the recording phase. If the SCI does not appear in this allotted time, **VisionEngine** will think that something went wrong, the **ExecutionLoop** will be suspended, any partial results of the last incomplete simulation run will be erased, and the corresponding error message will be shown to the user. No further actions will be made by our application until the user reacts to this error message. The user may then investigate and fix the issue, re-open the sensitivity analysis project as an existing one and continue simulation runs.

The second countermeasure includes saving the project state (metadata and the sample dataset $(\mathbf{p}_i, \tilde{f}_i)$) to disk at any change in them. When the user modifies something in the project

configuration during the recording phase, or when a new simulation result $\hat{\hat{f}}_i$ becomes available during the execution phase, it is immediately saved on disk. Because of that, when some failure occurs, the user can entirely close our SA automation application (for example, before rebooting the PC) and then launch it again when the failure is fixed. Our application will seamlessly read the serialized JSON metadata and the sample dataset, identify the precise integer index of the last successful simulation run (of the last stored $\hat{\hat{f}}_i$), and recommence the execution loop from it.

These two countermeasures constitute the failure recoverability guarantee of our application. They ensure that long parametric sweeps can be confidently executed in inherently volatile desktop environments.

Chapter 5

Evaluation

The transition from the architectural design and programmatic implementation of our sensitivity analysis automation software to its real-world use requires a rigorous testing (evaluation, validation) phase. The primary objective of the evaluation is to quantitatively and qualitatively assess the reliability, mathematical accuracy, and practical utility of the developed Minimum Viable Product.

5.1 Validation Framework

To systematically address the central research questions regarding the accuracy and usability of our sensitivity analysis automation software, we subjected it to a comprehensive, two-stage evaluation.

The first stage involved testing our application using a special software, which imitates a real-world simulator with simple GUI. We have developed such software (we call it "synthetic benchmark") on our own from scratch specifically for this evaluation phase. This software accepts some input parameters which user can specify via GUI text input fields and produces a color-coded picture. The true underlying scalar field $f(x, y)$ is known to us (since we programmed it), just as its exact sensitivities (partial derivatives at any point and Sobol indices). By evaluating our SA automation software on such a benchmark, we wanted to make sure it can handle the basic simulator-type GUI and assess how precisely it calculates the derivatives and Sobol indices.

After the accuracy of the sensitivity indices calculated by our application has been verified, we proceed to the second stage of the evaluation. It involves testing our SA automation software on multiple real, industrial-grade GUI simulation environments. This evaluation stage is mainly devoted to testing the robustness of the Robotic Process Automation implemented in our application in the real-world conditions when a simulation software can have a rich, complex, multi-window, diverse graphical user interface.

5.2 Synthetic Benchmark Analysis

Our synthetic benchmark (Figure 5.1) "simulates" the Ishigami function [53], which is a well-known test function in the sensitivity analysis literature due to its high non-linearity, non-monotonicity

and interaction effects [54–56]. It is defined mathematically as:

$$I(x_1, x_2, x_3, a, b) = \sin(x_1) + a \sin^2(x_2) + bx_3^4 \sin(x_1) \quad (5.1)$$

The user can enter the values for x_1 , x_2 and x_3 into the corresponding GUI text input fields. Then she can configure the remaining parameters a and b by specifying the Plot Parameters: a_{min} , a_{max} , b_{min} , b_{max} , and the number of bins for a and b . The parameters a and b of the Ishigami function are treated by us as "geographical" parameters: they define which "region" of the simulation will be shown in the color-coded picture. Just like in real-world simulation software, a user, for example, can specify which part of the building, the climate in which she is simulating, she wants to be displayed in the output color-coded image. The color-coded picture is drawn in these "geographical" parameters as its coordinates (a, b) for the specified ranges $[a_{min}, a_{max}]$ and $[b_{min}, b_{max}]$. The true scalar field produced by our synthetic benchmark is $f(a, b)$. At each point (a, b) this scalar field is equal to the Ishigami function for these values of a and b and for the rest variables x_1 , x_2 , x_3 equal to the ones specified by the user in the corresponding input fields mentioned above. The number of bins for a and b specifies into how many partitions the intervals $[a_{min}, a_{max}]$ and $[b_{min}, b_{max}]$ are split. In each such partition, the values of a and b are constant. It is worth mentioning that there is no point in specifying the number of bins for a and b larger than the width and height of the color-coded image in pixels on the screen on which it is displayed — it will not increase the level of detail beyond that limit.

Then the user can specify the colormap settings: which one to use and its min and max bounds in terms of the minimum and maximum function values. Finally, she can configure the minimum and maximum simulation time (it will be sampled uniformly from this range to imitate the random solver finish time) and click the "Run simulation" button, which will, after the mentioned randomly-sampled delay, produce the matrix M of the Ishigami function values $M_{ij} = I(x_1, x_2, x_3, a_j, b_i)$. This matrix has the shape $N_b \times N_a$, where N_a and N_b are the number of bins for a and b respectively. The parameters x_1 , x_2 , x_3 are constant for all the matrix, the parameters a_j and b_i vary as we move along the columns and rows of this matrix. The whole matrix is then transformed into the matrix of RGB pixels by applying the colormap specified by the user with its minimum and maximum function values taken as the corresponding minimum and maximum elements of the matrix M . The "Colormap min" and "Colormap max" set by the user are ignored at this point. If the user wants to enforce these specified limits (as well as potentially different values for the plot parameters a and b — she can change them as well), she should click the "Update plot" button. If she wants to reset everything, she should click the "Reset parameters" button.

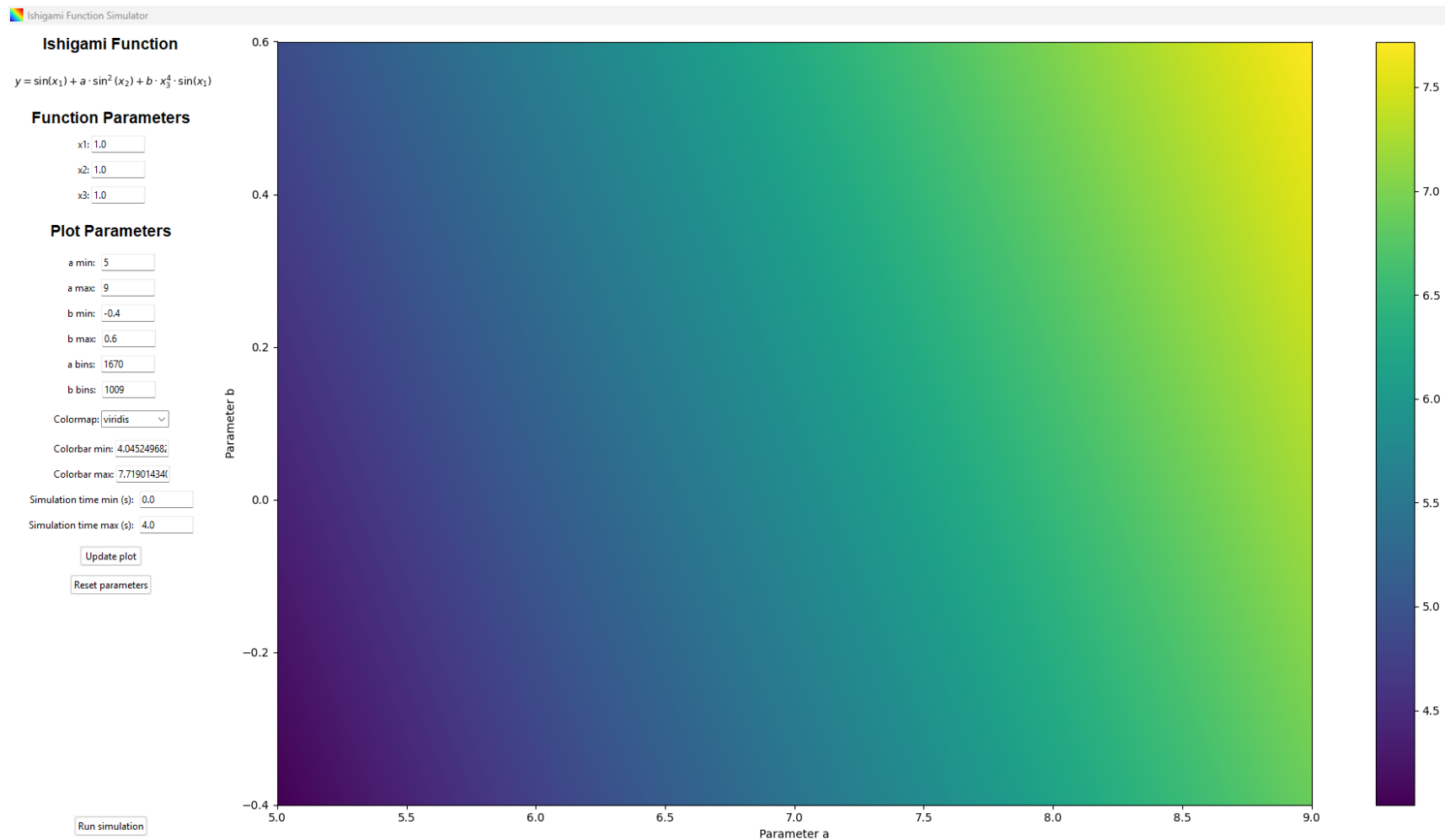


Figure 5.1: Synthetic Benchmark Application

Extensive experiments were conducted for both localized gradient computation and global Sobol index estimation. The experiments for the gradient calculation were split into two subgroups: one utilizing a dense binning strategy (with maximal number of bins, e.g., 1670 for a and 1009 for b — exactly the width and height of the color-coded image in pixels on the testing PC's monitor), and another utilizing few bins. The dense binning configuration produces a highly smoothed, continuous color gradient across the canvas, while the "few bins" configuration produces a brick-wall-like picture consisting of large, contiguous rectangular blocks of constant color (Figure 5.2). Because our sensitivity automation software extracts a multi-pixel Region of Interest and averages the reconstructed scalars, the huge reconstruction errors of individual scalars can cancel out and go unnoticed. Exactly because of this, we have conducted the experiments with few number of bins, while selecting the ROI entirely within a single bin. This leads to zero averaging effects: all the pixels are the same within a single bin, all the reconstructed scalars are also, thus, the same. This gives us the opportunity to evaluate the colormap inversion module standalone, without averaging effects interfering.

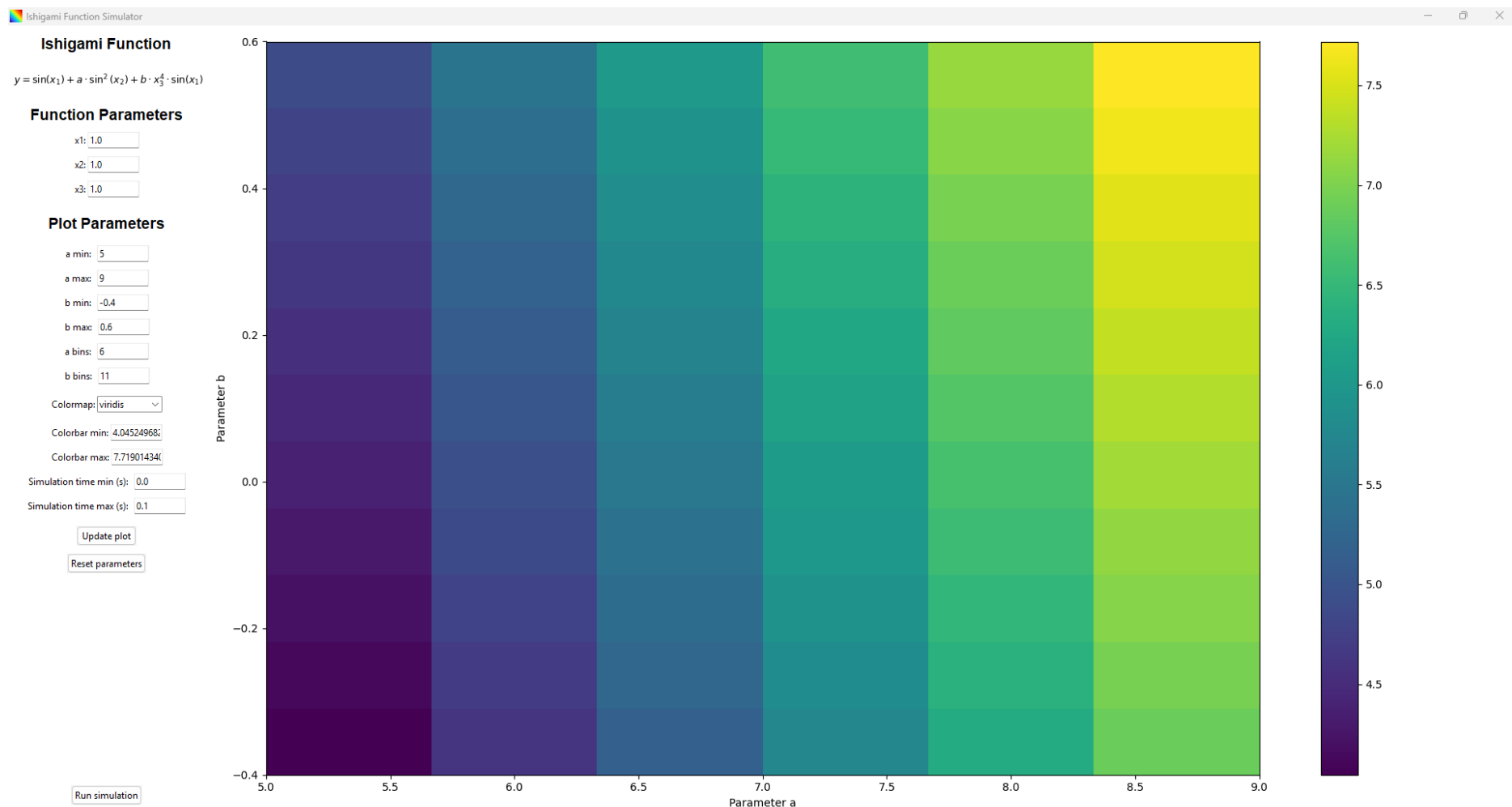


Figure 5.2: Synthetic Benchmark Application — Few Picture Bins

5.2.1 Local Gradient Calculation Tests

The commonly-used values for parameters a and b of the Ishigami function are 7 and 0.1 respectively [6, 57]. We evaluated the gradient calculation accuracy of our application for different values of a and b around these reference values (and using different values for x_1, x_2, x_3 as well). The comprehensive results of these tests are shown in Tables 5.1 and 5.2. When we are conducting tests with a "few bins" configuration, the values of a and b are constant within the ROI, as was said earlier. For the dense binning configuration, the ROI spans some rectangular region in coordinates a and b . That is why for the first 3 tests you can see the scalar values in the "a" and "b" columns of the result tables, and for the rest three — intervals.

Let us explain what each column contains, starting with the first table. *True Partial Derivative* (let us call it ∂) is self-explanatory: it is the true partial derivative value of the true Ishigami function at the given point (a, b, x_1, x_2, x_3 in the parameter space). For example, in the first experiment (an experiment is a single choice of a, b, x_1, x_2, x_3), the value 0.3241813835 is the value of $\frac{\partial}{\partial x_1} I(x_1, x_2, x_3, a, b)|_{x_1=1, x_2=1, x_3=1, a=5, b=0.4}$. *True Finite Difference Scheme Value* (let us call it ∂_{fd}) is the estimate of ∂ via the second-order finite difference scheme using the true Ishigami function in the numerator and the step size $h = 0.1$. For example, for the same experiment:

$$\partial_{fd} = \frac{I(x_1 + h, x_2, x_3, a, b) - I(x_1 - h, x_2, x_3, a, b)}{2h} \Big|_{x_1=1, x_2=1, x_3=1, a=5, b=0.4} = 0.3236413513 \quad (5.2)$$

How to calculate the True Partial Derivative and the *True Finite Difference Scheme Value* when we have not a single value for a and b , but an interval for each of them (the last three experiments)? For given intervals $[a_{min}, a_{max}]$ and $[b_{min}, b_{max}]$, we do not change the formulas above, but simply replace the original $I(x_1, x_2, x_3, a, b)$ in them with the corresponding mean-value-function $\bar{I}$ over the rectangle defined by $a_{min}, a_{max}, b_{min}, b_{max}$:

$$\bar{I}(x_1, x_2, x_3) = \frac{\int_{b_{min}}^{b_{max}} \int_{a_{min}}^{a_{max}} I(x_1, x_2, x_3, a, b) da db}{(a_{max} - a_{min})(b_{max} - b_{min})} \quad (5.3)$$

Moving on to the next columns. *Partial Derivative Estimate* (let us call it $\hat{\partial}$) is the output of our SA automation application. It is the same as ∂_{fd} , but using the reconstructed scalars in the numerator instead of the true Ishigami function values. *Conf. Interval* is the 100% confidence interval value C , so that ∂_{fd} is guaranteed to lie within the interval $[\hat{\partial} - C, \hat{\partial} + C]$. This $C = \frac{W}{2h}$ is derived from the colormap input domain quantization (see section 4.6) using interval arithmetic over the second-order finite difference quotient:

$$\frac{\left[-\frac{W}{2}, \frac{W}{2}\right] - \left[-\frac{W}{2}, \frac{W}{2}\right]}{2h} = \frac{[-W, W]}{2h} = \left[-\frac{W}{2h}, \frac{W}{2h}\right] \quad (5.4)$$

Here $[-\frac{W}{2}, \frac{W}{2}]$ is the 100% confidence interval for a single reconstructed scalar function value error, $W = w \times (f_{max} - f_{min})$ is the width of a colormap bin, scaled to the function value range used with this colormap.

Conf. Interval, % equals to $\frac{C}{\hat{\partial}} \times 100$.

Now let us go to the second Error Metrics table. *Abs. Error, %* (let us call it e_{fd}) equals to $\left| \frac{\partial_{fd} - \hat{\partial}}{\partial_{fd}} \right| \times 100$. *Conf. Int. Resp.* (Confidence Interval Respected) is a flag column to check whether ∂_{fd} is indeed inside $[\hat{\partial} - C, \hat{\partial} + C]$ (should always be "Yes"). *Total Abs. Error, %* (let us call it e_{true}) is the same as *Abs. Error, %*, replacing ∂_{fd} with ∂ . *Avg. / Max. Total Abs. Error, Local, %* are the mean and maximum of the three e_{true} 's (we have one e_{true} for each parameter x_1, x_2, x_3) within a single experiment. Finally, *Avg. / Max. Total Abs. Error, Global, %* are the mean and maximum of all the e_{true} 's across all the experiments.

Table 5.1: Synthetic Benchmark Experiments — Gradient (Part 1: Estimates and True Values)

a	b	Param. Name	Param. Value	Partial Derivative Estimate	Conf. Interval	Conf. Interval, %	True Finite Difference Scheme Value	True Partial Derivative
5	-0.4	x1	1	0.32469487	0.07167111	22.07337307	0.3236413513	0.3241813835
		x2	1	4.5232944	0.07142918	1.579140637	4.516237782	4.546487134
		x3	1	-1.35317738	0.07274096	5.375567245	-1.359817111	-1.346353576
7	0.1	x1	-1	1.38558954	0.32624143	23.54531559	1.402445856	1.404785995
		x2	2	-5.58941676	0.32704618	5.851168271	-5.262370585	-5.297617467
		x3	2	-2.6467155	0.33150118	12.5250024	-2.699438919	-2.692707151
9	0.6	x1	-2	-0.66678412	0.08226644	12.33779233	-0.6647257683	-0.6658349385
		x2	-2	6.76875158	0.08184621	1.209177335	6.765905038	6.811222458
		x3	-1	2.17770784	0.08342253	3.830749399	2.204136963	2.182313824
[5, 7]	[-0.4, 0]	x1	0	0.98571407	0.0554401	5.624359202	0.9858549894	0.9875
		x2	-1	-5.4258923	0.0549942	1.013551264	-5.419485338	-5.455784561
		x3	-0.5	0	0.05531824	-	0	0
[6, 8]	[0, 0.2]	x1	2	-1.04841954	0.28273585	26.96781577	-1.080179374	-1.081981775
		x2	0	0	0.2849341	-	0	0
		x3	-2	-2.76735427	0.28841955	10.42221277	-2.917026145	-2.909751766
[7, 9]	[0.4, 0.6]	x1	-0.5	0.87612066	0.07773408	8.872531325	0.8761206554	0.8775825619
		x2	-1.5	-1.11950289	0.07696322	6.874767425	-1.121448703	-1.128960064
		x3	0	0	0.07773502	-	0	0

Table 5.2: Synthetic Benchmark Experiments — Gradient (Part 2: Error Metrics)

a	b	Param. Name	Param. Value	Abs. Error, %	Conf. Int. Resp.	Total Abs. Error, %	Avg. Total Abs. Error, Local, %	Max Total Abs. Error, Local, %	Avg. Total Abs. Error, Global, %	Max Total Abs. Error, Global, %		
5	-0.4	x1	1	0.3255204237	Yes	0.1583948077	0.3917850172	0.5101242662	1.136883213	5.508123126		
		x2	1	0.1562499211	Yes	0.5101242662						
		x3	1	0.4882812103	Yes	0.5068359778						
7	0.1	x1	-1	1.201922739	Yes	1.366503889	2.860878312	5.508123126				
		x2	2	6.21480699	Yes	5.508123126						
		x3	2	1.953125106	Yes	1.708007919						
9	0.6	x1	-2	0.309654265	Yes	0.1425550794	0.3257191372	0.6235426611				
		x2	-2	0.04207185992	Yes	0.6235426611						
		x3	-1	1.199068982	Yes	0.2110596711						
[5, 7]	[-0.4, 0]	x1	0	0.01429412931	Yes	0.1808536709	0.2429179962	0.5479003178				
		x2	-1	0.1182208488	Yes	0.5479003178						
		x3	-0.5	0	Yes	0						
[6, 8]	[0, 0.2]	x1	2	2.94023699	Yes	3.101922398	2.665241532	4.893802197				
		x2	0	0	Yes	0						
		x3	-2	5.13097476	Yes	4.893802197						
[7, 9]	[0.4, 0.6]	x1	-0.5	0.00000052139	Yes	0.1665828326	0.3347572836	0.8376890182				
		x2	-1.5	0.1735088298	Yes	0.8376890182						
		x3	0	0	Yes	0						

The experiment results demonstrate that our application performs quite accurate estimations, with relative absolute errors being about 1.14% on average and about 5.51% in the worst case. In addition, in every single experimental trial, every estimated partial derivative fell within the error bounds derived from the colormap quantization. Coupled together, these two results prove that the implementation of the colormap inversion and its error quantification was performed correctly.

Beyond measuring the accuracy of the gradient calculation, we also evaluated how precisely our application computes the basic statistics over reconstructed simulation function values (Tables 5.3 and 5.4). Since the accuracy of this statistics calculation depends primarily on the same colormap inversion module, we expected results similar to the ones of the gradient experiments, and we got them: the relative absolute errors are about 0.95% on average and about 4.7% in the worst case, which again demonstrates the correctness of our implementation.

5.2.2 Global Sobol Index Calculation Tests

The tests measuring how accurately our sensitivity analysis automation software computes the Sobol indices were performed only for the "few bins" configuration and, thus, scalar (not interval) values for a and b . This is because the task to obtain a precise ground-truth estimator of S_1 , S_T and S_2 for the case when a and b are not scalars, but intervals, is highly nontrivial. This is not a significant lapse by any means: the region-based tests for a and b were performed for the gradient calculation and did not demonstrate significant difference to the point-based tests, which is explainable, because the error bounds of the average reconstructed scalar (dense binning configuration) are the same as for the one reconstructed scalar ("few bins" configuration):

$$\frac{1}{B} \sum_{b=1}^B \left[-\frac{W}{2}, \frac{W}{2} \right] = \left[-\frac{W}{2}, \frac{W}{2} \right] \quad (5.5)$$

Here B is the number of bins.

The experiment results are shown in Tables 5.5, 5.6, 5.7, 5.8, 5.9. For the S_1 index the relative error is about 0.42% in the average case and 0.92% in the worst case, for the S_T index it is 0.80% and 1.21% correspondingly, for the S_2 — 0.71% and 3.12%. A single error spike in the S_2 calculation is still quite small and looks like a rare outlier. The conclusions of the experiments are the same as those of the gradient calculation tests: high precision, which fully validates the Sobol indices calculation logic implemented in our sensitivity analysis automation application. The statistics calculation errors are also very small and they are significantly smaller than in the gradient calculation experiments, which is easily explainable: the number of simulation runs for the gradient calculation in our case was just 6, while for Sobol indices it was 32, so all the statistics were computed for the sample which was more than 5 times larger. Thus, the errors in individual reconstructed scalars had a much bigger opportunity to cancel out and average out.

Table 5.3: Synthetic Benchmark Experiments — Gradient (Part 3: Statistics)

a	b	x1	x2	x3	Statistic	Estimated Value	True Value	Abs. Error, %
5	-0.4	1	1	1	min	3.579325084	3.572887828	0.1801695577
					max	4.483983965	4.476135384	0.1753427865
					mean	4.037947941	4.030753233	0.1784953821
					median	4.049894483	4.042727372	0.1772840535
					STD	0.2994537461	0.2991439116	0.1035737306
					MAD	0.1353177379	0.1359817111	0.48828125
7	0.1	-1	2	2	min	2.995971015	3.028088314	1.060646067
					max	4.113854367	4.080562431	0.8158663533
					mean	3.603756489	3.581635963	0.6176095666
					median	3.643482266	3.610858122	0.9035011138
					STD	0.4028460763	0.3852620653	4.5641688
					MAD	0.2646715503	0.2699438919	1.953125
9	0.6	-2	-2	-1	min	5.256516198	5.251297813	0.09937324958
					max	6.610266514	6.604478821	0.08763285278
					mean	5.961838009	5.958469398	0.05653484588
					median	6.001514637	5.99378873	0.1288985498
					STD	0.452786777	0.4529815094	0.04298903908
					MAD	0.2177707843	0.2204136963	1.199068843
[5, 7]	[-0.4, 0]	0	-1	-0.5	min	3.685958712	3.681606284	0.1182208995
					max	4.771137173	4.765503352	0.1182208994
					mean	4.24517709	4.240145279	0.1186707164
					median	4.253463054	4.24844051	0.1182208995
					STD	0.3490181597	0.3486202398	0.1141413639
					MAD	0.09857140698	0.09858549894	0.014294152
[6, 8]	[0, 0.2]	2	0	-2	min	2.035042478	2.094302926	2.829602523
					max	2.588513332	2.677708155	3.331013622
					mean	2.317609776	2.390769373	3.0600859
					median	2.361230156	2.433940287	2.987342454
					STD	0.1907239381	0.2001197802	4.695109137
					MAD	0.1048419537	0.1080179374	2.940237245
[7, 9]	[0.4, 0.6]	-0.5	-1.5	0	min	7.275984122	7.289463824	0.1849203569
					max	7.5567404	7.570551644	0.1824337876
					mean	7.441258928	7.455022916	0.1846270499
					median	7.466714038	7.480520477	0.1845652172
					STD	0.09894829323	0.09906541409	0.1182257825
					MAD	0.05918421542	0.05921302578	0.04865544471

Table 5.4: Synthetic Benchmark Experiments — Gradient (Part 4: Statistics — Aggregated Errors)

Statistic	Avg. Abs. Error, %	Max Abs. Error, %	Global Avg. Abs. Error, %	Global Max Abs. Error, %
min	0.7454887756	2.829602523	0.949476347	4.695109137
max	0.7850850502	3.331013622		
mean	0.7026705768	3.0600859		
median	0.7499687146	2.987342454		
STD	1.606367975	4.695109137		
MAD	1.107276989	2.940237245		

Table 5.5: Synthetic Benchmark Experiments — Sobol (S_1 Estimates)

a	b	Param. Name	S1 Estimate	S1 True	Abs. Error, %	Avg. Abs. Error, Local, %	Max Abs. Error, Local, %	Avg. Abs. Error, Global, %	Max Abs. Error, Global, %
5	0.1	x1	0.14845655	0.148284	0.1163645437	0.3757291143	0.9238633825	0.4242555739	0.9238633825
		x2	-0.82535665	-0.826075	0.08695941652				
		x3	-0.1951681	-0.196988	0.9238633825				
7	0.5	x1	0.13603113	0.136405	0.2740881932	0.3718586966	0.4310906028		
		x2	-0.41969291	-0.42151	0.4310906028				
		x3	-0.31175828	-0.313043	0.4103972937				
9	-0.4	x1	0.0230313	0.023178	0.6329277763	0.525178911	0.6329277763		
		x2	0.50429834	0.502303	0.3972383203				
		x3	-0.19607956	-0.195016	0.5453706363				

Table 5.6: Synthetic Benchmark Experiments — Sobol (S_T Estimates)

a	b	Param. Name	ST Estimate	ST True	Abs. Error, %	Avg. Abs. Error, Local, %	Max Abs. Error, Local, %	Avg. Abs. Error, Global, %	Max Abs. Error, Global, %
5	0.1	x1	0.09908253	0.099028	0.05506523408	0.6451556681	0.9637566602	0.803108717	1.206459016
		x2	0.39011953	0.386576	0.9166451099				
		x3	0.0169144	0.017079	0.9637566602				
7	0.5	x1	0.04484401	0.045082	0.527904707	0.8814997776	1.206459016		
		x2	0.05363338	0.054126	0.9101356095				
		x3	0.03013203	0.0305	1.206459016				
9	-0.4	x1	0.00730461	0.00725	0.7532413793	0.8826707052	1.063910056		
		x2	0.09429566	0.093303	1.063910056				
		x3	0.02052513	0.020356	0.8308606799				

Table 5.7: Synthetic Benchmark Experiments — Sobol (S_2 Estimates)

a	b	Param. Pair	S2 Estimate	S2 True	Abs. Error, %	Avg. Abs. Error, Local, %	Max Abs. Error, Local, %	Avg. Abs. Error, Global, %	Max Abs. Error, Global, %
5	0.1	(x1, x2)	0.43391183	0.431687	0.5153803566	0.5206573051	0.6988344233	0.7134328871	3.122132401
		(x1, x3)	-0.50114272	-0.499406	0.3477571355				
		(x2, x3)	1.26169799	1.252942	0.6988344233				
7	0.5	(x1, x2)	0.24358955	0.244868	0.5220976199	0.4241644121	0.5220976199		
		(x1, x3)	0.11064855	0.111089	0.3964839003				
		(x2, x3)	0.42626504	0.427779	0.3539117161				
9	-0.4	(x1, x2)	-0.0424791	-0.041193	3.122132401	1.195476944	3.122132401		
		(x1, x3)	0.72928014	0.727095	0.3005301921				
		(x2, x3)	-0.29483706	-0.294355	0.163768239				

Table 5.8: Synthetic Benchmark Experiments — Sobol (Statistics)

a	b	Statistic	Estimated Value	True Value	Abs. Error, %
5	0.1	min	-5.551500055	-5.517495173	0.616310132
		max	5.408908994	5.410178396	0.02346321358
		mean	2.091892019	2.091732405	0.007630712905
		median	2.93143666	2.922303088	0.3125470622
		STD	2.318692947	2.317606957	0.04685823683
		MAD	1.423823626	1.414756488	0.640897408
7	0.5	min	-25.80001733	-25.7279638	0.2800592102
		max	9.190850396	9.181735717	0.09926967073
		mean	1.612540469	1.615915756	0.2088776313
		median	3.207164637	3.20088133	0.1962992861
		STD	7.112821079	7.093177895	0.2769306475
		MAD	2.119824753	2.121145588	0.06226985524
9	-0.4	min	-2.450895614	-2.44686297	0.1648087295
		max	25.86167818	25.96615132	0.4023435718
		mean	5.709595302	5.721772316	0.2128189209
		median	5.147075871	5.154715953	0.1482153853
		STD	6.190620063	6.212398415	0.3505627075
		MAD	4.58337373	4.59041363	0.1533609073

Table 5.9: Synthetic Benchmark Experiments — Sobol (Statistics — Aggregated Errors)

Statistic	Avg. Abs. Error, %	Max Abs. Error, %	Global Avg. Abs. Error, %	Global Max Abs. Error, %
min	0.3537260239	0.616310132	0.2335290716	0.640897408
max	0.1750254854	0.4023435718		
mean	0.1431090884	0.2128189209		
median	0.2190205779	0.3125470622		
STD	0.2247838639	0.3505627075		
MAD	0.2855093902	0.640897408		

5.3 Real-World Case Studies

While the Ishigami synthetic benchmark application has a very basic and simple 1-window GUI, real modern simulation software has many different windows, and in order to fully carry out one simulation, the user may have to go to many different places and separate graphical menus of the software and click on many different buttons in order to configure all the required settings. Also, as is often the case, the post-processing of the simulation results (their visualization as a color-coded image) may be done by a completely separate software, such as ParaView [58]. Our sensitivity automation application should be able to handle such complicated, multi-stage GUI workflows — and this is exactly what we wanted to test in this second phase of our SA automation software evaluation. For this, we have selected three different examples of modern GUI simulation software — ElmerFEM [59], SimFlow [60] and Ansys Electronics [61] — each with a different physical simulation project. For each of these three cases, we have carried out

the gradient sensitivity analysis. We avoided Sobol SA for this testing phase because it just takes longer time to complete and does not introduce anything new in terms of the GUI workflow.

5.3.1 ElmerFEM: Temperature Distribution

ElmerFEM is a robust open-source multiphysical simulation solver widely utilized in academic and industrial settings. The test case configured within the ElmerFEM's GUI is modeling the steady-state temperature distribution across a 2-dimensional L-shaped brace subjected to internal heating and constant boundary temperature. In Figure 5.3 it is shown how the project looks in ElmerFEM, the description of the project can be found in [62].

A specific architectural challenge posed by ElmerFEM was the absence of a built-in post-processing renderer capable of visualizing the simulation results as color-coded images. To successfully deal with this, our automation application was guided (during the recording phase) to run the ElmerFEM numerical solver, wait for it to finish via template matching, and subsequently launch ParaView to visualize the solver's output field $f(x, y)$ by selecting the corresponding ElmerFEM's simulation output file via the file system dialog. In ParaView, the temperature distribution across the brace was visualized using the "Turbo" colormap (Figure 5.4).

Our application successfully recorded all the long and multi-stage workflow of a single simulation run in ElmerFEM, successfully executed all the simulation runs required for the sensitivity analysis and extracted the visual output from the color-coded images in ParaView (the Region of Interest which we were analyzing the sensitivity at in this project is outlined in Figure 5.5 as a red rectangle). In this SA project we have analyzed the sensitivity to the four key input parameters of the temperature distribution simulation: the material's heat conductivity, the material's density, the heat source strength and the boundary temperature. The project results are shown in Figure 5.6. It can be seen that we have a small negative partial derivative ($-\epsilon$) for the heat conductivity, the same-in-absolute-value positive partial derivative (ϵ) for the density and the heat source, and a much larger positive partial derivative (1) for the boundary temperature.

Although in this second evaluation phase we do not have the goal to verify the correctness of the calculated sensitivity indices (it was successfully performed in the first evaluation phase), for this simple ElmerFEM project we can do it.

The governing equation and the generalized boundary condition on the domain Ω with boundary Γ are (see the ElmerFEM project description in [62]):

$$\begin{aligned} -\kappa \Delta T &= \rho f & \text{in } \Omega \\ T &= T_\Gamma & \text{on } \Gamma \end{aligned} \tag{5.6}$$

Let us define the total temperature field $T(x, y)$ as the sum of the constant boundary temperature T_Γ and a residual temperature field $u(x, y)$:

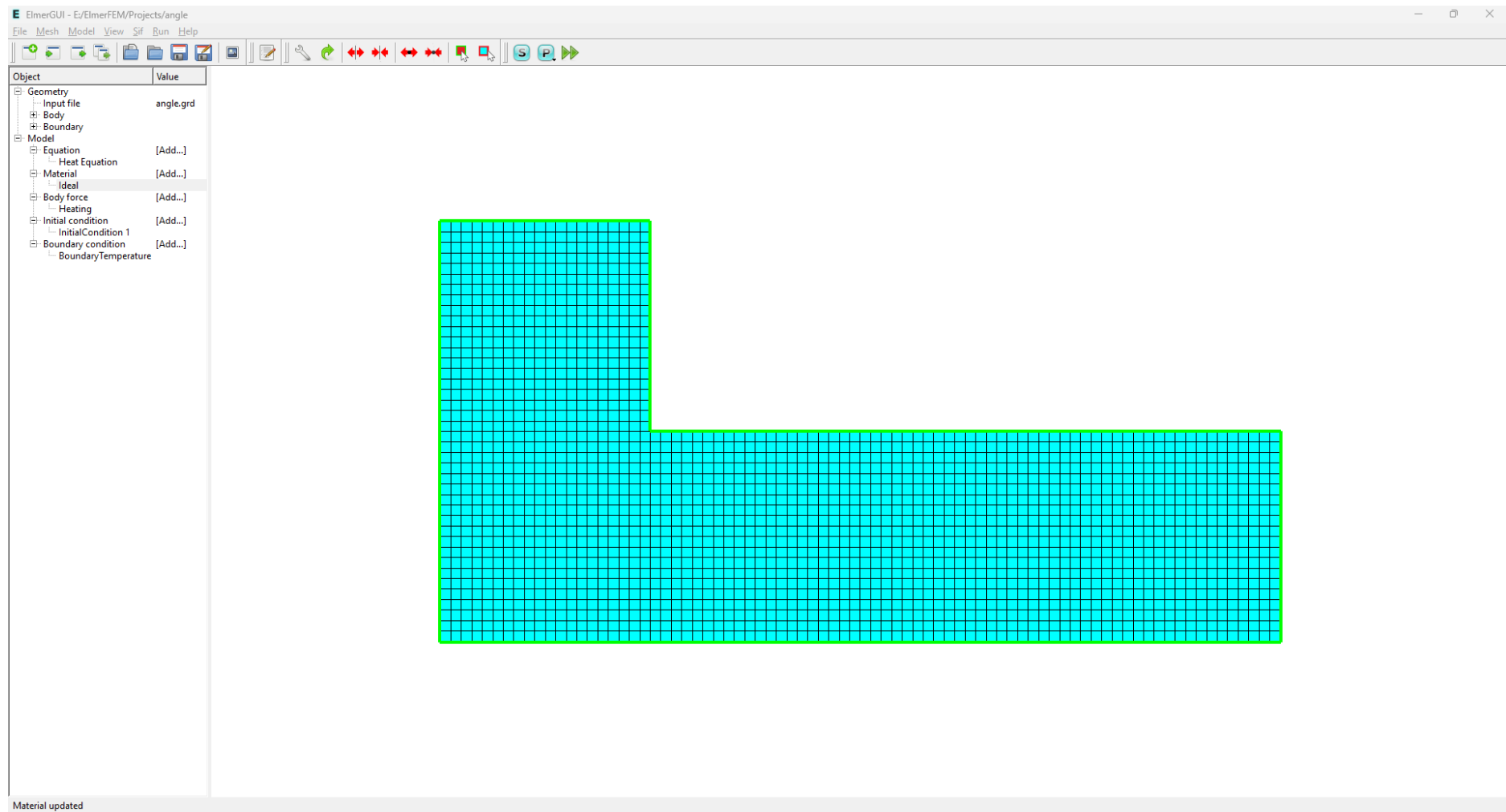


Figure 5.3: Heat Distribution Project in ElmerFEM

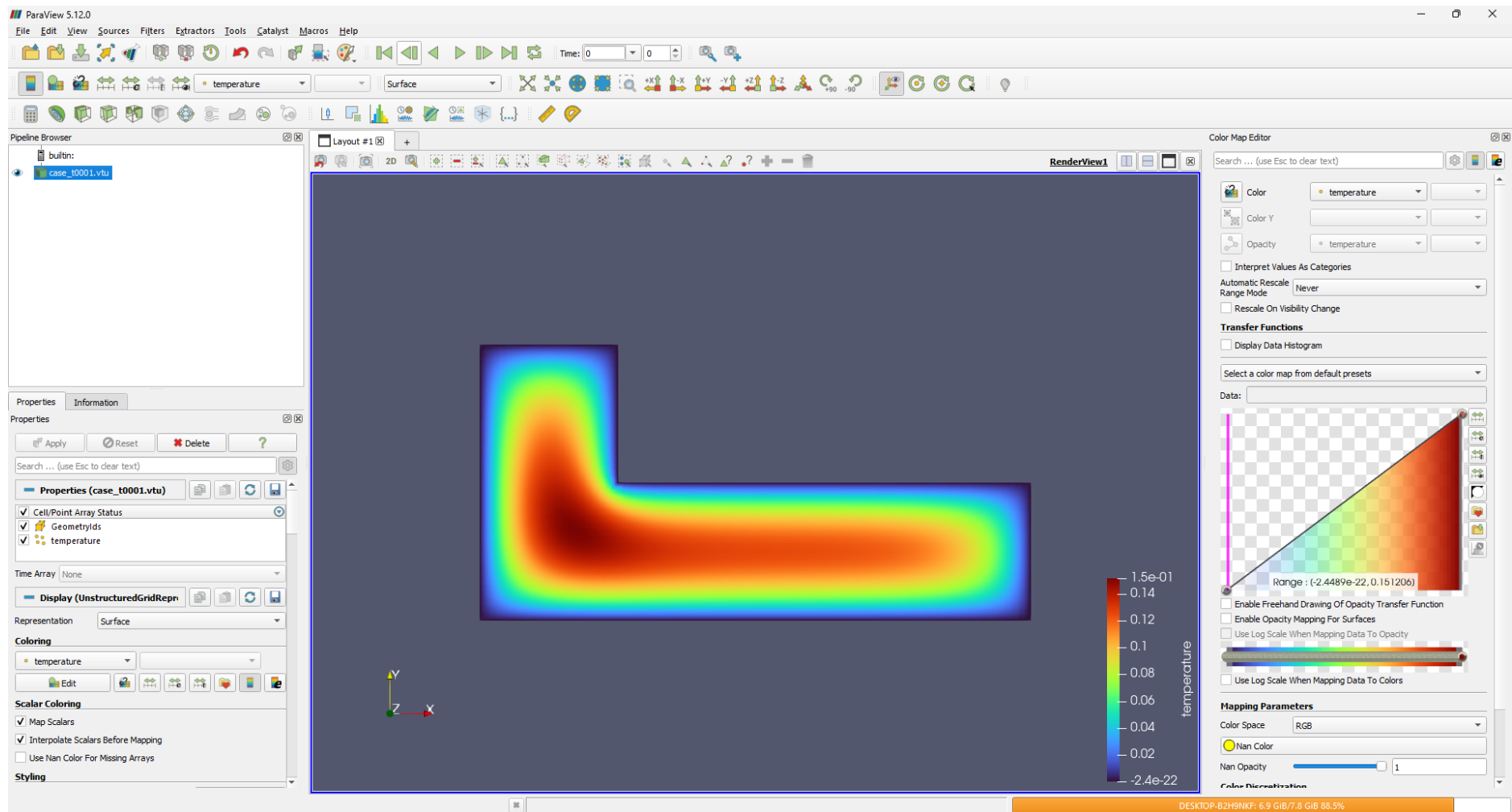


Figure 5.4: ElmerFEM Project — Temperature Distribution Visualized in ParaView

$$T(x, y) = T_\Gamma + u(x, y) \quad (5.7)$$

Substituting this transformation into the original partial differential equation gives:

$$-\kappa \Delta(T_\Gamma + u) = \rho f \quad (5.8)$$

Because the boundary temperature T_Γ is a constant, its Laplacian ΔT_Γ is zero. This simplifies the equation to:

$$-\kappa \Delta u = \rho f \implies -\Delta u = \frac{\rho f}{\kappa} \quad (5.9)$$

The boundary condition for our new variable u becomes $u = 0$ on Γ .

Now, let us introduce a function $u_0(x, y)$ which solves the following problem:

$$\begin{aligned} -\Delta u_0 &= 1 & \text{in } \Omega \\ u_0 &= 0 & \text{on } \Gamma \end{aligned} \quad (5.10)$$

We can express the temperature at any point (x, y) strictly in terms of our parameters and this function:

$$T(x, y) = T_\Gamma + \frac{\rho f}{\kappa} u_0(x, y) \quad (5.11)$$

We chose a point (a small region) very close to the bottom-left corner. According to the strong maximum principle [63], $u_0(x, y)$ must be strictly positive inside the domain and go to zero at the boundary. Therefore, at our chosen point near the boundary, $u_0(x, y)$ equates to a **small positive constant**. Let us denote this constant as ϵ , where $\epsilon > 0$.

The temperature at our specific point is governed by the algebraic relation:

$$T = T_\Gamma + \frac{\rho f}{\kappa} \epsilon \quad (5.12)$$

We evaluated the gradient of this function with respect to the parameters $(T_\Gamma, \kappa, \rho, f)$ at the point $(0, 1, 1, 1)$. Let us compute the analytical partial derivatives and plug in our values:

- **Boundary Temperature (T_Γ):**

$$\frac{\partial T}{\partial T_\Gamma} = 1 \quad (5.13)$$

Evaluated at our point (just like at any other point), this is 1.

- **Conductivity (κ):**

$$\frac{\partial T}{\partial \kappa} = -\frac{\rho f}{\kappa^2} \epsilon \quad (5.14)$$

Evaluated at $(0, 1, 1, 1)$, this becomes $-\frac{1 \cdot 1}{1^2} \epsilon = -\epsilon$.

- **Density (ρ):**

$$\frac{\partial T}{\partial \rho} = \frac{f}{\kappa} \epsilon \quad (5.15)$$

Evaluated at $(0, 1, 1, 1)$, this becomes $\frac{1}{1} \epsilon = \epsilon$.

- **Heat Source (f):**

$$\frac{\partial T}{\partial f} = \frac{\rho}{\kappa} \epsilon \quad (5.16)$$

Evaluated at $(0, 1, 1, 1)$, this becomes $\frac{1}{1} \epsilon = \epsilon$.

We can see that the gradient obtained by our sensitivity analysis application exactly matches the theoretical expectations.

5.3.2 SimFlow: Internal Pipe Pressure

SimFlow is a closed-source comprehensive pre- and post-processing GUI wrapper for the OpenFOAM solver architecture. The specific example project we have created in this environment was a Pipe Flow model [64] featuring a fully rendered 3D pipe mesh defined by discrete volumetric cubes (Figure 5.7).

For the given input parameters (velocity of water inflow at the left inlet and water pressure at the upper inlet), the solver computes the pressure distribution across the entire volume of the pipe. Our chosen experiment target this time was to estimate the sensitivity (the gradient) of the water pressure in some region of the central longitudinal cross-section of the pipe (Figure 5.8). As you can see in Figure 5.8, SimFlow has a built-in simulation results visualizer. However, it lacks many important settings, including the opportunity to select a specific non-default colormap (we wanted to do so because it was not clear for us which exactly colormap SimFlow uses as the default one). That is why we chose to use ParaView as the visualization tool for this experiment as well (Figure 5.9).

Just as in the case of ElmerFEM, our application successfully recorded and reproduced a multi-stage multi-window and even multi-application workflow, performed all the required simulation runs and computed the gradient. The Region of Interest in this project was the area just behind the vertical water inlet (Figure 5.11). Let us explain intuitively how the water pressure there should depend on the strengths (specified via velocity or pressure) of the incoming water flows at the pipe's inlets. Increasing the strength of the left horizontal inlet flow should increase the pressure in the ROI. On the other hand, the vertical inlet's flow collides with the horizontal one, diverting it downwards to the bottom of the pipe, creating a rarefied area exactly in the ROI (it can be clearly seen in Figures 5.9 and 5.11 — it is the darkest blue area in the whole color-coded picture). Increasing the strength of the vertical inlet should decrease the pressure in the ROI. This intuitive explanation perfectly matches the results of the sensitivity analysis shown in Figure 5.10.

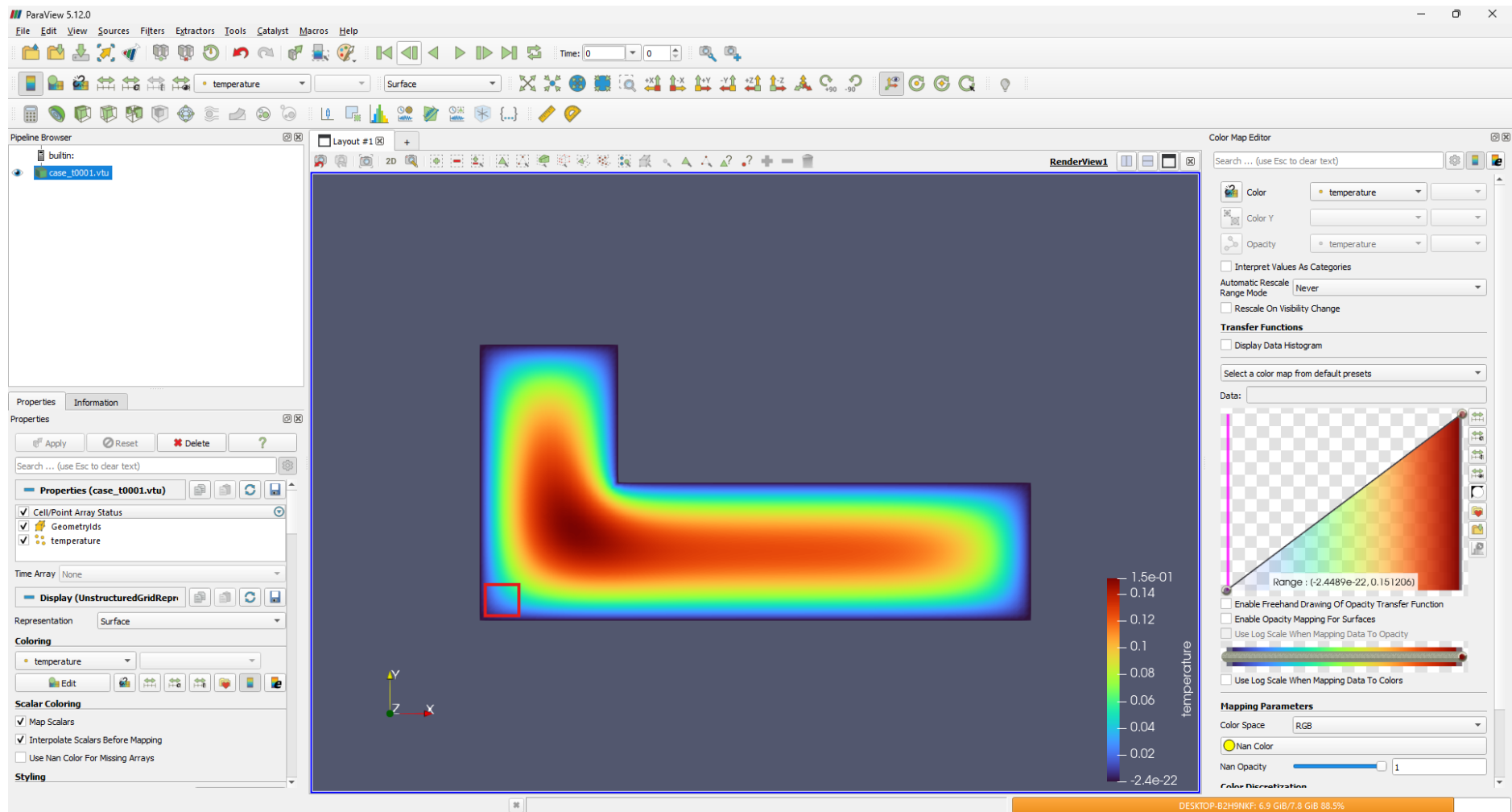


Figure 5.5: ElmerFEM Project — ROI Bounding Box

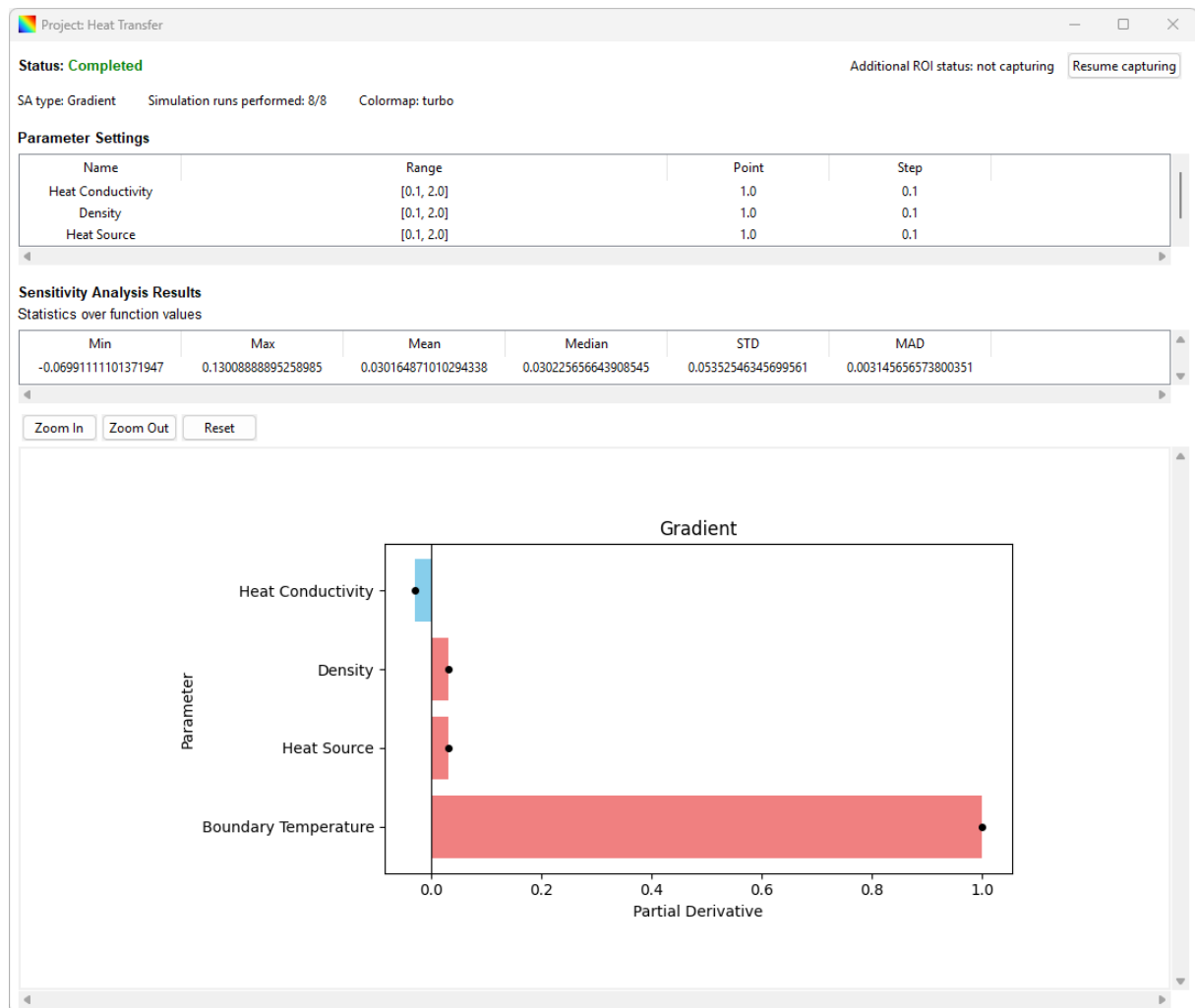


Figure 5.6: ElmerFEM SA Project Results

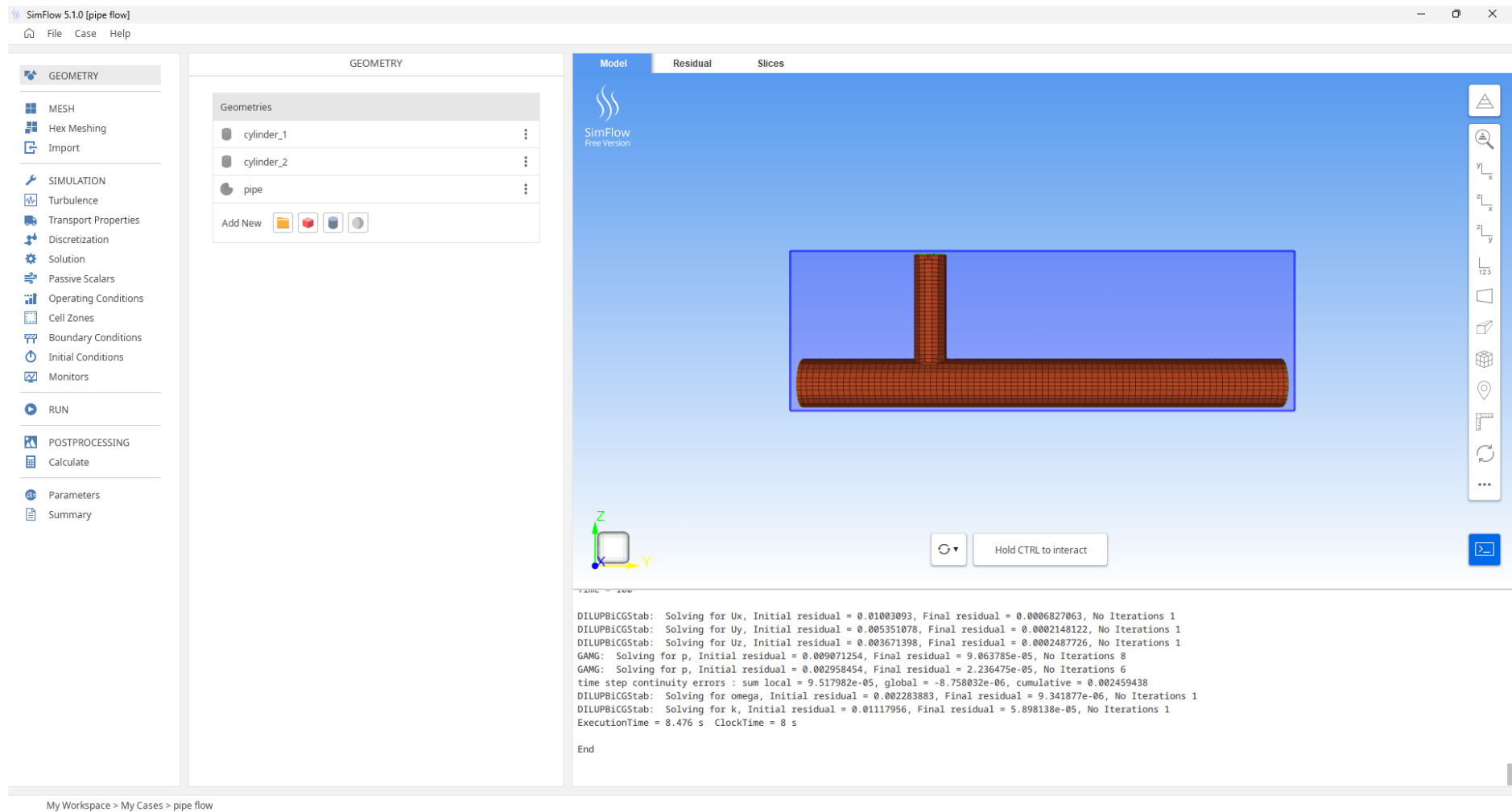


Figure 5.7: Pipe Flow Project in SimFlow

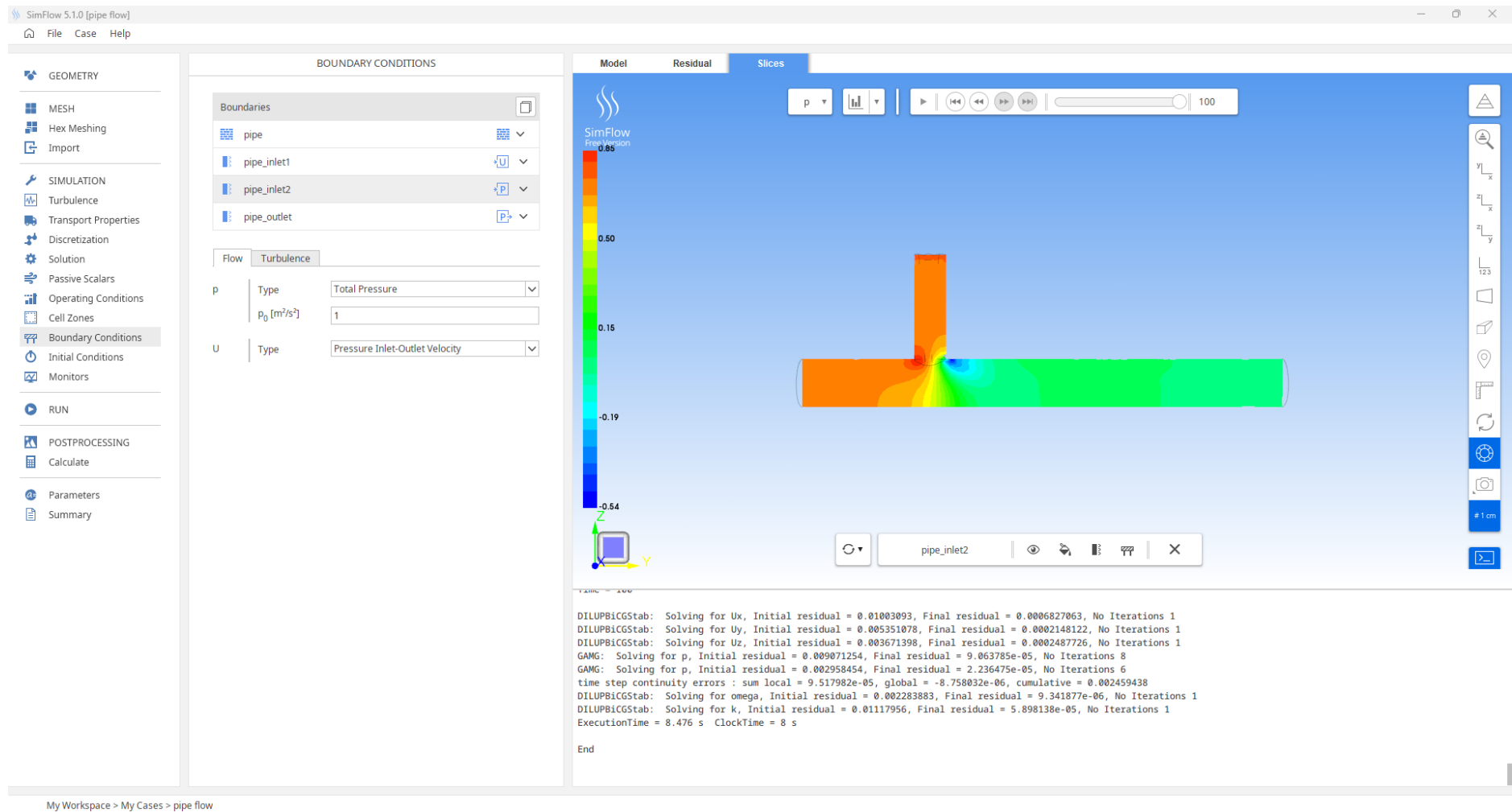


Figure 5.8: SimFlow Project — Pipe's Central Longitudinal Cross-Section Pressure Heatmap

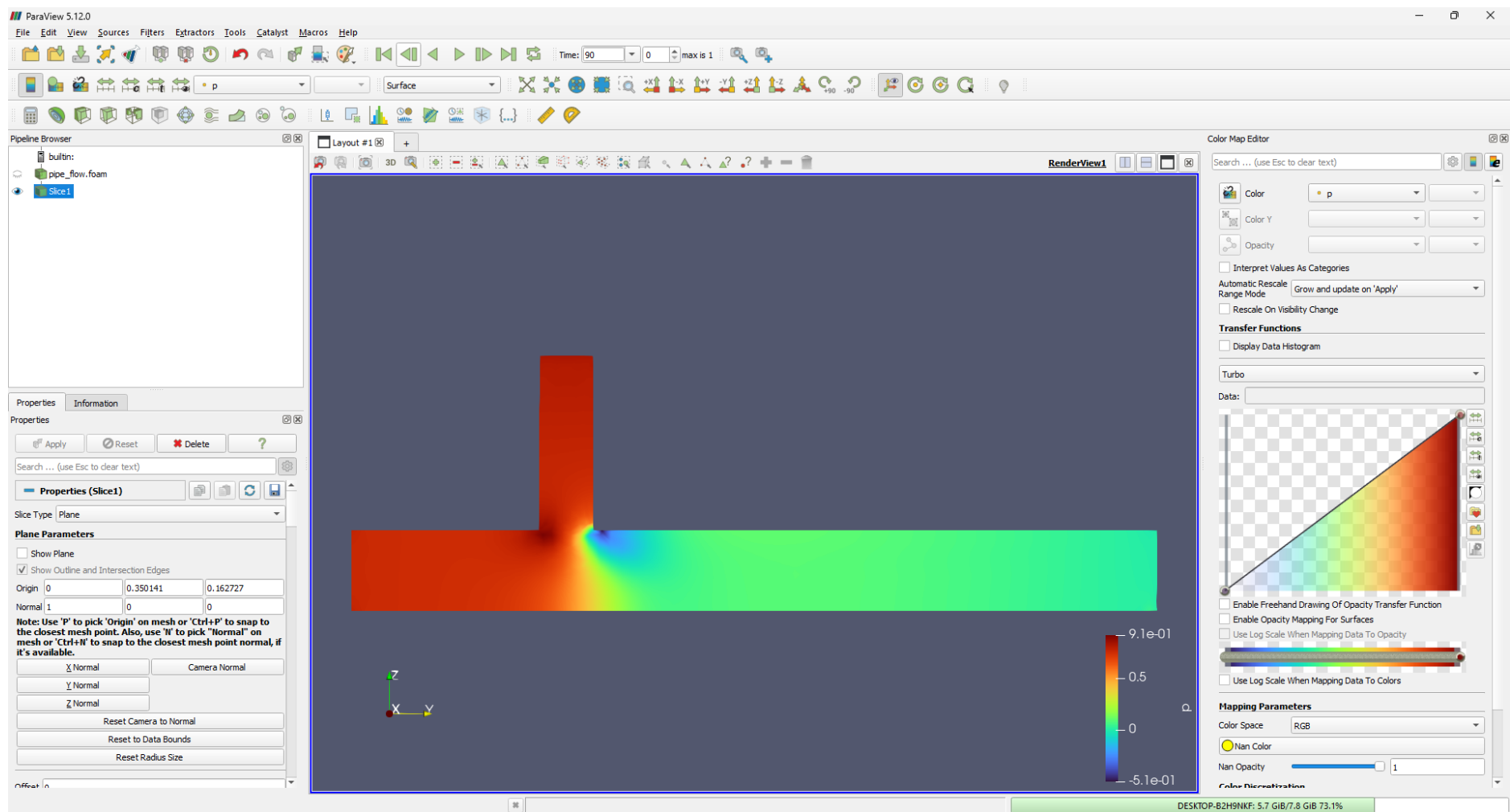


Figure 5.9: SimFlow Project — Pipe's Central Longitudinal Cross-Section Pressure Heatmap (ParaView)

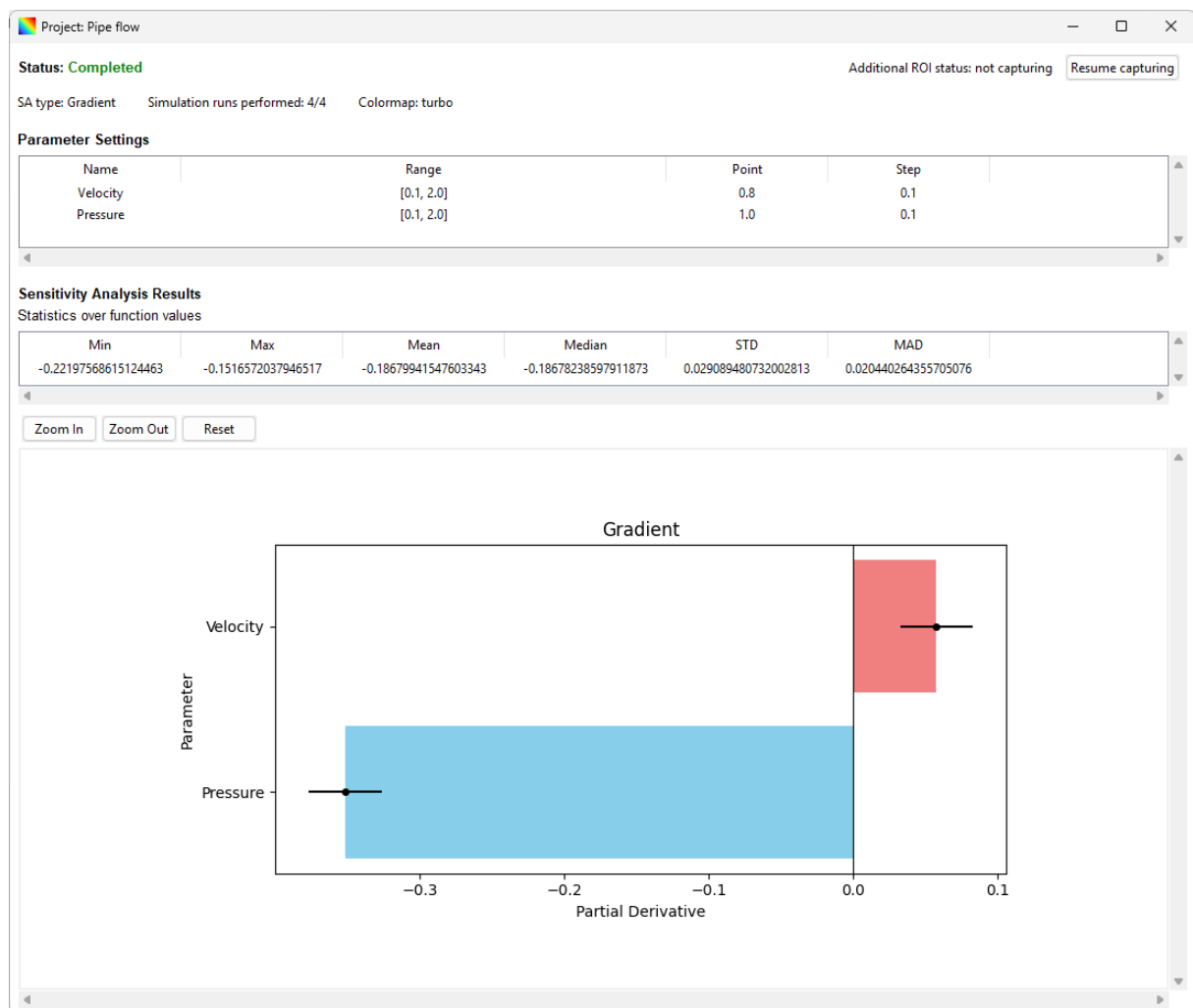


Figure 5.10: SimFlow SA Project Results

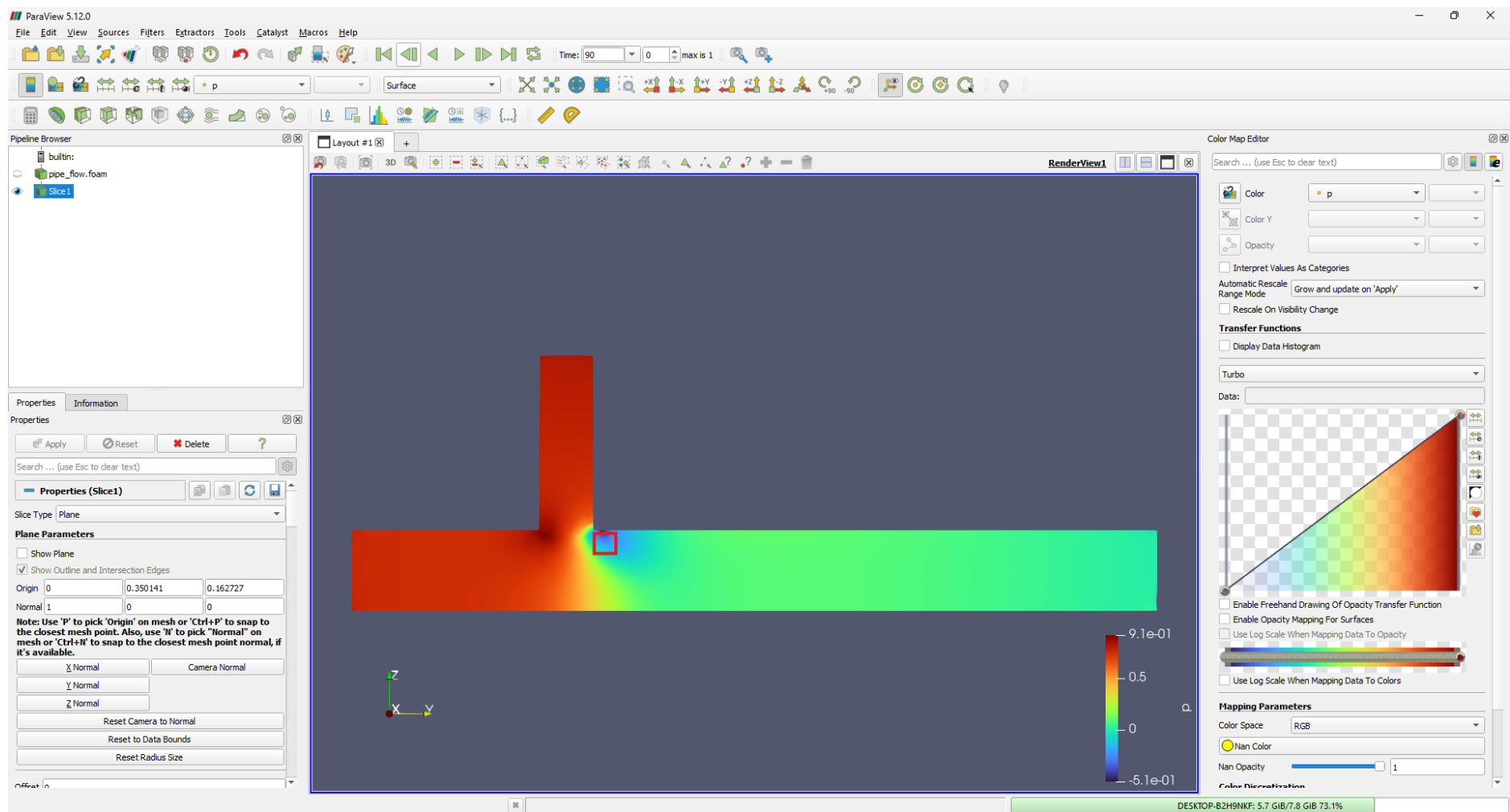


Figure 5.11: SimFlow Project — ROI Bounding Box

5.3.3 Ansys Electronics: Electromagnetic Brake (EMB)

Ansys Electronics is a massive closed-source industrial-grade software supporting electromagnetic, signal integrity, thermal and electromechanical simulations. Just as the previous software examples we considered, it has a rich, hierarchical, multi-window GUI with a huge number of various menus and input fields. The example project we built in this software for the purpose of our sensitivity analysis experiment was the model of an electromagnetic brake (the description of the project can be found in [65]). The Ansys Electronics GUI environment containing this project is shown in Figure 5.12.

The simulation of this project calculates the spatial distribution of the magnitude $\|\cdot\|_2$ of the magnetic flux density (vector $\mathbf{B}$) at the central cross-section of the EMB (Figure 5.13). In our SA project, we decided to estimate the sensitivity of $\|\mathbf{B}\|_2$ with respect to the air gap size between the EMB's base and its upper detached part. In contrast to SimFlow, Ansys Electronics possesses a highly functional simulation results visualizer. Thus, instead of routing data through an external application like ParaView, our application was guided to extract the ROIs directly from the color-coded images produced by Ansys. However, we were again unable to determine which exactly colormap Ansys uses as its default one. That is why we switched to the colormap our sensitivity automation application definitely supports — the grayscale one (Figure 5.14). The geometric boundary of the Region of Interest, in which we estimated the sensitivity, is outlined in Figure 5.15. Just as in the previous two experiments, the whole parametric sweep was successfully completed without problems. The SA project results are shown in Figure 5.16.

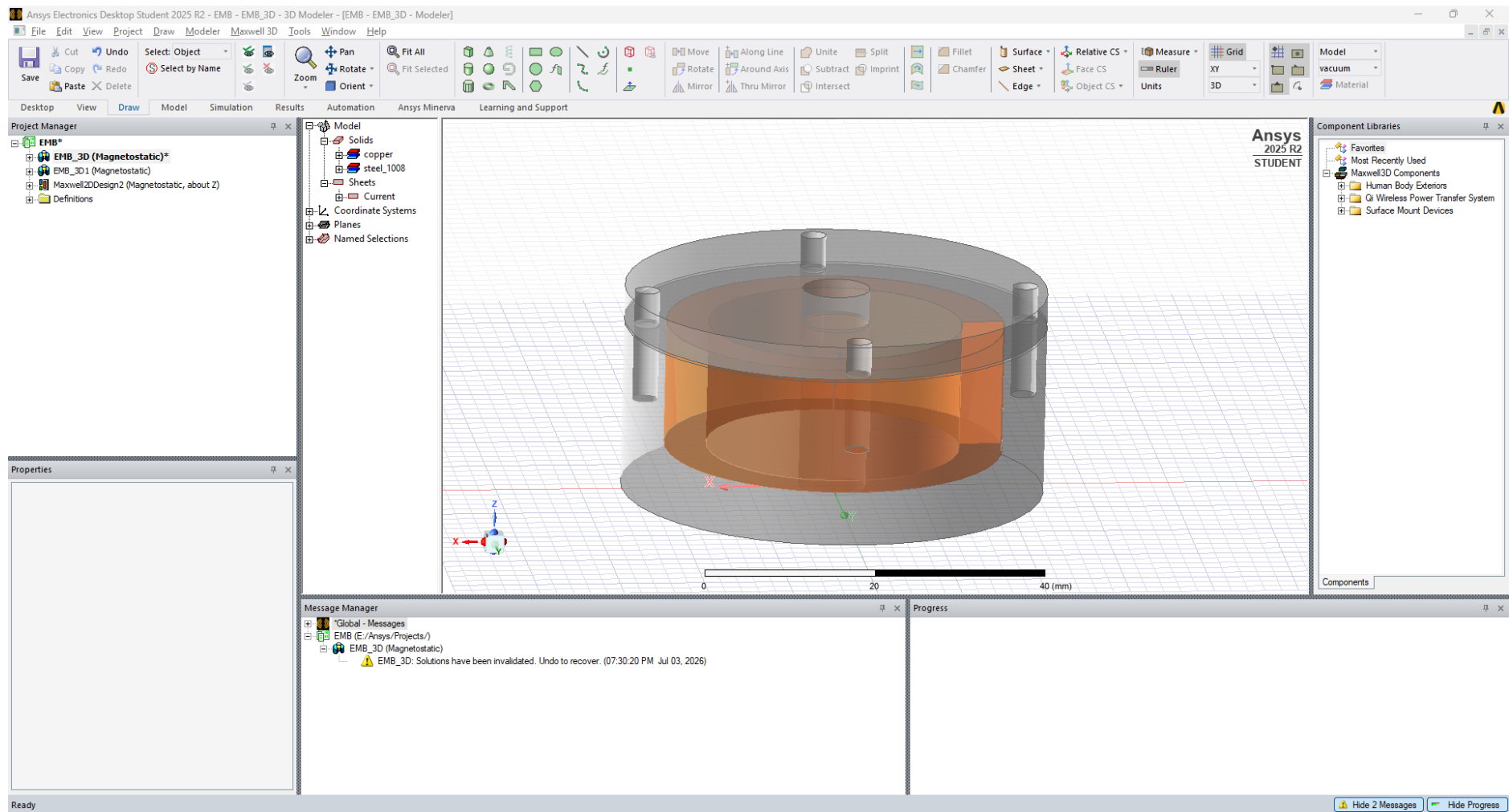


Figure 5.12: Electromagnetic Brake Project in Ansys Electronics

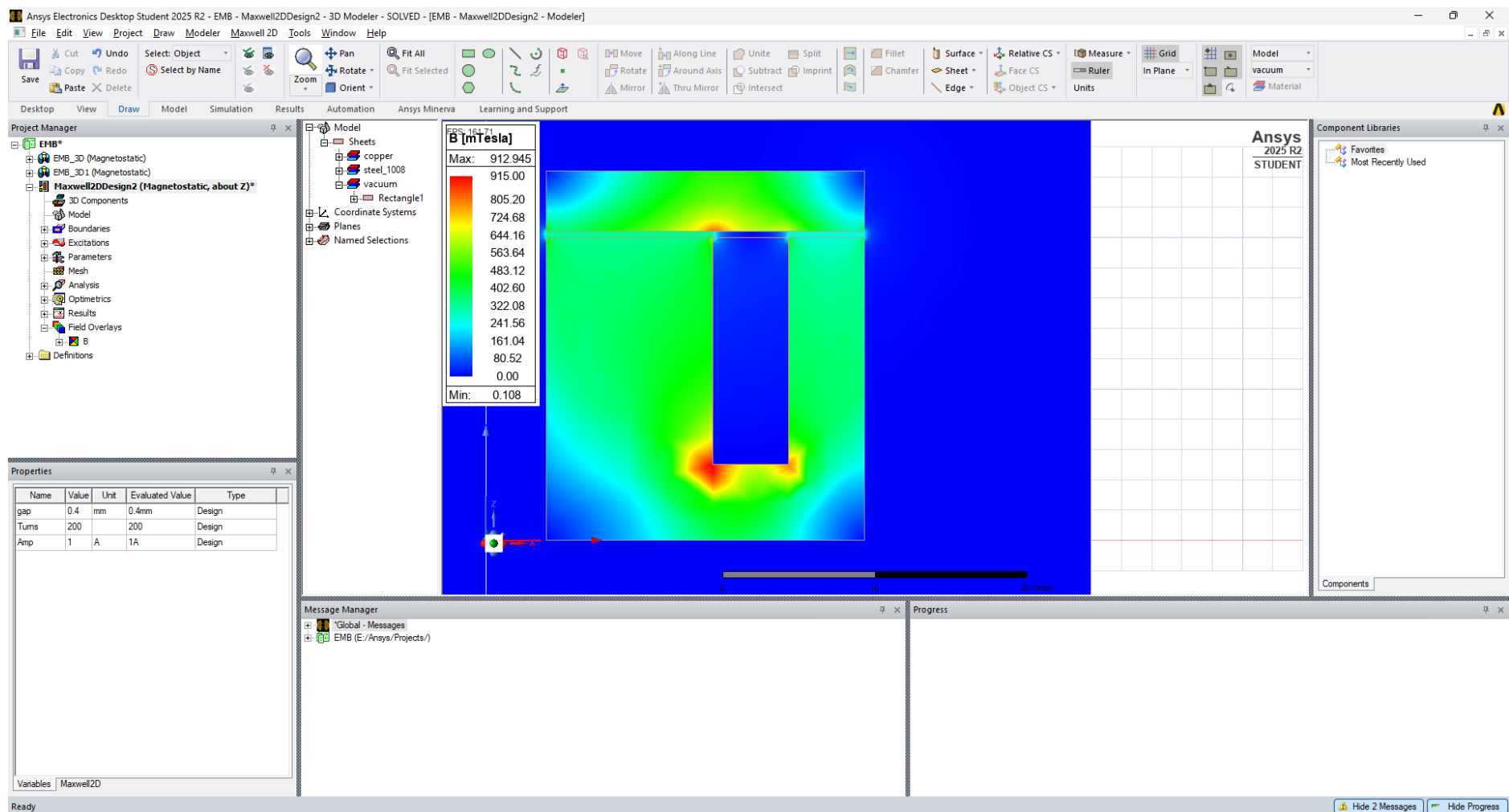


Figure 5.13: Ansys Electronics Project — Magnetic Flux Density Magnitude Heatmap (Central Cross-Section)

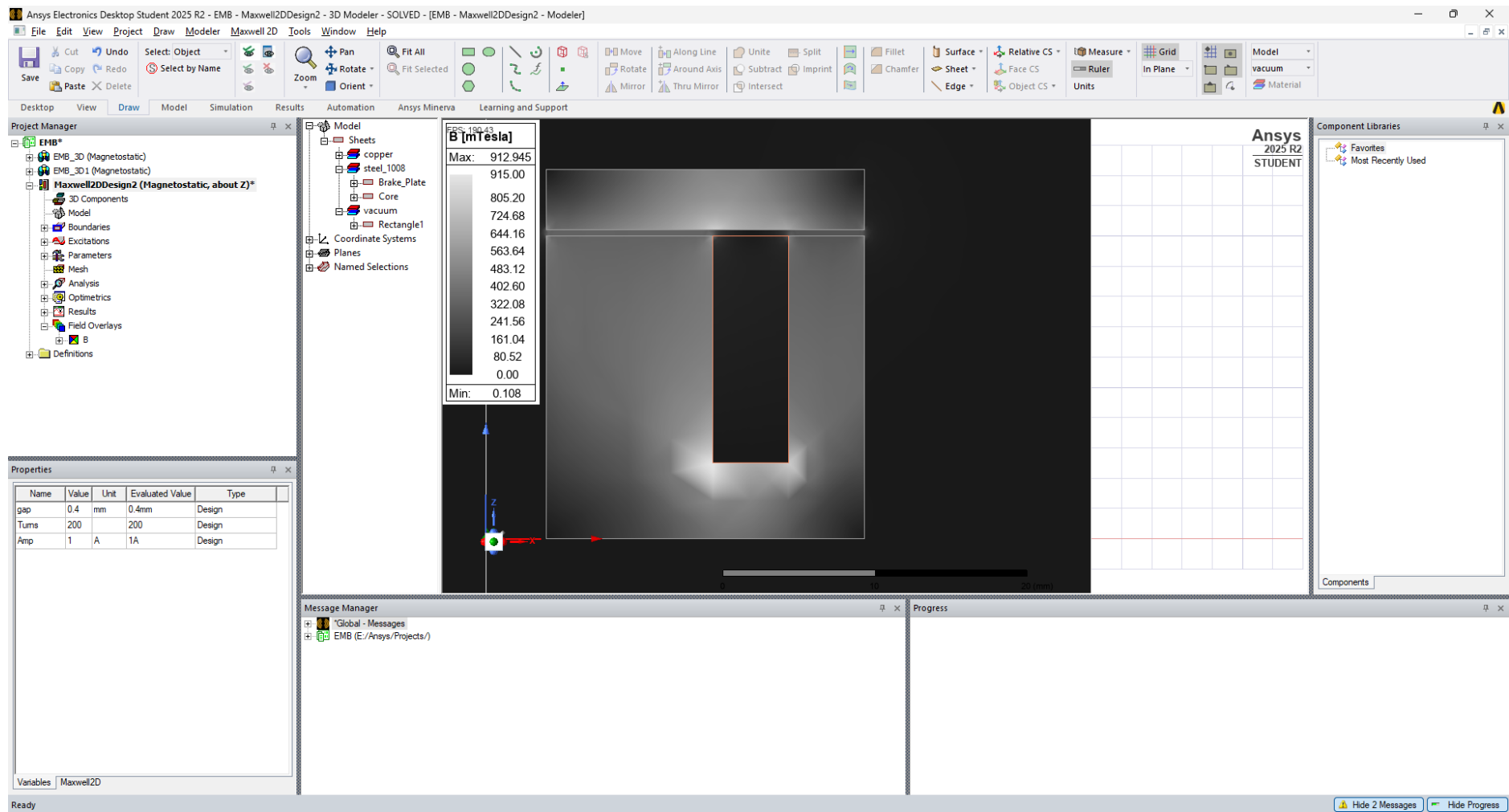


Figure 5.14: Ansys Electronics Project — Magnetic Flux Density Magnitude Grayscale Heatmap (Central Cross-Section)

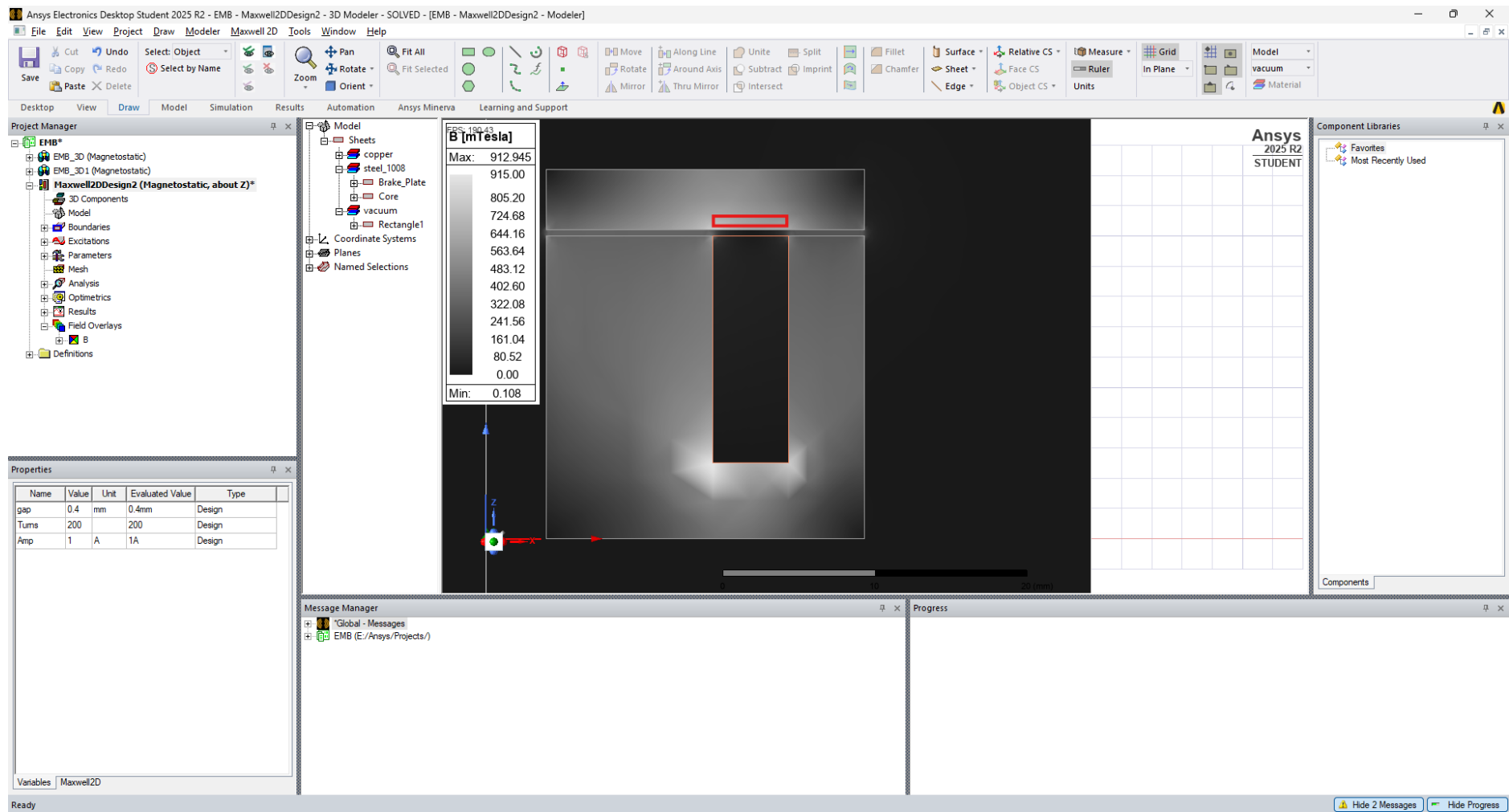


Figure 5.15: Ansys Electronics Project — ROI Bounding Box

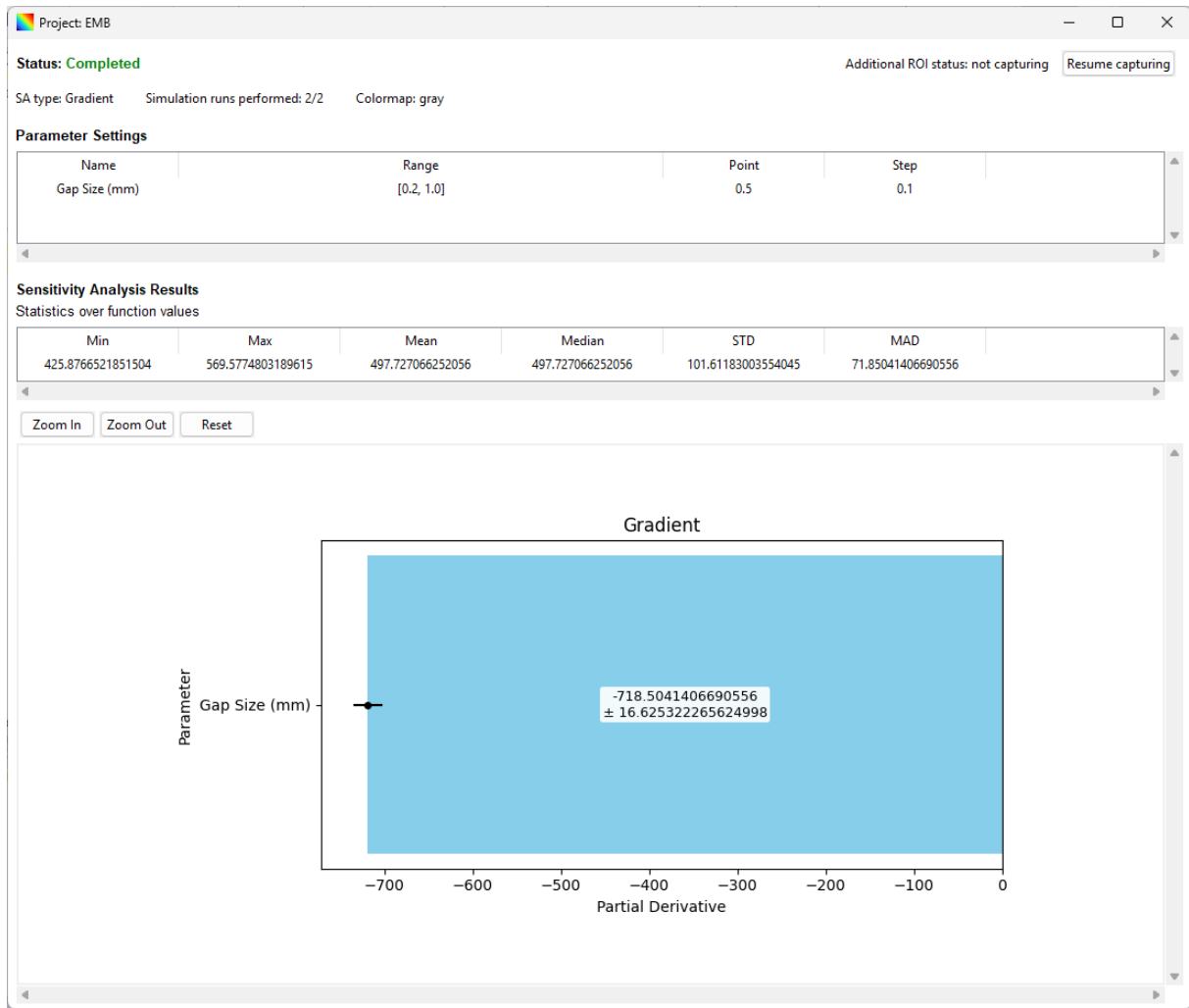


Figure 5.16: Ansys Electronics SA Project Results

5.4 Results

Having conducted extensive evaluation of our sensitivity automation application both on the synthetic benchmark and on multiple examples of real-world simulation software, we can confidently conclude that the application developed by us is capable of precisely estimating the sensitivity indices while successfully navigating arbitrarily complex graphical user interfaces of modern simulators and even switching to different applications (and navigating their GUIs as well!), for example, for color-coded visualization of the output data obtained from the simulation solver. In the following subsection, we would like to summarize the advantages and problems of our application, which became clear to us throughout its development and evaluation.

5.4.1 Advantages and Problems of Our Application

Let us begin with discussing the advantages of our sensitivity automation software. It has multiple of them, and they largely stem directly from the motivation of its development, formulated by us at the beginning of our work in the section 1.1:

1. **Complete Non-Intrusiveness:** Our application does not require absolutely any kind of low-level access to the target simulation software. Just like an average human user of such software, our application utilizes only the high-level GUI elements, being completely abstracted and unaware of any sort of implementation details and characteristics of a given simulator.
2. **Universal GUI and Workflow Length Support:** As a direct consequence of its complete non-intrusiveness, our application can work with any simulation software, which has a GUI. It is absolutely independent of the absence or presence of specific menus or control elements, their specific location, order of appearance throughout the dialog with the user, etc. In addition, our software supports arbitrarily long GUI workflows — no matter how many steps (GUI actions) it takes to configure a single simulation run, execute it, visualize and capture its output.
3. **Performance and Light-Weightiness:** Our application does not perform any kind of computation- or memory-intensive operations. It is a light-weight companion and orchestrator of simulation software, occupying little resources, so that the simulator can use most of them and not face any performance decreases.
4. **Failure Detection and Recoverability:** With the simulation finish detection mechanism in place, our application can understand if something went wrong and is preventing it from completing all the required simulation runs and suspend its execution until the user notices and resolves the issue. At the same time, instant saves of any new changes in the project state (new actions recorded by the user during the recording phase or a new simulation result obtained during the execution phase) help prevent any kind of data loss in the event of a failure and make our application able to recover its latest state and continue working from that state rather than from the complete beginning.

At the same time, as we were developing and evaluating our sensitivity analysis automation application, two its main groups of problems became clear. The first of them is related to the scalar reconstruction (colormap inversion) errors. The problems in that group are inherent not to our application, but rather to the colormaps and the simulation results visualization tools themselves. The second group of problems contains ones inherent to the basic RPA workflows which rely on fixed time delays between consecutive actions. Let us consider both of the limitation groups in order:

1. **Scalar Reconstruction Error.** There are multiple things that hurt the accuracy of the colormap inversion:
 - (a) Firstly and mainly, many popular colormaps (such as "Turbo", "Vidiris", "Inferno", etc.) have at most 256 unique colors (bins). For example, when using ParaView, we can specify a smaller number of bins to use when creating the color-coded image, and then a subset of the 256 colors will be used. However, we cannot make it use a number of bins larger than 256. This limitation in the maximum number of intervals a colormap domain may be split into directly affects our scalar reconstruction error

bounds: $[-\frac{W}{2}, \frac{W}{2}]$, where $W = \frac{f_{\max} - f_{\min}}{N}$ and $N = 256$ in our case. Once N is fixed at its maximum value of 256, any further tightening of the error bounds is only possible by decreasing the function value range $f_{\max} - f_{\min}$ of the scalar field $f(x, y)$ which is being visualized. This can be achieved by visualizing the $f(x, y)$ on a smaller rectangular subdomain of its original rectangular domain. From section 4.6, $f_{\min} = \min_{\substack{a \leq x \leq b \\ c \leq y \leq d}} f(x, y)$ and $f_{\max} = \max_{\substack{a \leq x \leq b \\ c \leq y \leq d}} f(x, y)$, where a, b, c, d are the screen coordinates defining the rectangular domain of the function $f(x, y)$: $a \leq x \leq b$ and $c \leq y \leq d$. Let us assume that we are interested only in the upper-right corner of the color-coded picture defined by coordinates $a_0 \leq x \leq b$ and $c_0 \leq y \leq d$, where $a \leq a_0$ and $c \leq c_0$. Then we can (try to) configure the simulation software to visualize only that corner. The number of bins N stays the same, but the numerator $f_{\max}^0 - f_{\min}^0$ is less than or equal to the original one $f_{\max} - f_{\min}$, because

$$f_{\min}^0 = \min_{\substack{a_0 \leq x \leq b \\ c_0 \leq y \leq d}} f(x, y) \geq f_{\min} \text{ and } f_{\max}^0 = \max_{\substack{a_0 \leq x \leq b \\ c_0 \leq y \leq d}} f(x, y) \leq f_{\max}.$$

This reduces the scalar reconstruction error. However, for that we have to sacrifice the size of the simulation output scalar field $f(x, y)$ displayed in the color-coded picture, which may not always be convenient and which may not always be possible to configure.

- (b) Secondly, there is a specific issue with some (not all) colormaps (e.g., "Viridis") occurring when displaying their colors on a digital PC screen. As we know, a colormap transforms scalars into RGB vectors. There is a small technical detail: it transforms them into RGB vectors having each of its three components being not an integer in the interval $[0, 255]$, but a floating-point number in the interval $[0, 1]$. To get corresponding integers, we just need to multiply each component by 255 and then either round or floor (throw away the decimal part). And here comes a problem (it is present for both rounding and flooring): some close (standing next to each other in the colormap color list) colors (RGB vectors), while being different in their floating-point representation, become perfectly equal in their integer representation. This means we now have duplicate colors and we are now unable to reconstruct precisely not just the scalar, but the bin in which this scalar lies. On the other hand, we know that the duplicate colors stand next to each other in the color list. As a result of this phenomenon, the initial (no duplicates) error bounds $[-\frac{W}{2}, \frac{W}{2}]$ become $[-\frac{W}{2} - (m-1)W, \frac{W}{2} + (m-1)W]$ if we pick a random bin of the ones the duplicate colors occupy. Here m is the number of subsequent bins the duplicate colors occupy, $m = 1$ means there are no duplicates. In the current first version of our MVP, our error whiskers in the gradient and Sobol index plots rendered in the Project Dashboard do not account for such potential color duplicates. On the other hand, throughout the evaluation phase we used "safe" colormaps which are proven to be duplicate-free: "Turbo" and the grayscale one — so all our error whiskers you have seen in this chapter are correct!
- (c) Thirdly, the simulation results visualization software (e.g., ParaView) can silently perform some graphical processing operations (interpolation, anti-aliasing, shading, etc.) which may alter the original colormap colors and introduce new ones not present in the colormap color list when rendering the final image. This may lead to the distance for a given RGB image pixel to its closest color in the colormap color list being greater

than zero: we cannot find exactly such color in the list, because it is a new, mutated one. In such a case our application will print a specific warning message to the console: *[WARNING] rgb_to_scalar: the exact matching color was not found, max(min distances) = D*

Here $D = \max_i \min_j ||\text{image pixel}_i - \text{colormap color}_j||_2$. So for each ROI pixel we look for the closest one in the colormap color list in terms of Euclidean distance and remember its distance and then we print the maximum of such minimal distances (because printing all of them would take a huge place).

If a user sees such a message in the console, she should try to fix the issue (disable all the graphical processing operations which may alter the original colormap colors), otherwise (if colors are distorted) we cannot guarantee that the closest-in-Euclidean-distance colormap color (and its corresponding bin) is the correct one for a given pixel. This, in turn, will lead to a similar error bounds inflation as in the case of colormap color duplicates described above.

2. Variable Action Delays and Unexpected Pop-Up Windows. Despite properly handling the unknown-in-advance variable simulation time and persisting the project state regularly in order to prevent the data loss if an error occurs, our application still has a few limitations regarding the correctness and safety of the recorded workflow execution:

- (a) First, currently it adaptively handles the variable time only of the solver's execution. What if some GUI window suddenly takes significantly longer to open? What if ParaView launches not in 10 seconds as usual, but in 40 seconds due to the Windows update suddenly started in parallel? In such cases our application workflow replay will be broken down, because for such events (intermediate GUI actions) it still relies on the exact same time delays between the consecutive actions as it has logged during the workflow recording phase. Here we would like to notice that, although the problem is definitely there, it is not something critically bad (especially as our goal was to develop an MVP and not the complete polished software) — all the workflows with different simulation software which we have executed during our large Evaluation phase have run without any problems or interruptions! Our PC was not overloaded during the simulation runs and we have recorded each GUI action with a few seconds more delay than it usually takes.
- (b) The second limitation of our currently-implemented workflow execution is related to the case when a suddenly popped-up GUI window (e.g., some temporary-visible notification or a complete permanent focus stealing performed by another running application) occludes the simulation software's buttons and text input fields. In such a case, of course, our SA automation application will suspend its parametric sweep as soon as it hits the first simulation run timeout (the SCI will not appear in the allotted time). However, it will still perform some mouse clicks and keyboard button presses before that happens. This may have arbitrary (including bad) consequences if it accidentally clicks on some important button in the popped-up GUI window. This is why before running a potentially lengthy parametric sweep with our application

it is recommended to ensure that it will not be interrupted by any kind of sudden interference from the side of another running applications and/or OS.

As one can see, our application possesses several important structural advantages which enable it to successfully orchestrate modern real-world industrial-grade simulation software and calculate and present correct sensitivity indices estimates. On the other hand, there are also some problems and limitations our MVP currently has. However, what is important to mention here is that not a single identified problem is a critical one. In addition, every of them has its mitigation and/or workaround. Furthermore, in the next chapter, additional mitigation and/or complete fixes are proposed for some of the problems.

Chapter 6

Conclusion and Future Work

In modern computational science and engineering, numerical simulation software plays a critical role in modeling physical systems, enabling virtual prototyping, and reducing the reliance on expensive physical experiments. Sensitivity analysis is a very important tool in this context, allowing researchers and engineers to quantify how variations in input parameters influence model outputs, identify key drivers of uncertainty, and optimize system designs. However, when simulation software is distributed as closed-source binaries encapsulated within Graphical User Interfaces — lacking source code access, application programming interfaces, or command-line pathways and producing the simulation output mainly in the form of color-coded visualizations — performing sensitivity analysis quickly and in an automated fashion becomes a challenge. Standard SA workflows often require hundreds to thousands of model evaluations, which is infeasible to perform via manual GUI interaction.

To address this challenge, we conducted extensive research into automating GUI-based sensitivity analysis from different angles and perspectives. We developed a specialized software capable of performing sensitivity analysis in an automated and completely non-intrusive manner: by interacting with the simulation software’s GUI and extracting its output directly from the color-coded images without the need for any kind of low-level access. The source code of our software is available under the MIT license [66] at <https://github.com/alekseilemberrwth/gui-sa-automation>. Our implementation was extensively evaluated across both synthetic mathematical benchmarks with known true sensitivities and diverse real-world engineering tools, including ElmerFEM, SimFlow, and Ansys Electronics. The results demonstrate that our application performs quite well: it successfully navigates highly-complex GUIs of modern simulation software and delivers reliable local gradient and global variance-based Sobol indices estimates.

At the same time, the software developed in this thesis represents a Minimum Viable Product. While it successfully proves the feasibility and utility of GUI-based SA automation, it also opens several potential directions for future work and research regarding its capabilities, precision, and resilience:

- **Support for Additional GUI Input Types:** Currently, our application supports automatic parameter injection only for standard text input fields. Future iterations can expand this to other GUI input elements such as drop-down lists, checkboxes, slider bars,

etc.

- **Integration of Another Sensitivity Analysis Methods:** The current analytical engine of our application can calculate gradients and Sobol indices. Integrating additional sensitivity analysis methodologies such as the Morris Method [67] or Fourier Amplitude Sensitivity Test [68] (both of which are supported by `SALib`) would provide users with broader analytical flexibility.
- **Gradient Calculation via Iterative Algorithms:** Currently, we calculate the gradient estimates using a simple second-order central finite difference scheme with a fixed step size, which should be specified by the user. A valuable direction for future work is incorporating adaptive numerical differentiation algorithms, such as those implemented in SciPy [69] (`scipy.diffentiate.derivative` [70]). These methods iteratively refine step sizes and compare derivative estimates until they reach convergence (no significant derivative change when changing the step size); however, this involves an inherent trade-off: achieving higher accuracy in partial derivative estimates (through using the optimal step size) requires additional simulation runs, increasing the total computational time.
- **Enhanced Error Bounds Accounting for Colormap Color Duplicates:** In section 5.4 we described a scenario when a colormap can end up having multiple bins with the exact same associated color. In such cases, the naive error bounds for a single bin, which our MVP currently calculates, do not apply. Future work should make our application account for the duplicates structure of specific colormaps (which exactly colors are duplicated in which of them) and always report the correct error bounds.
- **Extending Simulation Completion Indicators:** Currently, the logic which waits for a remembered visual template to appear on the screen in order to determine that some event has happened is used only to detect the completion of the simulation run. We can extend this logic to dynamically detect occurrence (or non-occurrence after a timeout, in which case we will understand that something went wrong) of a much broader class of events. For example, we can detect that a specific GUI window of our simulator has appeared on the screen and is ready for our input, or that the ParaView application has successfully launched. This will eliminate the need for unreliable fixed pre-specified time delays that we currently use.
- **Post-Processing Tools with High-Resolution Colormaps:** A primary constraint on scalar reconstruction accuracy is the 256-bin limit inherent to standard discrete colormaps in common visualization suites (e.g., ParaView). Further research into alternative post-processing engines that support high-resolution colormaps with significantly more than 256 bins could help us understand whether there is an opportunity for us to increase the accuracy of the reconstructed scalar values and thereby sharpen the resulting sensitivity estimates.
- **Coupling RPA with Agentic AI Automation:** While programmatic Robotic Process Automation provides exceptional execution speed and precision of the actions, it lacks

flexibility when unexpected events occur. Future work should investigate coupling our RPA application with Vision-Language AI automation tools [71–73]. In such a hybrid setup, if our application detects an abnormal state or error during its parametric sweep, it could suspend its execution and query an AI agent, which would analyze the screen, temporarily take control of the workstation to resolve the problem, and then return the control back to our application so it can resume running simulations.

Addressing these further work and research opportunities may significantly improve our MVP and turn it into a fully-fledged sensitivity analysis automation software.

Bibliography

- [1] Sharon C Glotzer. *International Assessment of Research and Development in Simulation-Based Engineering and Science*. IMPERIAL COLLEGE PRESS, 2011.
- [2] Ulrich Rüde, Karen Willcox, Lois Curfman McInnes, and Hans De Sterck. Research and education in computational science and engineering. *SIAM Review*, 60(3):707–754, 2018.
- [3] Liang Zhou and Charles D. Hansen. A survey of colormaps in visualization. *IEEE Transactions on Visualization and Computer Graphics*, 22(8):2051–2069, 2016.
- [4] Kenneth Moreland. Diverging color maps for scientific visualization. In George Bebis, Richard Boyle, Bahram Parvin, Darko Koracin, Yoshinori Kuno, Junxian Wang, Renato Pajarola, Peter Lindstrom, André Hinkenjann, Miguel L. Encarnação, Cláudio T. Silva, and Daniel Coming, editors, *Advances in Visual Computing*, pages 92–103, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.
- [5] Fabio Crameri, Grace E. Shephard, and Philip J. Heron. Choosing suitable color palettes for accessible and accurate science figures. *Current Protocols*, 4(8):e1126, 2024.
- [6] Andrea Saltelli, M. Ratto, Terry Andres, Francesca Campolongo, Jessica Cariboni, Debora Gatelli, Michaela Saisana, and Stefano Tarantola. *Global Sensitivity Analysis. The Primer*, volume 304. 01 2008.
- [7] Francesca Pianosi, Keith Beven, Jim Freer, Jim W. Hall, Jonathan Rougier, David B. Stephenson, and Thorsten Wagener. Sensitivity analysis of environmental models: A systematic review with practical workflow. *Environmental Modelling & Software*, 79:214–232, 2016.
- [8] Bertrand Iooss and Paul Lemaître. *A Review on Global Sensitivity Analysis Methods*, pages 101–122. Springer US, Boston, MA, 2015.
- [9] I.M Sobol’. Global sensitivity indices for nonlinear mathematical models and their monte carlo estimates. *Mathematics and Computers in Simulation*, 55(1):271–280, 2001. The Second IMACS Seminar on Monte Carlo Methods.
- [10] Valentina Lenarduzzi and Davide Taibi. Mvp explained: A systematic mapping study on the definitions of minimal viable product. In *2016 42th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 112–119, 2016.

- [11] Dobrila Rancic Moogk. Minimum viable product and the importance of experimentation in technology startups. *Technology Innovation Management Review*, 2:23–26, 03/2012 2012.
- [12] *Calculating Derivatives*, pages 193–219. Springer New York, New York, NY, 2006.
- [13] Walter Rudin. *Real and complex analysis, 3rd ed.* McGraw-Hill, Inc., USA, 1987.
- [14] Andrea Saltelli, Paola Annoni, Ivano Azzini, Francesca Campolongo, Marco Ratto, and Stefano Tarantola. Variance based sensitivity analysis of model output. design and estimator for the total sensitivity index. *Computer Physics Communications*, 181(2):259–270, 2010.
- [15] Andrea Saltelli. Making best use of model evaluations to compute sensitivity indices. *Computer Physics Communications*, 145(2):280–297, 2002.
- [16] SeleniumHQ. Selenium: A browser automation tool. <https://www.selenium.dev/>. Accessed: 2026-07-20.
- [17] Google LLC. Puppeteer: A javascript library which provides a high-level api to control chrome or firefox. <https://pptr.dev/>. Accessed: 2026-07-20.
- [18] Microsoft Corporation. Playwright: Fast and reliable end-to-end testing for modern web apps. <https://playwright.dev/>. Accessed: 2026-07-20.
- [19] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: benchmarking multimodal agents for open-ended tasks in real computer environments. In *Proceedings of the 38th International Conference on Neural Information Processing Systems, NIPS '24*, Red Hook, NY, USA, 2024. Curran Associates Inc.
- [20] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, Lawrence Keunho Jang, and Zheng Hui. Windows agent arena: Evaluating multi-modal OS agents at scale. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 4874–4910. PMLR, 13–19 Jul 2025.
- [21] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *Proceedings of the 33rd ACM International Conference on Multimedia, MM '25*, page 8778–8786, New York, NY, USA, 2025. Association for Computing Machinery.
- [22] Rehan Syed, Suriadi Suriadi, Michael Adams, Wasana Bandara, Sander J.J. Leemans, Chun Ouyang, Arthur H.M. ter Hofstede, Inge van de Weerd, Moe Thandar Wynn, and Hajo A. Reijers. Robotic process automation: Contemporary themes and challenges. *Computers in Industry*, 115:103162, 2020.

- [23] Lucija Ivančić, Dalia Suša Vugec, and Vesna Bosilj Vukšić. Robotic process automation: Systematic literature review. In Claudio Di Ciccio, Renata Gabryelczyk, Luciano García-Bañuelos, Tomislav Hernaus, Rick Hull, Mojca Indihar Štemberger, Andrea Kő, and Mark Staples, editors, *Business Process Management: Blockchain and Central and Eastern Europe Forum*, pages 280–295, Cham, 2019. Springer International Publishing.
- [24] Uwe Naumann. *The Art of Differentiating Computer Programs*. Society for Industrial and Applied Mathematics, 2011.
- [25] Johannes Lotz, Klaus Leppkes, and Uwe Naumann. dco/c++ – derivative code by overloading in c++. Aachener Informatik-Berichte AIB-2011-06, RWTH Aachen University, 2011.
- [26] Iso/iec/ieee international standard - systems and software engineering – software life cycle processes. *ISO/IEC/IEEE 12207:2026(en)*, pages 1–154, 2026.
- [27] Iso/iec/ieee international standard - systems and software engineering – life cycle processes – requirements engineering. *ISO/IEC/IEEE 29148:2018(E)*, pages 1–104, 2018.
- [28] Karl E Wieggers and Joy Beatty. *Software Requirements 3*. Microsoft Press, USA, 2013.
- [29] Matplotlib Contributors. Colormap reference. https://matplotlib.org/stable/gallery/color/colormap_reference.html, 2026. Matplotlib 3.11 documentation, accessed 2026-08-09.
- [30] D. L. Parnas. On the criteria to be used in decomposing systems into modules. *Commun. ACM*, 15(12):1053–1058, December 1972.
- [31] Grady Booch, Robert Maksimchuk, Michael Engle, Bobbi Young, Jim Conallen, and Kelli Houston. *Object-oriented analysis and design with applications, third edition*. Addison-Wesley Professional, third edition, 2007.
- [32] Python Software Foundation. Python. <https://www.python.org/>, 2026. Accessed: 2026-08-10.
- [33] Python Software Foundation. Tkinter — python interface to tcl/tk. <https://docs.python.org/3/library/tkinter.html>, 2026. Accessed: 2026-08-10.
- [34] Pynput Contributors. Pynput package documentation. <https://pynput.readthedocs.io/>, 2026. Accessed: 2026-08-10.
- [35] Mss Contributors. Mss: An ultra fast cross-platform multiple screenshots module in pure python using ctypes. <https://python-mss.readthedocs.io/>, 2026. Accessed: 2026-08-10.
- [36] Pillow Contributors. Pillow (pil fork) documentation — imagegrab module. <https://pillow.readthedocs.io/en/stable/reference/ImageGrab.html>, 2026. Accessed: 2026-08-10.
- [37] rjpp1998. Imagegrab.grab() method is too slow. Stack Overflow, 2017. Question asked May 23, 2017.

- [38] Monty. Imagegrab.grab() method is too slow but alternatives don't return the rgb colour. Stack Overflow, 2022. Question asked March 23, 2022.
- [39] NumPy Contributors. Numpy: The fundamental package for scientific computing with python. <https://numpy.org/>, 2026. Accessed: 2026-08-10.
- [40] Matplotlib Contributors. Matplotlib: Visualization with python. <https://matplotlib.org/>, 2026. Accessed: 2026-08-10.
- [41] SALib Contributors. Salib: Sensitivity analysis library in python. <https://salib.readthedocs.io/>, 2026. Accessed: 2026-08-10.
- [42] OpenCV Contributors. Opencv: The world's biggest computer vision library. <https://opencv.org/>, 2026. Accessed: 2026-08-10.
- [43] Pynput Contributors. Pynput.mouse.listener — monitoring the mouse. <https://pynput.readthedocs.io/en/latest/mouse.html#monitoring-the-mouse>, 2026. Accessed: 2026-08-10.
- [44] Pynput Contributors. Pynput.keyboard.listener — monitoring the keyboard. <https://pynput.readthedocs.io/en/latest/keyboard.html#monitoring-the-keyboard>, 2026. Accessed: 2026-08-10.
- [45] Pynput Contributors. Pynput.mouse.controller — controlling the mouse. <https://pynput.readthedocs.io/en/latest/mouse.html#controlling-the-mouse>, 2026. Accessed: 2026-08-10.
- [46] Pynput Contributors. Pynput.keyboard.controller — controlling the keyboard. <https://pynput.readthedocs.io/en/latest/keyboard.html#controlling-the-keyboard>, 2026. Accessed: 2026-08-10.
- [47] Matplotlib Contributors. Matplotlib.colors.listedcolormap — matplotlib api reference. https://matplotlib.org/stable/api/_as_gen/matplotlib.colors.ListedColormap.html. Accessed: 2026-07-18.
- [48] Matplotlib Contributors. Matplotlib listed colormaps raw color arrays. https://github.com/matplotlib/matplotlib/blob/main/lib/matplotlib/_cm_listed.py, 2026. Accessed: 2026-08-10.
- [49] Pyperclip Contributors. Pyperclip: A cross-platform python module for copying and pasting text to the clipboard. <https://pyperclip.readthedocs.io/>, 2026. Accessed: 2026-08-10.
- [50] SALib Contributors. Salib.sample.saltelli.sample — salib documentation. <https://salib.readthedocs.io/en/latest/api/SALib.sample.html#SALib.sample.saltelli.sample>, 2026. Accessed: 2026-08-10.
- [51] SALib Contributors. Salib.analyze.sobol.analyze — salib documentation. <https://salib.readthedocs.io/en/latest/api/SALib.analyze.html#SALib.analyze.sobol.analyze>, 2026. Accessed: 2026-08-10.

- [52] NumPy Contributors. Numpy.save — numpy api reference. <https://numpy.org/doc/stable/reference/generated/numpy.save.html>, 2026. Accessed: 2026-08-10.
- [53] T. Ishigami and T. Homma. An importance quantification technique in uncertainty analysis for computer models. In *[1990] Proceedings. First International Symposium on Uncertainty Modeling and Analysis*, pages 398–403, 1990.
- [54] SALib Contributors. Salib documentation: Basic usage — an example. https://salib.readthedocs.io/en/latest/user_guide/basics.html#an-example, 2026. Accessed: 2026-08-11.
- [55] Francesca Pianosi and Thorsten Wagener. A simple and efficient method for global sensitivity analysis based on cumulative distribution functions. *Environmental Modelling & Software*, 67:1–11, 2015.
- [56] Ivano Azzini and Rossana Rosati. Sobol’ main effect index: an innovative algorithm (ia) using dynamic adaptive variances. *Reliability Engineering & System Safety*, 213:107647, 2021.
- [57] SALib Contributors. Salib.test_functions.ishigami module — ishigami function. https://salib.readthedocs.io/en/latest/api/SALib.test_functions.html#SALib.test_functions.Ishigami.evaluate, 2026. Accessed: 2026-08-11.
- [58] Kitware Inc. Paraview: The world’s leading open source post-processing visualization engine. <https://www.paraview.org>, 2026. Accessed: 2026-08-11.
- [59] CSC IT Center for Science Ltd. Elmerfem – open source multiphysical simulation software. <https://www.elmerfem.org/blog/>, 2026. Accessed: 2026-07-19.
- [60] SimFlow Technologies. Simflow — cfd software made easy. <https://sim-flow.com>, 2026. Accessed: 2026-07-19.
- [61] Inc. Synopsys and Inc. ANSYS. Ansys electronics desktop student. <https://ansys.synopsys.com/academic/students/ansys-electronics-desktop-student>, 2026. Accessed: 2026-07-19.
- [62] CSC IT Center for Science. Elmer tutorials. <https://www.nic.funet.fi/pub/sci/physics/elmer/doc/ElmerTutorials.pdf>, 2026. Accessed: 2026-08-11.
- [63] Jesús Ildefonso Díaz and Jesús Hernández. Beyond the classical strong maximum principle: Sign-changing forcing term and flat solutions. *Advances in Nonlinear Analysis*, 13(1):20230128, 2024.
- [64] SimFlow Technologies. Simflow tutorials: Internal pipe flow — cfd simulation. <https://help.sim-flow.com/tutorials/pipe-flow>, 2026. Accessed: 2026-08-11.
- [65] Inc. Synopsys and Inc. ANSYS. Ansys learning: Intro to ansys maxwell – magneto-static simulation – ansys innovation course. <https://www.youtube.com/playlist?list=PLtt6-ZgUFmMK0wGV0OK8pArvIuIcV1-tF>, 2026. Accessed: 2026-08-11.

- [66] Open Source Initiative. The mit license. <https://opensource.org/license/mit>, 2026. Accessed: 2026-08-12.
- [67] Max D. Morris. Factorial sampling plans for preliminary computational experiments. *Technometrics*, 33(2):161–174, 1991.
- [68] A. Saltelli, S. Tarantola, and K. P.-S. Chan. A quantitative model-independent method for global sensitivity analysis of model output. *Technometrics*, 41(1):39–56, 1999.
- [69] SciPy Contributors. Scipy: Fundamental algorithms for scientific computing in python. <https://scipy.org>, 2026. Accessed: 2026-08-11.
- [70] SciPy Contributors. Scipy.differentiate.derivative — scipy v1.18 manual. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.differentiate.derivative.html#scipy.differentiate.derivative>, 2026. Accessed: 2026-08-11.
- [71] Minghao Chen, Xinyi Hu, Zhou Yu, and Yufei Yin. Autorpa: Efficient gui automation through llm-driven code synthesis from interactions, 2026.
- [72] Osama Hosam Abdellaif, Abdelrahman Nader, and Ali Hamdi. Lmrpa: Large language model-driven efficient robotic process automation for ocr, 2025.
- [73] Osama Abdellatif, Ahmed Ayman, and Ali Hamdi. Lmv-rpa: Large model voting-based robotic process automation, 2025.